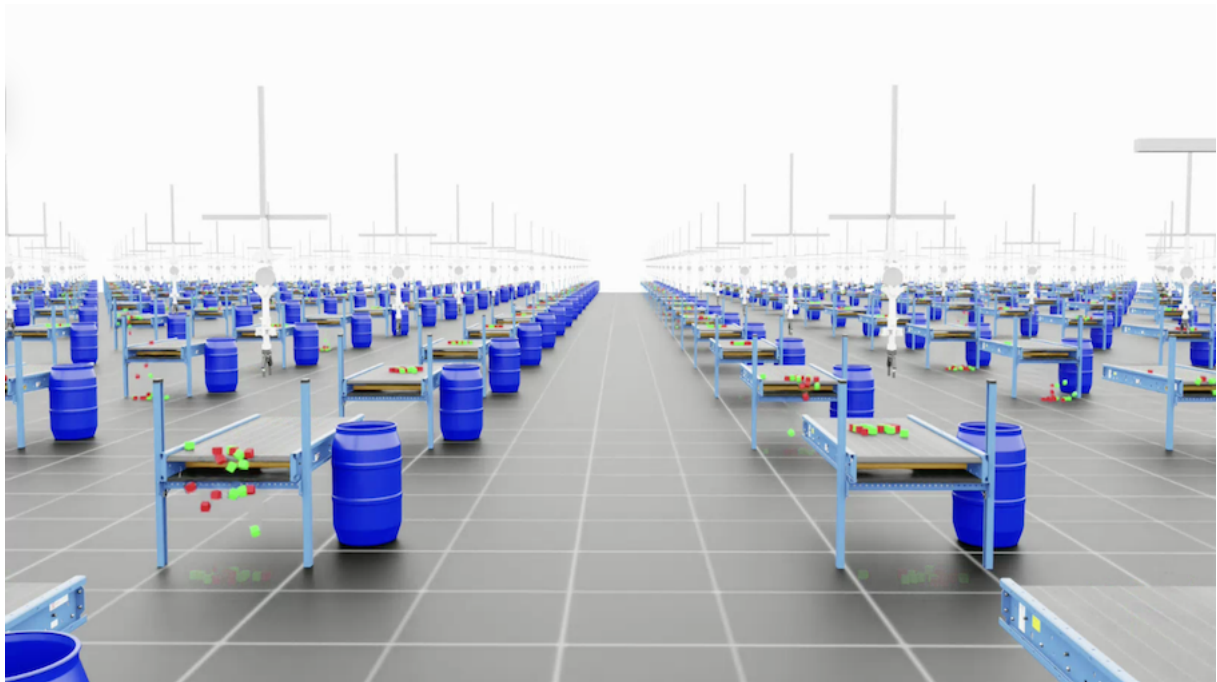


Learning-Based Manipulation for Textile Sorting: Sim-to-Real Training of a Tendon-Driven Robot Using Isaac Lab and ROS 2

Project Thesis im Studiengang Computational Engineering M. Sc.

Friedrich-Alexander-Universität Erlangen-Nürnberg
Lehrstuhl für Fertigungsautomatisierung und Produktionssystematik
Prof. Dr.-Ing. J. Franke



Bearbeiter: Georg Meyer

Matrikelnr.: 22791103

Betreuer: Prof. Dr.-Ing. J. Franke
Dipl.-Ing. M. Landgraf

Abgabetermin: 01.07.2026

Bearbeitungszeit: x Monate

Erklärung

Ich versichere, dass ich die vorliegende Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Erlangen, den 5. März 2026

Georg Meyer

Contents

List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
List of Symbols.	x
-1 Guideline	1
-1.1 Format and Style	1
-1.1.1 Page Layout and Typography	1
-1.1.2 Page Numbering and Binding	1
-1.1.3 Section and Chapter Formatting	1
-1.1.4 Paragraphs, Spacing, and Whitespace	2
-1.1.5 Figures	2
-1.1.6 Tables	3
-1.1.7 Diagrams	3
-1.1.8 Formulas	3
-1.1.9 Appendix	4
-1.1.10 Eidesstattliche Erklärung (Statutory Declaration).	4
-1.2 Thesis Structure and Content	4
-1.2.1 Required Document Order.	4
-1.2.2 Page Count Guidelines	5
-1.2.3 Red Thread (Roter Faden)	5
-1.2.4 Chapter: Einleitung und Zielstellung (Introduction).	5
-1.2.5 Chapter: Grundlagen und Stand der Technik (Background and State of the Art)	6
-1.2.6 Chapter: Handlungsbedarf (Research Gap)	6
-1.2.7 Chapter: Methodenteil (Methodology)	7
-1.2.8 Chapter: Ergebnisteil (Results)	7
-1.2.9 Chapter: Diskussionsteil (Discussion)	7
-1.2.10 Chapter: Zusammenfassung und Ausblick (Summary and Outlook)	7
-1.3 Scientific Writing Style.	8
-1.3.1 General Principles	8
-1.3.2 Sentence Construction	8

-1.3.3 Grammar	8
-1.3.4 Technical Language.	9
-1.3.5 Expression and Phrasing	9
-1.3.6 Content Guidelines	9
-1.3.7 Introductions and Transitions Between Sections	10
-1.3.8 Consistency and Formatting Rules	10
-1.4 Citations and Bibliography	11
-1.4.1 Citation Style	11
-1.4.2 Citation Placement	11
-1.4.3 Direct Quotes	11
-1.4.4 Indirect Quotes	11
-1.4.5 Secondary Citations	11
-1.4.6 Plagiarism	12
-1.4.7 Source Quality Requirements	12
-1.4.8 Electronic Sources	12
-1.5 Literature Research	13
-1.6 Figures and Graphics — Additional Rules	13
-1.7 Review Checklist Before Submission	13
-1.7.1 Content Review	13
-1.7.2 Language and Expression Review.	14
-1.7.3 Visual and Formatting Review	14
-1.8 Grading Criteria	14
-1.9 Presentation Guidelines (BA/PA)	15
0 Chapter Outline	16
0.1 Chapter 1: Introduction (5 pages).	16
0.1.1 Motivation and Context.	16
0.1.2 Problem Statement and Objectives	16
0.1.3 Thesis Outline	17
0.2 Chapter 2: Background and State of the Art (20 pages)	17
0.2.1 Section 2.1: Reinforcement Learning for Robotic Manipulation	17
0.2.2 Section 2.2: Physics Simulation in Isaac Sim 5.1	17
0.2.3 Section 2.3: NVIDIA Isaac Sim and IsaacLab Architecture	19
0.2.4 Section 2.4: Tendon-Driven Robot Kinematics and Modeling	20
0.2.5 Section 2.5: Policy Learning Pipeline (MDP Workflow)	21
0.2.6 Section 2.6: Prior Work in Robotic Cloth Manipulation	21

0.2.7	Section 2.7: Robot and Scene Description Formats (USD, URDF, MJCF)	22
0.3	Chapter 3: Problem Statement and Research Gap (3 pages)	22
0.3.1	Section 3.1: Critical Analysis of State of the Art	22
0.3.2	Section 3.2: Research Gap and Justification	23
0.3.3	Section 3.3: Refined Problem Definition	23
0.3.4	Section 3.4: Success Criteria (Optional)	23
0.4	Chapter 4: Methodology and Implementation (15–18 pages)	24
0.4.1	Section 4.1: Overview of Approach	24
0.4.2	Section 4.2: Simulation Environment Setup	24
0.4.3	Section 4.3: Robot Model Integration (Tendon-Driven Arm)	24
0.4.4	Section 4.4: Reinforcement Learning Setup.	26
0.4.5	Section 4.5: Initial Experiments (Basic Task Policies)	27
0.4.6	Section 4.6: Advanced Task Implementation (Cloth Grasping and Stretching)	27
0.4.7	Section 4.7: Software and Hardware Integration (ROS 2).	27
0.5	Chapter 5: Results (15 pages)	28
0.5.1	Section 5.1: Preliminary Experiments (Baseline Results)	28
0.5.2	Section 5.2: Performance of Tendon-Driven Robot on Simple Tasks	28
0.5.3	Section 5.3: Cloth Manipulation Task Results	29
0.5.4	Section 5.4: Summary of Key Results (Optional)	30
0.6	Chapter 6: Discussion (5 pages)	30
0.6.1	Section 6.1: Evaluation of Outcomes	30
0.6.2	Section 6.2: Comparison with Related Work	30
0.6.3	Section 6.3: Limitations of the Study	31
0.6.4	Section 6.4: Practical Implications	31
0.6.5	Section 6.5: Recommendations and Lessons Learned.	31
0.7	Chapter 7: Conclusion and Outlook (3 pages).	32
0.7.1	Section 7.1: Conclusion (Summary of Findings).	32
0.7.2	Section 7.2: Outlook (Future Work).	32
0.8	Appendices (not counted in page total).	33
1	Introduction	34
1.1	Motivation and Context	34
1.2	Problem Statement and Objectives	34
1.3	Thesis Outline	36

2	Background	37
2.1	Reinforcement Learning for Robotic Manipulation	37
2.2	Physics Simulation in Isaac Sim 5.1	41
2.2.1	Rigid Body Dynamics.	41
2.2.2	Cloth and Deformable Object Simulation	44
2.3	NVIDIA Isaac Sim and IsaacLab Architecture	49
2.3.1	Isaac Sim in the Omniverse application stack.	50
2.3.2	Core simulation features in Isaac Sim 5.1	50
2.3.3	IsaacLab as a unified robot learning framework	52
2.3.4	Rationale for selecting Isaac Sim 5.1 with IsaacLab	54
2.3.5	Comparison to alternative simulation platforms.	55
2.4	Tendon-Driven Robot Kinematics and Modeling	56
2.4.1	Tendon-driven actuation principles	56
2.4.2	Kinematic modeling: from tendon space to task space	57
2.4.3	Robot configuration used in this project	58
2.4.4	Simulation considerations for tendon-driven systems in Isaac Sim . . .	59
2.4.5	Precision, dynamics, and compliance: implications for cloth manipula- tion	61
2.4.6	Workspace analysis and dexterity metrics	61
2.5	Policy Learning Pipeline	64
2.5.1	Agent–Environment Cycle and Markov Decision Processes	64
2.5.2	IsaacLab MDP Workflow for Task Construction	65
2.5.3	MDP Definition for Cloth Manipulation	66
2.5.4	Reward Shaping and Curriculum Strategies	71
2.5.5	GPU-Vectorized Training and Environment Wrappers	72
2.6	Prior Work in Robotic Cloth Manipulation	73
2.6.1	Task taxonomy and recurring technical challenges.	73
2.6.2	Model-based and analytical approaches	74
2.6.3	Learning-based approaches	75
2.6.4	Benchmarks, datasets, and evaluation protocols.	77
2.6.5	Sim-to-real transfer strategies for deformable objects	78
2.6.6	Open problems and positioning of the thesis	79
2.7	Robot and Scene Description Formats	79
2.7.1	USD as a scene graph for simulation worlds	80
2.7.2	URDF as a robotics-oriented robot model format	81
2.7.3	MJCF as MuJoCo’s native modeling language	82
2.7.4	Format use in this thesis and tendon-driven modeling workarounds . .	83

3	Methodology	88
3.1	Overview of Approach	88
3.2	Simulation Environment Setup	88
3.3	Robot Model Integration	88
3.3.1	2-DoF Articulated Base (Linear Y–Z Stage)	88
3.3.2	3-DoF Tendon-Driven Arm	89
3.3.3	Robotiq 2F-140 Gripper Integration.	90
3.3.4	Modular Assembly via Isaac Sim’s Robot Assembler.	91
3.3.5	Control Gain Tuning and Validation	93
3.4	Reinforcement Learning Setup	94
3.4.1	State Space	94
3.4.2	Action Space	94
3.4.3	Reward Space	94
3.4.4	RL Algorithm and Training Parameters	94
3.4.5	MDP Workflow in IsaacLab	94
3.5	Initial Experiments	94
3.5.1	Reach Pose Task	94
3.5.2	Simple Pick and Place Task	94
3.5.3		95
3.6	Advanced Task Implementation	95
3.7	Software and Hardware Integration	95
4	Results.	96
4.1	Preliminary Experiments	96
4.2	Performance of Tendon-Driven Robot on Simple Tasks	96
4.3	Cloth Manipulation Task Results	96
4.4	Summary of Key Results	96
5	Discussion	97
5.1	Evaluation of Outcomes	97
5.2	Comparison with Related Work	97
5.3	Limitations of the Study	97
5.4	Practical Implications	97
5.5	Recommendations and Lessons Learned	97
6	Conclusion	98

6.1 Conclusion	98
6.2 Outlook	98
Bibliography	99
Source Directory	114

List of Figures

2.1	Isaac Sim physics stack.	41
2.2	Isaac Sim and IsaacLab stack.	50
2.3	Robot kinematic chain.	59
2.4	Workspace voxel map.	62
2.5	IsaacLab manager-based MDP decomposition.	65
2.6	Taxonomy of cloth manipulation tasks.	77
2.7	Model and scene description pipeline.	80
3.1	Integrated tendon-driven robot model.	88
3.2	Exploded view of robot assembly.	92

List of Tables

2.1	Selected rigid-body simulation parameters relevant to robot learning in Isaac Sim / PhysX..	43
2.2	Qualitative comparison of simulators with respect to thesis requirements.. . . .	56
2.3	Pose-relevant degrees of freedom for workspace analysis. Joint limit values are defined by the robot specification and are applied during sampling.. . . .	59
2.4	Recommended Markov Decision Process (MDP) specification elements for cloth grasp-and-stretch in simulation..	66
2.5	Structured comparison of representative prior work in robotic cloth and garment manipulation, covering analytical/model-based pipelines, learning-based approaches, and benchmarks..	86
2.6	Roles of Universal Scene Description (USD), Unified Robot Description Format (URDF), and MuJoCo XML Format (MJCF) in robotics simulation pipelines.. . . .	87
3.1	Actuated joints of the integrated robot and their motion ranges..	93

List of Abbreviations

API	Application Programming Interface
CAD	Computer-Aided Design
CPU	Central Processing Unit
DoF	Degree of Freedom
DR	Domain Randomization
dVRK	da Vinci Research Kit
FEM	Finite Element Method
FK	Forward Kinematics
GAE	Generalised Advantage Estimation
GPU	Graphics Processing Unit
IK	Inverse Kinematics
IL	Imitation Learning
MDP	Markov Decision Process
MJCF	MuJoCo XML Format
MPC	Model Predictive Control
MPM	Material Point Method
PBD	Position-Based Dynamics
PD	Proportional-Derivative
PGS	Projected Gauss–Seidel
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RGB-D	Red-Green-Blue plus Depth
RL	Reinforcement Learning
ROS	Robot Operating System
SDF	Signed Distance Field
TGS	Temporal Gauss–Seidel
URDF	Unified Robot Description Format
USD	Universal Scene Description
VBD	Vertex Block Descent
XML	Extensible Markup Language
XPBD	Extended Position-Based Dynamics

List of Symbols

$\mathcal{A}$	Action space
A_t	Advantage estimator
ϵ	Clipping parameter (PPO)
γ	Discount factor
h	Simulation time step
$J(\theta)$	Expected discounted return
k_j	Stiffness parameter (PBD)
$\mathbf{M}$	Mass matrix
π_θ	Policy (parameterized by θ)
$r(s_t, a_t)$	Reward function
$\mathcal{S}$	State space
θ	Policy parameters

-1 Guideline

-1.1 Format and Style

-1.1.1 Page Layout and Typography

- Paper format: white DIN A4 paper
- Font: Arial, 12pt (in this LaTeX template, the sans-serif option achieves this)
- Line spacing: 1.2 (the template uses 1.25)
- Text alignment: justified (Blocksatz) with automatic hyphenation enabled
- Font emphasis: use only in exceptional cases; headings follow the template style and must not be underlined
- Bullet points: only square bullet points (FAPS green), never round ones; use the template's built-in list style
- FAPS color scheme: FAPS-Grün (RGB 151, 193, 57) and white

-1.1.2 Page Numbering and Binding

- Page numbering starts with the introduction (Arabic numerals)
- Directories (table of contents, list of figures, etc.) receive separate Roman numeral numbering; the cover page is not counted
- Print single-sided unless otherwise agreed with the supervisor
- Two bound copies required: adhesive binding with clear cover foil and green (FAPS green) back sheet
- Each copy must include a CD/DVD containing:
 - Digital version of the thesis (PDF)
 - All digital sources (papers, scanned documents, websites)
 - All images, tables, measurement data, models, and other data

-1.1.3 Section and Chapter Formatting

- Chapters are numbered numerically with at most three levels (e.g., 1.1.1); no trailing dot after the last digit

-
- If a subsection X.1 exists, there must also be at least an X.2 (applies to all levels)
 - Main chapters should have at least 3 subchapters; aim for a balanced structure (e.g., 5 chapters at level 1, each with 4 at level 2, each with 3 at level 3)
 - Headings in the table of contents must match the headings in the text verbatim
 - Every heading must be followed by actual content (including introductory text at the start of main chapters)

-1.1.4 Paragraphs, Spacing, and Whitespace

- Each paragraph must consist of at least two sentences
- No blank line between paragraphs within a chapter; one blank line before the next heading at the end of a chapter
- No double spaces anywhere in the document; use search-and-replace to eliminate them before submission
- Use non-breaking spaces (in LaTeX: ~) between:
 - Titles and names: “Prof. Dr. Beispielhausen”
 - Date expressions: “19. September 2007”
 - Abbreviations: “z. B.”, “u. a.”, “d. h.”
 - Numbers and units: “44 mm”
 - Numbers and words: “1.350 Personen”, “Nr. 5”
 - Citation references
- Check automatic hyphenation carefully for errors (e.g., incorrect compound-word breaks)

-1.1.5 Figures

- Every figure receives a caption below it (Unterschrift), formulated so the figure is understandable without the surrounding text
- All figures are numbered consecutively with Arabic numerals and listed in the list of figures
- Every figure must be referenced in the body text
- Title image: no caption, no numbering, not included in the list of figures
- Appendix figures: numbered within each appendix separately (e.g., Appendix A – Figure 1); not included in the list of figures
- Ensure sufficient image quality; recreate figures if originals are blurry, low-resolution, or scanned
- Use the same font in figures as in body text (Arial); font size 10pt is acceptable but must be consistent across all figures

-
- All figure components must be fully labeled: photo components, axes, parameters, legends with complete process parameters
 - Fill areas with color only if the color conveys meaningful information; otherwise keep figures clean and minimal
 - For recreated/modified figures from sources, cite with “in Anlehnung an [xy]” (based on [xy])
 - Use a consistent color scheme across all figures; preferably FAPS colors unless industry cooperation requires otherwise
 - Use a self-created graphic on the first page of the thesis

-1.1.6 Tables

- Every table receives a caption above it (Überschrift) with a meaningful, descriptive title
- Tables are numbered consecutively with Arabic numerals and listed in the table directory
- Every table must be referenced in the body text
- Use the same font as in body text; maintain consistent font size across all tables
- Preferred alignment inside tables: left-aligned or centered (not justified)
- If a table spans a page break, do not break within a row; repeat the table header on the new page
- Extensive tables belong in the appendix

-1.1.7 Diagrams

- Follow DIN 461 for axis labeling in diagrams
- Label axes with an arrow in the direction of increase (except for trial numbers etc.)
- Replace the axis label at the second-to-last tick mark with the unit
- Display all relevant information within the diagram
- Place a frame around diagrams
- Use Arial font inside diagrams with a legible font size
- Diagrams are captioned and numbered as figures (no separate numbering type)

-1.1.8 Formulas

- Formula numbers are enclosed in round parentheses

-
- Numbering is chapter-based in the form X.Y (X = chapter number, Y = formula number within the chapter)
 - Numbering restarts within each main chapter
 - Variables used in a formula must be explained directly below the formula in tabular form

-1.1.9 Appendix

- Technical drawings, extensive tables, supplementary documents, etc. go into appendices
- Each appendix starts on a new page and may span multiple pages
- Appendices are numbered with consecutive uppercase Latin letters (Appendix A, Appendix B, etc.)

-1.1.10 Eidesstattliche Erklärung (Statutory Declaration)

- A signed declaration of independent authorship must be included on a separate page without a page number, placed after the cover page
- It must be signed by hand

-1.2 Thesis Structure and Content

-1.2.1 Required Document Order

1. Deckblatt (Cover Page)
2. Erklärung des Kandidaten (Statutory Declaration)
3. Inhaltsverzeichnis (Table of Contents)
4. Abbildungsverzeichnis (List of Figures)
5. Tabellenverzeichnis (List of Tables)
6. Abkürzungsverzeichnis (List of Abbreviations)
7. Einleitung und Zielstellung (Introduction and Objective)
8. Grundlagen / Stand der Technik (Background / State of the Art)

-
9. Handlungsbedarf (Research Gap / Need for Action)
 10. Methodenteil (Methodology)
 11. Ergebnisteil (Results)
 12. Diskussionsteil (Discussion)
 13. Zusammenfassung und Ausblick (Summary and Outlook)
 14. Literaturverzeichnis (Bibliography)
 15. Anhang (Appendix)
 16. Lebenslauf (Curriculum Vitae)

-1.2.2 Page Count Guidelines

- Bachelorarbeit: ca. 60–80 pages
- Projekt-/Studienarbeit: ca. 60–80 pages
- Master-/Diplomarbeit: ca. 80–120 pages
- These counts exclude appendices and directories

-1.2.3 Red Thread (Roter Faden)

- A clear red thread must run through the entire thesis, starting from the central research question in the introduction
- The reader must always be able to see how any part of the thesis relates to the research question
- The thesis must proceed purposefully and not lose itself in tangential topics
- All claims and statements must be causally justified; results must not merely be listed but convincingly explained
- Each main chapter must begin with a short introduction motivating the content and end with a transition to the next
- Verify the red thread by checking all headings alone (without reading the text) — the logical flow should be recognizable

-1.2.4 Chapter: Einleitung und Zielstellung (Introduction)

- Length: approximately 2 pages, maximum 3 pages

-
- Purpose: establish contact with the reader; provide orientation (“What can the reader expect?”)
 - Content requirements:
 - Introduce the topic and its general background; explain why it is relevant
 - Present the specific research question
 - Justify the scope and boundaries of the thesis (what is included, what is excluded, and why)
 - Optionally describe the state of research (to show what exists and where gaps are)
 - Explain the methodological approach
 - Outline the structure of the thesis for the reader

-1.2.5 Chapter: Grundlagen und Stand der Technik (Background and State of the Art)

- Clarify central concepts and define relevant terms
- Present the current state of the art objectively and without judgment (evaluation happens in the next chapter)
- Only cover theories, definitions, and knowledge directly relevant to the research question; avoid overly broad coverage
- If multiple competing theories exist, provide an overview
- Definitions are only needed for terms not commonly known by domain experts
- Present content in your own words; near-verbatim copying or translation from sources counts as insufficient
- Use primarily English-language sources where possible
- Prefer journal articles and conference papers over websites
- Avoid popular-science sources (e.g., Schülerduden, Brockhaus); only quote newspapers for current events
- Internet sources are acceptable only if no current literature exists elsewhere (e.g., software documentation); verify their credibility

-1.2.6 Chapter: Handlungsbedarf (Research Gap)

- Critically discuss the literature from the previous chapter regarding unsolved problems and errors
- Derive current deficits and research gaps; formulate the specific research question
- Explain who considers the problem significant and what its broader impact is

-
- Describe the contribution the thesis makes to both research (theory, models, methods, facts) and practice
 - Narrow the problem statement into a concrete, achievable objective with clear expected outcomes

-1.2.7 Chapter: Methodenteil (Methodology)

- Together with Results and Discussion, this forms the main body (2/3 of the thesis)
- Describe the investigation plan, procedures, and all steps and tools in sufficient detail for reproducibility
- Justify why specific methods were chosen over alternatives (pros/cons analysis)
- For experimental studies: describe the population and sample in all relevant characteristics; justify sample adequacy
- Scientific criteria for methods: objectivity, reliability, and validity
- The description must enable other researchers to reproduce the investigation
- Only describe analysis procedures in detail if they are novel or non-standard

-1.2.8 Chapter: Ergebnisteil (Results)

- Present results of the investigation objectively and without interpretation
- Structure the chapter logically, aligned with the theory and methodology sections
- Use figures, tables, and diagrams to visualize results
- No interpretation or evaluation at this stage

-1.2.9 Chapter: Diskussionsteil (Discussion)

- Summarize and critically discuss the results
- Evaluate and interpret results in relation to the original problem and objective from the introduction
- Argue the significance for scientific discourse and practical implications

-1.2.10 Chapter: Zusammenfassung und Ausblick (Summary and Outlook)

- Maximum 3 pages
- Reconnect to the central problem and objective from the introduction
- Compactly summarize the most important results

-
- Place results in a broader context
 - Address:
 - What are the key findings with respect to the research questions?
 - How can the results be interpreted or evaluated?
 - What questions remain open, and why?
 - What do the results mean for future work on this topic?
 - Where are opportunities for further research?
 - Discuss the limitations of the investigation
 - Provide suggestions for future work and improvements based on newly emerged questions

-1.3 Scientific Writing Style

-1.3.1 General Principles

- Write in present tense (Präsens) and nominal style (Nominalstil)
- Use impersonal form throughout; never use “ich” (I) or “man” (one/you)
- Develop the red thread of argumentation before formulating sentences
- Every sentence and every claim must be precise and well-placed
- Write result-oriented: “Die Einflüsse führen zu Veränderungen.” not “Im Rahmen der Arbeit wurden untersucht...”

-1.3.2 Sentence Construction

- Use passive voice sparingly — prefer active formulations
- Avoid sentences with “man” entirely
- Be careful with infinitive constructions
- Avoid chaining too many nouns together (“Substantivaneinanderreihungsproblem”)
- No nested or overly complex sentences (Schachtelsätze)

-1.3.3 Grammar

- Avoid relative time references (“kürzlich”, “später”, “modern”) since the thesis should remain valid for years
- Do not use subjunctive mood (könnte, sollte, müsste, dürfte)

-1.3.4 Technical Language

- Use precise adjectives; avoid vague terms like “innovativ” or “intelligent”
- No pleonasms (redundant word pairs)
- Do not artificially inflate terms
- Be cautious with superlatives and comparative forms
- Do not invent words
- Use anglicisms and foreign words sparingly; only when no equivalent German term exists
- Use precise technical terminology; avoid colloquial language
- Use technical terms consistently throughout (e.g., always “CAD-Modell”, never alternating with “CAD Modell”)

-1.3.5 Expression and Phrasing

- Spell out numbers from zero to twelve; use digits from 13 onward
- Avoid colloquialisms and journalistic clichés
- Do not use rhetorical questions; make clear statements instead
- Avoid vague qualifiers: “im Allgemeinen”, “in der Regel”, “beispielsweise”, “teilweise”
- Avoid filler words: “nämlich”, “also”, “so”, “eben”
- Avoid repetition of both words and arguments
- Always formulate concretely and precisely; define unambiguously; enumerate completely; avoid generalizations
- Use nominal style: instead of “wird bearbeitet”, write “Zur bedarfsgerechten Bearbeitung ist auf die Einhaltung von xy zu achten.”

-1.3.6 Content Guidelines

- Do not document iterative development processes or suboptimal intermediate solutions; present only the final approach
- Do not describe device/hardware development in excessive detail; focus on scientifically relevant, generalizable, transferable aspects
- Use direct quotes rarely and only pointedly; prefer evaluating and developing ideas further in your own words
- Introduce abbreviations only when necessary for space; spell out on first use and include in the abbreviation index

-
- Emphasize scientific quality rather than engineering challenges
 - Emphasize fundamental, generalizable, and transferable findings rather than application-specific details
 - Instead of giving examples, enumerate completely, structure, and evaluate
 - Make all enumerations and representations as complete and non-overlapping as possible (MECE principle)
 - Always support claims with your own analyses or cited sources
 - No implied personal opinion (e.g., “jedoch”, “überaus”) and no subjective assessments except in the Outlook section
 - Where possible, summarize each chapter’s key message in a figure for quick reference

-1.3.7 Introductions and Transitions Between Sections

- Every main chapter or main section must begin with a brief introduction and end with an outlook/transition to the next section
- Example introduction: “Hasen und Igel sind, betrachtet man ihre Laufgeschwindigkeit, auf einem vollkommen unterschiedlichen Niveau.”
- Example transition: “Deshalb wird im folgenden Abschnitt der Wettlauf zwischen Hase und Igel näher betrachtet.”
- When introducing detailed explanations, name all relevant points in the introductory sentence
- Bad: “Im Folgenden werden die beiden Protagonisten näher betrachtet.”
- Good: “Im Folgenden werden die beiden Protagonisten Hase und Igel näher betrachtet.”

-1.3.8 Consistency and Formatting Rules

- Introduce each abbreviation exactly once and then use it consistently
- Never write a paragraph consisting of only one sentence (minimum two sentences)
- Never write a list with only one bullet point (minimum two)
- Use nominal style for bullet-point items
- Maintain consistent spelling of expressions throughout (e.g., always “Aerosol-Jet-Druck”, never “Aerosoljet-Druck”)

-1.4 Citations and Bibliography

-1.4.1 Citation Style

- Citation style: DIN ISO 690 (FAPS custom style via `bibstyleFAPS.bst`)
- Reference sources in the text using numbers in square brackets: [xy]
- Page numbers are not required in citations
- Multiple consecutive citations use separate brackets: [xy] [yz], not [xy; yz]
- The bibliography is ordered chronologically and numbered consecutively; no grouping by source type
- Only actually cited works appear in the bibliography

-1.4.2 Citation Placement

- A citation before the period refers to the specific sentence [xy].
- A citation after the period of the last sentence refers to the entire paragraph. [xy]

-1.4.3 Direct Quotes

- Must be letter-perfect and character-perfect reproductions
- Enclose in quotation marks; cite the source at the end: "... [20]
- Quote foreign-language texts in the original language; do not translate
- Mark own additions or rearrangements with square brackets
- Mark omissions with [...]
- Quotes longer than three lines: indent as a block, set in italic, omit quotation marks
- Do not correct errors in the original; mark them with [!] or [sic!]
- Emphasis within quotes: append [Hervorhebung durch Verfasser]

-1.4.4 Indirect Quotes

- Paraphrased ideas from other sources; not placed in quotation marks
- Still require a source reference in the same placement rules as above

-1.4.5 Secondary Citations

- Always prefer citing the original source

-
- Use secondary citations only when the original is unavailable; reference the secondary source in the bibliography

-1.4.6 Plagiarism

- Provide a source reference for every piece of knowledge that goes beyond common knowledge
- Originality and independence are the most important quality criteria
- Even when rephrasing or summarizing, the source must be cited
- It must be unambiguously clear to the reader at all times what is borrowed intellectual property

-1.4.7 Source Quality Requirements

- Every source must be: (1) citeable (publicly accessible), (2) citeworthy (scientific quality), and (3) relevant
- Prefer English-language sources
- Prefer journal articles and conference papers over websites
- Avoid websites as sources wherever possible; especially avoid Wikipedia, hausarbeiten.de, and similar
- If no conventional sources exist (e.g., for software topics), internet sources are permissible with critical evaluation
- Prioritize current sources from renowned journals or scientists over older work
- Quality indicators: scientific question, clear structure, precise terminology, theoretical foundation, appropriate methodology

-1.4.8 Electronic Sources

- Books, journals, and publications that also exist in print are not “electronic sources” even when accessed online
- Each source entry must answer: Who? What? Where? When?
- Use DOI when available; otherwise provide the full URL and access date
- Do not insert hyphens in URLs (could be misinterpreted as part of the address)
- If author is unknown: use “o. V.” (ohne Verfasser); if date is unknown: use “o. J.” or “o. D.”
- Schema: Nachname, Vorname (Jahr): Titel. [ggf. weitere Angaben]. DOI oder online unter: URL (Abrufdatum). No period after DOI.

-1.5 Literature Research

- Recommended databases: OPAC (FAU library), Google Scholar, IEEE Digital Library, Scopus
- Strategy: create a search table with key terms, synonyms (abstract and specific) across multiple columns
- Use primarily English-language sources where possible
- Prefer journal articles and conference papers; avoid websites
- Submit a detailed outline and relevant literature to the supervisor within 2 weeks (for Abschlussarbeiten)

-1.6 Figures and Graphics — Additional Rules

- Use a self-created graphic on the first page of each chapter or major section
- If independent creation is not possible or the graphic is assembled from multiple sources, provide a source note on the first page where it appears
- Poorly formatted, inconsistent, blurry, or illegible graphics create a bad impression; avoid copies and scans
- Recreate relevant graphics, diagrams, formulas, and tables to ensure uniform formatting and meet formal requirements
- Maintain a balanced ratio of text to graphics (e.g., avoid placing two graphics directly after each other)
- Ensure consistent design across all graphics: shapes, fonts/sizes, colors

-1.7 Review Checklist Before Submission

-1.7.1 Content Review

- Verify all headings form a coherent red thread (recognizable without reading the text)
- Verify transitions and introductions exist at the beginning of every main chapter
- Is the central research question clearly developed in the introduction?
- Is the relevant literature reviewed with appropriate focus (not quantity, but quality of selection)?
- Can the derivation of the target concept from the analysis be followed?
- Are strengths and development potentials considered in the solution space?

-
- Is the proposed solution feasible and implementable?
 - Does the outlook present innovative approaches for future work?
 - Are all tables and figures clear and understandable?
 - Is the context of existing literature properly established?
 - Is there a concise summary that still covers all essential points?
 - Is the overall structure clear and comprehensible?
 - Is the scope sufficient yet as concise as possible?
 - Were all key work items of the thesis topic fulfilled and the objectives achieved?

-1.7.2 Language and Expression Review

- Check for scientific, objective tone throughout
- Check for linguistic expression quality
- Run spell-check and grammar-check
- Have at least one other person proofread the thesis

-1.7.3 Visual and Formatting Review

- Check in single-page view for equal spacing between sections
- Verify balanced ratio of text and graphics (no dense clustering of figures)
- Verify consistent design of all graphics (shapes, fonts, sizes, colors)
- Eliminate all double spaces
- Check for sensible page breaks (no orphan headings, no broken table rows)

-1.8 Grading Criteria

- Systematic approach and methodology
- Personal effort and creativity
- Fulfillment of the thesis assignment
- Independence during the work
- Application of domain expertise
- Discussion and critical evaluation of results
- Presentation of the initial situation
- Thoroughness, completeness, and comprehensibility of the presentation
- External form and appearance of the thesis
- Expression and style

-
- Spelling and grammar
 - Performance is measured as work divided by time — high quality within deadlines demonstrates strong performance

-1.9 Presentation Guidelines (BA/PA)

- Duration: 20 minutes $\pm$ 10% deviation
- Bring your own laptop; have a backup on a USB drive
- Submit slides to the supervisor several days before the presentation date for review
- Follow FAPS presentation guidelines
- Dress code: at minimum business casual
- Structure:
 1. Title slide
 2. Motivation slide (between title and table of contents)
 3. Table of contents slide (all relevant topics except bibliography)
 4. Content slides
 5. Summary slide
 6. Closing slide
 7. Bibliography (hidden; only shown if asked during Q&A). Cite sources on individual slides in (Author, Year) format

0 Chapter Outline

0.1 Chapter 1: Introduction (5 pages)

0.1.1 Motivation and Context

- Introduce the importance of autonomous robotic cloth manipulation in manufacturing and recycling contexts
- Highlight the challenges of handling deformable objects (textiles) and why simulation with reinforcement learning is a promising approach
- Emphasize the specific scenario: using a 6-DOF tendon-driven robotic arm to pick up and spread cloth for inspection on a conveyor
- Establish the industrial relevance: textile sorting, quality inspection, and recycling as concrete application domains
- Briefly frame the sim-to-real challenge as a central theme connecting simulation training to physical deployment

0.1.2 Problem Statement and Objectives

- Clearly state the thesis problem: developing an RL-based control strategy for a tendon-driven robot to manipulate cloth (grasp, stretch, present) in simulation and eventually real life
- List the research objectives:
 1. Establishing a high-fidelity Isaac Sim simulation environment
 2. Designing and training RL policies for cloth manipulation tasks
 3. Integrating a custom tendon-driven robot model
 4. Addressing sim-to-real transfer (ensuring the learned policy can generalize or be adapted)
- Explain that initial simpler manipulation tasks have already been implemented as a stepping stone and the thesis builds on this foundation
- Scope delimitation: explicitly state what is NOT addressed (e.g., full autonomous perception pipeline, multi-robot coordination) and justify why

0.1.3 Thesis Outline

- Summarize the structure of the thesis (Chapters 2–7) for the reader
- Chapter 2: background theory and related work
- Chapter 3: research gap and refined problem
- Chapter 4: methodology including simulation setup and RL approach
- Chapter 5: experimental results
- Chapter 6: discussion of findings and limitations
- Chapter 7: conclusion and future work

0.2 Chapter 2: Background and State of the Art (20 pages)

0.2.1 Section 2.1: Reinforcement Learning for Robotic Manipulation

- Introduce Markov Decision Processes (MDP) as the formalism for RL: define states, actions, transitions, rewards, and discount factor
- Outline key RL algorithms relevant to continuous robot control: model-free policy gradient methods (DDPG, PPO, SAC)
- Discuss why these algorithms are suitable for high-dimensional, continuous action problems like controlling robotic arms
- Compare on-policy (PPO) vs. off-policy (SAC, DDPG) approaches: sample efficiency, stability, and suitability for GPU-parallel simulation
- Highlight specific algorithms or frameworks used in this thesis (e.g., RL Games, Stable Baselines, skrl)
- Briefly cover policy network architectures: MLPs, actor-critic structure, and optional recurrent layers for partial observability
- Summarize prior successes of RL in simulation for manipulation tasks (e.g., OpenAI Dexterous Hand, QT-Opt)

0.2.2 Section 2.2: Physics Simulation in Isaac Sim 5.1

- Introduce the physics engine stack of Isaac Sim 5.1: NVIDIA PhysX 5 as the core solver, with GPU-accelerated simulation capabilities

-
- Explain why understanding the underlying physics models is essential for sim-to-real transfer and RL training fidelity

0.2.2.1 Rigid Body Dynamics

- Cover how Isaac Sim handles rigid body simulation via PhysX: articulated bodies, joints, contacts, and friction models
- Explain the Temporal Gauss-Seidel (TGS) solver used by PhysX for rigid body constraint solving
- Describe relevant simulation parameters: time step, solver iterations, contact offsets, friction coefficients
- Discuss GPU-accelerated rigid body simulation for parallel RL environments
- Explain how articulated robots (including the tendon-driven arm and gripper) are simulated as rigid body chains with joint constraints
- Cover collision detection: mesh vs. convex decomposition vs. primitive shapes; performance vs. accuracy tradeoffs

0.2.2.2 Cloth and Deformable Object Simulation

- Explain why deformable objects break standard rigid-body assumptions: applied forces not only translate the object but also change its shape and contact state
- Describe common cloth modeling approaches in context: mass-spring models, particle-based constraint methods, continuum formulations (FEM/MPM)
- Emphasize the unique challenges of cloth dynamics: high degrees of freedom, self-collision, sensitivity to material parameters (bending stiffness, friction, damping)

0.2.2.3 Position-Based Dynamics (PBD) — PhysX Particle Cloth

- Explain PBD as the solver used by PhysX for particle cloth: particles with mass connected via holonomic constraints
- Describe the PBD time-step algorithm: external-force velocity update, position prediction, iterative Gauss-Seidel constraint projection, velocity correction
- Present the constraint correction formula and explain how stiffness depends on time step and iteration count
- Introduce XPBD as an extension with compliance formulation and Lagrange multipliers for more physically meaningful stiffness
- Explain why PhysX uses PBD for particle cloth: GPU parallelism via graph coloring, robustness, interaction with rigid bodies and articulations

-
- Discuss how PBD particle cloth is characterized in Omniverse/Isaac Sim: a PBD mass-spring system distinct from newer deformable formulations

0.2.2.4 Vertex Block Descent (VBD) — Newton Thin Deformables

- Explain VBD as an implicit, energy-based solver targeting variational implicit Euler time integration
- Describe the global energy minimization formulation and the vertex-level block coordinate descent approach
- Present the local Newton update step (3×3 Hessian solve per vertex)
- Discuss key advantages: unconditional stability, convergence to implicit Euler solution, GPU parallelizability via vertex coloring
- Explain Newton's role in NVIDIA's robotics context: improved performance and stability of VBD for thin deformables such as clothing
- Discuss why VBD is attractive for cloth manipulation: better tolerance of stiffness ratios (near-inextensible stretch with softer bending) and larger time steps

0.2.2.5 Comparison: PBD vs. VBD for Cloth in RL Workflows

- Compare mathematical targets: PBD projects positions to satisfy constraints vs. VBD minimizes variational implicit Euler energy
- PBD: simple, robust, fast, but effective stiffness depends on time step and iteration count; harder to map to physical material properties
- VBD: principled implicit integration, strong stability guarantees, favorable convergence in large-DOF systems
- Relate to Omniverse ecosystem: particle cloth = PBD mass-spring; newer deformable features moving toward implicit/compliance-based formulations
- Motivate the choice for this thesis: PhysX/PBD particle cloth for throughput and robustness in Isaac Sim 5.1; Newton/VBD when greater stability under stiff cloth and larger time steps is needed

0.2.3 Section 2.3: NVIDIA Isaac Sim and IsaacLab Architecture

- Introduce NVIDIA Isaac Sim (version 5.1) as the simulation environment
- Describe core features: built on Omniverse, high-fidelity physics (PhysX 5), photo-realistic rendering with RTX, ROS 2 support, GPU acceleration for parallel simulations

-
- Explain IsaacLab: NVIDIA’s unified robotics learning framework built on Isaac Sim (evolved from the Orbit framework)
 - Note that IsaacLab combines Isaac Gym’s GPU-accelerated physics with Orbit’s environment library, providing a modular toolkit for RL, imitation, and motion planning
 - Discuss how IsaacLab’s MDP workflow organizes simulation tasks: defining robots, environments, and training pipelines in a reusable way
 - Clarify why IsaacLab + Isaac Sim 5.1 is well-suited for this thesis (integrated cloth dynamics support, tools for distributed RL training)
 - Compare with alternative simulation platforms (MuJoCo, PyBullet, SOFA) to justify the choice of Isaac Sim for this specific use case

0.2.4 Section 2.4: Tendon-Driven Robot Kinematics and Modeling

- Explain what tendon-driven robots are: actuators apply force via tendons (cables) to move joints, allowing lighter, more compliant designs but requiring complex modeling
- Cover basic kinematic principles: how pulling tendons causes joint rotation or continuous bending, constant-curvature assumptions, differences in inverse kinematics/control from rigid joint robots
- Discuss the specific robot model relevant to the project: the 2-DOF linear stage + 3-DOF tendon arm + Robotiq gripper assembly
- Describe challenges in simulating tendon-driven systems in Isaac Sim: modeling tendon routing and elasticity, approximation via constraint forces or URDF/MJCF model with mimic joints
- Include literature context on tendon-driven manipulators: how their precision and dynamics differ from direct-drive robots
- Discuss the advantages of tendon-driven designs for compliant cloth interaction (inherent compliance, force limiting, safe contact)
- Introduce the concept of workspace analysis for serial manipulators: the set of all end-effector poses reachable within joint limits
- Define forward kinematics (FK) as the mapping from joint configuration q to end-effector position/orientation; explain why FK-based sampling is used when analytical workspace boundaries are intractable (high DOF, non-standard kinematics)
- Present the Monte Carlo sampling approach: uniformly sample joint configurations within joint limits, compute FK for each sample, and aggregate results in Cartesian

voxels (analogous to MathWorks `generateRobotWorkspace`)

- Define the **Yoshikawa manipulability index** $w(q) = \sqrt{\det(J(q)J(q)^T)}$: measures how well the end effector can be moved in all Cartesian directions; zero at singularities, higher values indicate better dexterity [Yoshikawa 1985]
- Define the **inverse condition number** (isotropy index) $\kappa_{\text{inv}} = \sigma_{\text{min}}/\sigma_{\text{max}}$ of the geometric Jacobian: ranges from 0 (singular) to 1 (perfectly isotropic); provides a complementary measure of motion quality [Salisbury & Craig 1982]
- Define **reachability density**: the number of FK samples per voxel, proportional to the joint-space volume mapping to that Cartesian region; higher density indicates more configurations can reach that area
- Discuss how these measures inform task-space design: placing objects and setting action bounds in regions with high manipulability and reachability, avoiding singular zones

0.2.5 Section 2.5: Policy Learning Pipeline (MDP Workflow)

- Detail the standard RL pipeline in the context of IsaacLab’s MDP workflow
- Define MDP components for the cloth manipulation task:
 - State (observations)
 - Action space (robot joint torques/velocities or high-level gripper motions)
 - Reward function design (what constitutes success for grasping/stretching cloth)
 - Episode termination conditions
- Explain curriculum learning or reward shaping concepts if used
- Describe how IsaacLab provides API/structure for tasks (environment wrappers, vectorized environments on GPU)

0.2.6 Section 2.6: Prior Work in Robotic Cloth Manipulation

- Review existing research on robotic cloth/deformable object manipulation
- Cover both model-free approaches (deep RL, learning-based methods) and model-based/analytical approaches (vision-based controllers, heuristic methods)
- Cite key studies and benchmarks: folding/unfolding tasks, cloth smoothing/flattening (Seita et al.), GarmentBench/GarmentLab benchmarks
- Note outcomes: model-free RL achieves good simulation performance but needs many training samples and can struggle to generalize
- Highlight open problems from surveys: generalizing to varying fabrics, dealing with noisy perception, accomplishing long-horizon multi-step cloth tasks

-
- Discuss sim-to-real approaches specifically for deformable objects: domain randomization of cloth parameters, visual domain adaptation, teacher-student policy distillation
 - Situate the thesis in context: how it builds on prior work and what gaps it addresses
 - Provide a structured comparison table of key related works (method, robot type, task, simulation used, sim-to-real status)

0.2.7 Section 2.7: Robot and Scene Description Formats (USD, URDF, MJCF)

- Explain USD (Universal Scene Description): extensible 3D scene graph format for constructing simulation scenes in Isaac Sim (objects, lighting, physics properties)
- Explain URDF (Unified Robot Description Format) and MJCF (MuJoCo XML) as common robot model formats
- Clarify which format is used for the tendon-driven robot and why
- Explain the role of each format: URDF/MJCF capture kinematic structure, joint limits, inertial properties; USD integrates robot and cloth into the simulated world
- Note any modifications to the robot's URDF to model tendons (workarounds for standard URDF limitations)

0.3 Chapter 3: Problem Statement and Research Gap (3 pages)

0.3.1 Section 3.1: Critical Analysis of State of the Art

- Build directly on Chapter 2's review and summarize what has been done and where limitations remain
- Note that while prior works achieve either cloth manipulation with traditional robots or basic RL policies in simulation, the combination of a tendon-driven arm with complex cloth tasks is unexplored
- Point out unresolved questions:
 - How to effectively train an RL policy for a high-DOF tendon-driven manipulator on cloth tasks without extensive manual reward shaping?
 - How to maintain sim-to-real fidelity for cloth interactions and tendon dynamics?

0.3.2 Section 3.2: Research Gap and Justification

- Clearly articulate the Handlungsbedarf (need for action) per FAPS guidelines
- Key gap: most existing studies use rigid-joint robots or simplified simulations; literature lacks demonstrations of RL-based control on tendon-driven robots for cloth handling; sim-to-real strategies for deformable tasks are not well established
- Justify scientific value: contributing knowledge on tendon-driven robot control and RL in deformable object tasks
- Justify practical relevance: automating textile recycling or clothing handling tasks
- Include a summary figure or diagram visualizing the research gap: map existing work along axes (robot type vs. object type vs. method) and show the empty quadrant this thesis fills

0.3.3 Section 3.3: Refined Problem Definition

- Formulate precise research questions or hypotheses:
 - How can a simulation environment be configured to realistically train cloth manipulation policies for a tendon-driven robot?
 - Which RL algorithms and reward formulations yield stable learning for the cloth stretching task?
 - What strategies can improve transferability of the learned policy to a physical robot (using ROS 2)?
- Define the scope: specific cloth size/type, specific robot design, what is not addressed (e.g., vision-based perception may be simplified)
- List the main thesis contributions:
 1. Integration of a tendon-driven robot model in Isaac Sim
 2. An RL pipeline for cloth grasp-and-stretch
 3. Evaluation of policy performance and generalization

0.3.4 Section 3.4: Success Criteria (Optional)

- Define how success is measured: e.g., cloth spreading coverage in simulation, stability of tendon arm control, comparisons to baseline methods
- Tie the problem statement to the forthcoming methodology

0.4 Chapter 4: Methodology and Implementation (15–18 pages)

0.4.1 Section 4.1: Overview of Approach

- Provide a high-level overview of the solution strategy
- Include a diagram of the system architecture or workflow (IsaacLab environment loop: state $\rightarrow$ policy $\rightarrow$ action $\rightarrow$ environment)
- Describe development stages:
 1. Stage 1: set up a simple simulation with a generic robot and train basic RL tasks
 2. Stage 2: integrate the tendon-driven robot model
 3. Stage 3: tackle the cloth manipulation task with the full system

0.4.2 Section 4.2: Simulation Environment Setup

- Detail the Isaac Sim 5.1 environment configuration for training
- Describe the USD scene: table/conveyor, cloth object (size, mesh/primitive, physical properties like mass, friction, stiffness), initial conditions (crumpled/folded/randomized)
- Robot placement and any sensors in simulation
- If ROS 2 is used within Isaac Sim (ROS bridge extension), explain how ROS topics are configured
- Simulation parameters: physics time-step, faster-than-real-time execution for RL training
- Domain randomization strategy: which parameters are randomized (cloth mass, friction, stiffness, initial pose, lighting) and their ranges
- Include a figure of the complete simulation scene with labeled components

0.4.3 Section 4.3: Robot Model Integration (Tendon-Driven Arm)

- Explain how the 6-DOF tendon-driven robotic arm is modeled and imported
- Describe the 2-DOF articulated base (linear Y–Z stage)
- Describe the 3-DOF tendon-driven arm

-
- Describe the Robotiq 2F-140 gripper integration
 - Explain the modular assembly via Isaac Sim's Robot Assembler
 - Describe any verification of the robot model in simulation (open-loop tests, joint motion verification)
 - Calibration of tendon tensions, spring constants, joint limits, and stiffness
 - Control gain tuning and validation
 - Include a figure of the robot model

0.4.3.1 Robot Simulation Setup and Validation

- Detail the physics simulation configuration for the articulated robot: articulation solver type (TGS), solver iteration counts, physics time step, and their effect on joint stability
- Describe the tendon actuation modelling decision: why PhysX articulation tendons were not used (fidelity and stability concerns per NVIDIA docs) and how direct PD joint drives approximate tendon behaviour instead
- Explain the PD drive parameterisation: stiffness and damping per joint, how initial values were derived from estimated tendon elasticity, and the iterative tuning process using Isaac Sim's Gain Tuner
- Describe collision geometry choices: convex hull generation from CAD meshes, performance vs. accuracy trade-off, and verification that no self-interpenetration occurs across the full joint range
- Present the step-response validation methodology: sinusoidal and step position commands applied to each joint independently; metrics recorded (settling time, overshoot, steady-state error); comparison with Klein's (2023) PID controller parameters and Jacobian transpose mapping
- Explain gravity compensation handling for the ceiling-mounted configuration and its effect on the vertical axis tuning
- Include a figure showing the Gain Tuner output: commanded vs. actual joint trajectories before and after tuning

0.4.3.2 Workspace Analysis Methodology

- Describe the Monte Carlo FK sampling procedure: 200 000 uniform random joint configurations within joint limits, FK computed via the IsaacSim 5.1 physics engine (not analytical FK, to capture the exact simulated kinematics)
- Explain parallelisation: 4096 environments running simultaneously on GPU, each evaluating independent joint configurations per step

-
- Detail the data collected per sample: end-effector position (Cartesian), geometric Jacobian, Yoshikawa manipulability index, inverse condition number
 - Describe voxelisation of the Cartesian workspace for density and quality-metric aggregation
 - Explain collision handling: scene objects (conveyors, drums) disabled during sampling so the kinematic envelope is fully explored; robot self-collision retained
 - Define the desired workspace region derived from Klein’s (2023) task geometry and how it is compared against the sampled reachable workspace
 - Describe the visualisation pipeline: 3D scatter plots, 2D cross-section heatmaps through the workspace midpoint, colour-coded by manipulability and reachability density
 - Explain how the results inform task-space bounds and action limits for the RL environment configuration

0.4.4 Section 4.4: Reinforcement Learning Setup

- **State Space:** what observations the agent receives (joint angles/velocities, gripper status, cloth features/key points, camera images if used)
- **Action Space:** what actions the agent outputs (continuous joint commands: position increments, velocity, or torque); clarify how tendon coupling affects the action space
- **Reward Function:** reward design for cloth manipulation (e.g., negative distance from desired flat configuration, increase in covered area); intermediate shaping rewards (grasping, lifting); reward schedule or curriculum
- **RL Algorithm and Training Parameters:** which algorithm is used (e.g., PPO); library (RL Games or Stable Baselines); key hyperparameters (learning rate, discount factor, policy network architecture, training duration); number of parallel simulation instances for GPU-accelerated training; any pre-training or demonstrations
- **MDP Workflow in IsaacLab:** how the custom task class is defined, specifying observation and reward functions; IsaacLab wrappers for interfacing with the RL algorithm; role of ROS 2 during training (if any)
- **Reproducibility:** provide a complete hyperparameter table and specify exact software versions (Isaac Sim, IsaacLab, CUDA, Python, RL library) for full reproducibility per FAPS methodology requirements

0.4.5 Section 4.5: Initial Experiments (Basic Task Policies)

- Describe initial RL experiments with simpler tasks and robots to validate the RL pipeline
- **Reach Pose Task:** setup, purpose, and what was learned/adjusted
- **Simple Pick and Place Task:** a simpler robot model (standard manipulator), dummy object instead of flexible cloth
- Purpose: test the training setup, reward shaping, hyperparameters before introducing tendon robot and deformable cloth
- Note adjustments from these initial trials (missing observations, hyperparameter tuning)

0.4.6 Section 4.6: Advanced Task Implementation (Cloth Grasping and Stretching)

- Detail the final task scenario: tendon-driven robot grasps cloth from surface and stretches it out
- Break down the complex task into phases: grasp, lift, stretch, place
- Strategies for making learning feasible:
 - Curriculum learning (start with cloth already grasped, or progressively increase difficulty)
 - Auxiliary rewards for sub-goals (reward for grasping, reward for moving upward)
- Gripper simulation: how grasp detection is handled (contact sensors, constraint on gripper close)
- Safety/stability measures: limiting force on tendons to realistic ranges to prevent simulation instabilities

0.4.7 Section 4.7: Software and Hardware Integration (ROS 2)

- Describe how ROS 2 is incorporated, especially with an eye towards real-world deployment
- During simulation: ROS 2 as bridge (echoing states, visualization in RViz)
- Outline how the trained policy interfaces with a real robot via ROS 2:
 - Robot's joint controllers subscribe to commanded positions from the RL policy
 - Policy subscribes to sensor feedback (joint states, possibly depth camera for cloth observation)

-
- Any preliminary setup for sim-to-real: using ROS 2 control for the tendon robot in sim to mirror what would be done on hardware
 - Exporting the trained policy for a ROS 2 node

0.5 Chapter 5: Results (15 pages)

0.5.1 Section 5.1: Preliminary Experiments (Baseline Results)

- Report performance of the RL policy on simple pick-and-place or reaching tasks with the simplified robot
- Include learning curves: reward over training iterations for the baseline task
- Number of episodes needed to reach a success criterion
- If multiple algorithms or settings were tried, compare briefly (e.g., PPO vs. DDPG in reaching task)
- Purpose: demonstrate that the RL pipeline works on a simpler problem and record insights
- Justify choices for the harder task (confirm hyperparameters or algorithm selection)

0.5.2 Section 5.2: Performance of Tendon-Driven Robot on Simple Tasks

- Present results if the tendon-driven arm was tested on a simple task to validate its controllability
- Report differences due to the robot's dynamics: whether learning was slower or required dealing with more complex dynamics
- Quantitative metrics: success rate, final accuracy for basic manipulation
- Validation of the simulated model: joint motions correspond to expected behavior, open-loop tests confirm model accuracy

0.5.2.1 Robot Simulation Validation Results

- Present step-response plots for each joint: commanded vs. actual position over time, showing settling time, overshoot, and steady-state tracking
- Report quantitative tuning outcomes per joint: final stiffness and damping values, settling time ($\sim 0.1\text{--}0.3\text{s}$), overshoot ($< 5\%$), and steady-state error
- Compare the tuned PD behaviour with Klein's (2023) PID controller results: discuss differences due to the simplified tendon model and their practical implications

-
- Show that the vertical axis (base Z) required higher damping to counteract gravity-induced oscillations; wrist joints use lower stiffness reflecting tendon compliance
 - Validate collision geometry: confirm no interpenetration across the full joint range, including extreme configurations
 - Discuss any simulation artefacts observed during tuning (e.g., jitter at joint limits, contact instabilities at high stiffness) and how they were resolved

0.5.2.2 Workspace Analysis Results

- Present the reachable workspace envelope: 3D scatter plot and volume estimate; compare against the desired workspace region from Klein’s (2023) task geometry
- Show 2D cross-section heatmaps (e.g., Y–Z and X–Z slices through workspace mid-point) colour-coded by reachability density and manipulability
- Report Yoshikawa manipulability index distribution: mean, percentiles, and spatial variation; identify high-dexterity and near-singular regions
- Report inverse condition number distribution: highlight regions with isotropic vs. anisotropic motion capability
- Discuss coverage: what fraction of the desired task workspace is reachable, and at what quality level (e.g., “95% of the target volume is reachable with $w > 0.01$ ”)
- Identify practical implications: regions where the robot operates well vs. zones to avoid in task design; how workspace limits constrain object placement and RL action bounds
- Compare briefly with the workspace reported by Klein (2023) for the original arm design (without the 2-DOF base) to show the benefit of the linear stage

0.5.3 Section 5.3: Cloth Manipulation Task Results

- **Learning Curve:** plot of reward vs. training steps or success rate vs. episodes; discuss how quickly the agent learned and whether it plateaued
- **Quantitative Performance:** define success metric (e.g., percentage of cloth spread flat above threshold); report final policy performance averaged over many test episodes (e.g., “85% cloth coverage on average, with $\geq 90\%$ coverage in 70% of trials”); use tables and graphs
- **Policy Behaviors:** describe what the learned policy actually does (e.g., grasps corner, lifts, shakes, lays down stretched); illustrate with sequential images from simulation showing keyframes
- **Ablation or Variant Results:** different reward formulations, disabling certain

observations; e.g., “without tactile feedback, success rate dropped by 15%”

- **Computational Cost:** report wall-clock training time, GPU memory usage, number of environment steps; quantify speedup from GPU parallelism
- **Sim-to-Real Considerations:** tests with added noise to gauge robustness; effect of domain randomization on policy performance; preliminary tests on real robot if available

0.5.4 Section 5.4: Summary of Key Results (Optional)

- Concise bullet-point recap of the most important findings
- Answer the objectives as evidenced by results:
 - Achieved reliable cloth picking in simulation
 - Partial/full stretching accomplished (flatness percentage)
 - RL training converged in approximately N million steps

0.6 Chapter 6: Discussion (5 pages)

0.6.1 Section 6.1: Evaluation of Outcomes

- Discuss to what extent the thesis objectives (from Introduction) were achieved
- For each objective: was a robust policy learned? How effective is it?
- Provide reasoning for the performance observed (e.g., learning curve plateau indicates suboptimal but acceptable strategy)
- Connect with theory: high-dimensional cloth state making dense reward difficult, exploration challenges

0.6.2 Section 6.2: Comparison with Related Work

- Compare approaches and results with literature from Section 2.6
- Discuss how the approach compared: advantages of GPU simulation enabling more training, using tendon-driven robot vs. standard robots
- Analyze shortcomings: e.g., pure RL approach vs. imitation learning may require additional strategies for real-world deployment
- Situate the contribution: either showing improvement or highlighting differences

0.6.3 Section 6.3: Limitations of the Study

- Limitations in simulation fidelity: cloth simulation as approximation (PhysX may not capture all fine wrinkles), tendon model may not include cable elasticity perfectly
- Risk of policy overfitting to simulated environment quirks
- Generalization concerns: does the policy only work for a specific cloth size or initial pose? What about varying material properties?
- If ROS 2/hardware integration was not fully realized, state that results are limited to simulation
- Training time considerations and policy sensitivity to parameter changes
- Discuss the validity of chosen evaluation metrics: are they sufficient to judge real-world deployment readiness?

0.6.4 Section 6.4: Practical Implications

- What the results mean for real-world applications (e.g., automated cloth handling in recycling facilities)
- Discuss the gap to reality: sensor noise, unpredictable cloth variations
- Whether the tendon-driven robot in simulation behaved similarly to expectations for reality; what adjustments might be needed
- Reflect on the decision to use IsaacLab/Isaac Sim: was it effective? Did GPU acceleration meaningfully speed up training?

0.6.5 Section 6.5: Recommendations and Lessons Learned

- Key lessons from the development process, for example:
 - Careful reward shaping was essential; purely sparse reward led to no learning
 - Introducing incremental rewards at different task stages was crucial
 - Calibration of joint limits and stiffness in simulation greatly affects training stability
 - An improperly tuned model caused unstable training
- These reflective points demonstrate critical thinking and provide valuable takeaways

0.7 Chapter 7: Conclusion and Outlook (3 pages)

0.7.1 Section 7.1: Conclusion (Summary of Findings)

- Recap the core accomplishments in a few concise paragraphs
- Reiterate the problem addressed and how it was solved
- Example summary: “developed an RL framework for a tendon-driven robotic arm to manipulate cloth in simulation; built a high-fidelity Isaac Sim environment with cloth physics; integrated a custom tendon-driven robot model; using policy-gradient RL, the robot learned to pick up and stretch cloth”
- Highlight the significance: possibly one of the first demonstrations of RL on a tendon-driven robot for cloth manipulation, or a novel combination of IsaacLab with a custom robot model
- Touch on each objective from the introduction, confirming whether it was met

0.7.2 Section 7.2: Outlook (Future Work)

- Specific next steps:
 1. **Real-world implementation:** deploying the learned policy on the physical tendon-driven robot via ROS 2; evaluating on real cloth; dealing with sim-to-real gaps (domain randomization, fine-tuning with real data)
 2. **Improving the policy/generalization:** exploring more advanced RL algorithms; incorporating vision-based observation for handling varying cloth shapes or unknown cloth pieces
 3. **Task expansions:** moving beyond stretching to folding, handling multiple pieces, integrating higher-level planning for sequential tasks
 4. **Simulation enhancements:** more accurate cloth models, additional sensors (depth camera for cloth state perception)
- Open research questions: adaptability of learned policy to different fabrics (varying stiffness); training with a range of material properties for robustness
- Closing remark: reflect on broader impact — integration of advanced simulation with RL demonstrates a path toward robots that can handle complex, deformable objects autonomously

0.8 Appendices (not counted in page total)

- Supplementary material: extensive tables, hyperparameter lists, additional diagrams, code snippets
- Full URDF/MJCF of the robot
- Detailed network architectures
- Additional results plots
- Complete reward function formulations and their evolution during development
- Domain randomization parameter ranges table
- Software version matrix (Isaac Sim, IsaacLab, PhysX, CUDA, Python packages)

1 Introduction

1.1 Motivation and Context

1.2 Problem Statement and Objectives

Autonomous sorting of textiles from mixed waste streams is a challenging manipulation problem: garments such as T-shirts are deformable, exhibit highly variable shapes, and interact with their environment through complex contact dynamics. In industrial recycling contexts, reliable textile extraction is a key prerequisite for downstream steps such as material recovery or further sorting by condition (e.g., damaged vs. reusable). Classical, model-based manipulation pipelines typically rely on accurate state estimation and carefully tuned controllers, which becomes difficult when both the manipulated object and the contact conditions change continuously.

This thesis addresses the problem of learning a robust control policy for a custom tendon-driven robot to selectively retrieve a T-shirt from a conveyor transporting heterogeneous waste items and to deposit it into a designated bin. The employed robotic system is realized in NVIDIA Isaac Sim and consists of three main subsystems: (i) a central tendon-driven arm section derived from an existing tendon-robot design, (ii) an articulated base that provides additional linear actuation in the y - and z -directions to expand the reachable workspace and to improve access to items on the conveyor, and (iii) a Robotiq gripper as the end-effector. The control policy is trained in IsaacLab using Proximal Policy Optimization (PPO) implemented with `skrl`. The overall learning objective is defined as a selective pick-and-throw behavior: *grasp the T-shirt, lift it from the moving belt, and reliably eject it into the target bin while leaving non-target waste items behind.*

The scope of this work focuses on the learning-based manipulation stack in simulation and its preparation for sim-to-real transfer. Perception is treated as an enabling component (e.g., via ground-truth labels in simulation or externally provided detections) rather than the primary research contribution. In the broader project context, this selective retrieval capability forms the foundation for future extensions towards *condition-based* T-shirt sorting, where policies must not only identify garments but also manipulate and present them in ways that enable inspection and quality assessment.

The thesis pursues the following objectives:

1. **High-fidelity simulation environment:** Develop a reproducible Isaac Sim scene that models a conveyor-based waste stream, a target bin, and representative waste items including a deformable T-shirt, with randomized initial conditions and physical parameters to support robust learning.
2. **Robot model integration and validation:** Integrate the custom robot (tendon-driven arm segment, linear y/z base actuation, and Robotiq gripper) into IsaacLab, and verify the correctness of kinematics, actuation interfaces, joint limits, and controller settings required for stable simulation-based training.
3. **MDP formulation of the manipulation task:** Formulate the problem as an MDP within the IsaacLab workflow by specifying observation space, action space, reward design, and termination criteria. The formulation must explicitly encode *selectivity* (T-shirt vs. non-target items) and *successful disposal* into the bin.
4. **Curriculum from basic skills to the final task:** Implement a staged training procedure that progresses from foundational behaviors (e.g., reaching, grasping, simple pick-and-place) to the full conveyor-based selective pick-and-throw task, enabling sample-efficient learning and reducing reward sparsity.
5. **Policy training and quantitative evaluation:** Train PPO policies using `skrl` and evaluate performance with metrics such as task success rate, false-pick rate (non-target retrieval), missed-pick rate (target left on conveyor), bin placement success, cycle time, and robustness under domain randomization (e.g., belt speed, friction, mass, cloth stiffness).
6. **Sim-to-real readiness and integration pathway:** Assess transfer robustness via noise and parameter randomization and define an executable integration pathway towards real-system deployment (e.g., Robot Operating System (ROS) 2 interfaces, logging, and safety constraints), identifying remaining gaps that must be closed for real-world operation.
7. **Foundation for condition-based sorting:** Derive requirements for extending the learned manipulation policy towards condition-based T-shirt sorting, including which additional observations, interaction primitives (e.g., stretching/presentation), and evaluation criteria would be needed in subsequent work.

1.3 Thesis Outline

This thesis is structured as follows. Chapter ?? introduces the fundamentals required for this work, covering reinforcement learning for robotic manipulation with an emphasis on PPO, deformable-object (cloth) manipulation challenges, and the simulation tools used in this project (Isaac Sim and IsaacLab). Chapter ?? describes the simulated system and task setup in detail: the conveyor scenario with mixed waste items, the selective retrieval objective, and the custom robot model consisting of the tendon-driven arm segment, the additional linear y/z base actuation, and the Robotiq gripper.

Chapter ?? formulates the learning problem as an MDP and explains the design of observations, actions, reward terms, and termination conditions to realize selective pick-and-throw behavior (retrieve only the T-shirt and deposit it into a bin). Chapter ?? presents the training procedure in IsaacLab using `skr1` PPO, including curriculum design from simpler sub-skills to the full conveyor task and the use of randomization to improve robustness.

Chapter ?? defines the experimental protocol and evaluation metrics, focusing on sorting-relevant criteria such as success rate (T-shirt in bin), false retrievals of non-target objects, misses of the target, and robustness under variations of object and conveyor properties. Chapter 4 reports the quantitative and qualitative results of the learned policies and analyzes typical success behaviors and failure cases.

Finally, Chapter 5 discusses the implications and limitations of the approach, including simulation fidelity and sim-to-real considerations, and Chapter ?? concludes the thesis and outlines future work towards condition-based T-shirt sorting, where retrieval is extended to classification and handling based on garment condition.

2 Background

2.1 Reinforcement Learning for Robotic Manipulation

Robotic manipulation requires generating control commands that achieve a goal while dealing with uncertainty, partial observability, contacts, and high-dimensional configuration spaces. Classical approaches rely on analytical models, trajectory optimisation, and carefully engineered feedback control. However, accurate models of frictional contact, deformable objects, or cluttered interactions are often difficult to obtain, and manually designing robust policies can be time-consuming. Reinforcement Learning (RL) provides an alternative: it enables learning control policies from interaction data by optimising a task objective, thereby reducing reliance on explicit modelling and hand-crafted strategies. General RL background and formalism are treated in standard texts such as Sutton and Barto [1], and robotics-specific perspectives (including practical constraints and common failure modes) are discussed in Kober *et al.* [2].

MDP viewpoint and partial observability. A manipulation task can be modelled as an MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(s_{t+1} \mid s_t, a_t)$ the transition dynamics, $r(s_t, a_t)$ the reward, and $\gamma \in [0, 1)$ the discount factor [1]. A policy $\pi_\theta(a \mid s)$ (parameterized by θ) induces trajectories $(s_0, a_0, r_0, \dots, s_T)$ and is trained to maximize the expected discounted return $J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$ [1]. In real manipulation systems, the full state s_t is typically not directly accessible; instead, control is based on observations o_t (e.g., proprioception, gripper state, pose estimates, perception features). This partial observability motivates careful observation design and, where appropriate, augmenting policies with memory (e.g., recurrent networks), or stacking observation histories to approximate the Markov property in practice [2].

Key challenges in manipulation. Three aspects commonly make RL for manipulation demanding: (i) **continuous, high-dimensional control**, where actions may correspond to joint torques, velocities, or Cartesian increments for many degrees of freedom; (ii) **contact-rich and nonlinear dynamics**, where small modelling errors in friction or contact impulses can lead to qualitatively different outcomes; and (iii) **sparse or delayed**

feedback, where success may only be measurable after long sequences of intermediate steps. Further practical constraints include safety, limited real-world interaction budgets, and the need for robustness across object variations and perception noise [2]. These characteristics influence algorithm choice, reward design, termination conditions, and the role of simulation during training.

Reward design and task specification. In RL, the reward function encodes the task objective. For manipulation, rewards are often composed of shaping terms (e.g., distance-to-target, alignment, grasp quality proxies) and sparse success signals (e.g., “object lifted above height”). Well-shaped rewards can accelerate learning but risk biasing the learned behavior toward unintended shortcuts; conversely, sparse rewards better reflect the true objective but can make exploration difficult. Robotics RL workflows therefore often iterate on reward definitions and termination criteria to balance learnability and task fidelity [2, 1].

2.1.0.1 Learning objectives and algorithm families

For continuous control, modern deep RL methods commonly use *policy gradient* and *actor-critic* approaches. Policy gradient methods directly optimise the policy parameters by ascending an estimate of $\nabla_{\theta} J(\theta)$, which naturally supports continuous actions through differentiable policy parameterizations [1]. Actor-critic methods introduce a learned critic (value function) that provides lower-variance learning signals for the actor, often improving stability and learning speed in practice [1]. A central quantity is the advantage function $A^{\pi}(s_t, a_t)$, which measures how much better an action is compared to the policy’s baseline behaviour at a state. In practice, advantage estimates $\hat{A}_t$ are computed from value predictions and collected rollouts, and then used to weight policy updates [1].

Bias-variance trade-off in advantage estimation. A practical challenge is balancing low-variance learning signals with accurate credit assignment over time. Generalised Advantage Estimation (GAE) provides a widely used mechanism to interpolate between Monte Carlo returns and bootstrapped temporal-difference estimates by exponentially weighting temporal-difference residuals [3]. This improves learning stability in many continuous-control applications and is commonly paired with actor-critic training.

2.1.0.2 Exploration and data collection in manipulation

A defining challenge in RL is the *exploration–exploitation trade-off*: the agent must explore sufficiently to discover high-reward behaviours while exploiting learned knowledge to improve performance [1]. In manipulation, exploration is complicated by (i) long horizons, (ii) sparse success signals, and (iii) irreversible or unsafe states (in real systems) [2]. Common practical measures include:

- **Stochastic policies and action sampling**, which induce exploratory behavior while still allowing gradient-based optimization [1].
- **Reward shaping and curricula**, where training starts from simplified conditions (e.g., easier initial object poses) and progressively increases difficulty to maintain a learnable signal [2].
- **Episode resets and termination design**, which affect state visitation and can substantially change the data distribution seen by the learner [2].

These aspects tie the learning algorithm closely to environment design: even a robust optimiser can fail if exploration is insufficient or if the reward/termination specification induces pathological behavior [2, 1].

2.1.0.3 Simulation-to-reality considerations

RL policies are frequently trained in simulation to avoid wear, risk, and slow data collection on real robots [2]. However, transferring policies to the real world is challenging due to the *reality gap*: discrepancies in geometry, contact parameters, actuator dynamics, sensor noise, and unmodeled effects can degrade performance. A common strategy is **domain randomization**, where visual and physical parameters are randomized during training so that the policy learns behaviors that are robust across a range of plausible conditions [4]. A closely related approach is **dynamics randomization**, which targets uncertainties in physical parameters (e.g., mass, friction, motor gains, delays) to improve transfer for control policies [5]. In practice, these methods motivate designing training environments that reflect not only the nominal task but also a distribution of plausible perturbations, thereby improving robustness of learned manipulation policies when deployed outside the simulator [2, 4, 5].

2.1.0.4 Proximal Policy Optimization (PPO)

Among actor–critic methods, PPO is a popular choice due to its robustness and simplicity. PPO performs iterative policy improvement by collecting trajectories with the current

policy and updating the policy with a surrogate objective that discourages overly large steps in policy space [6]. Define the importance ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$. PPO maximizes the clipped objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)], \quad (2.1)$$

where $\hat{A}_t$ is an advantage estimate (commonly computed with GAE) [3, 6]. Intuitively, clipping limits the incentive to change the policy too aggressively, which helps prevent performance collapse from unstable updates—a practical consideration in contact-rich manipulation where learning can be sensitive to abrupt policy changes [6, 2].

2.1.0.5 Why PPO is often used for robotic manipulation

From a robotics perspective, PPO has several properties that make it attractive in manipulation pipelines:

- **Stable learning dynamics:** The clipped objective yields conservative and typically reliable updates [6].
- **Compatibility with continuous actions:** PPO naturally supports Gaussian policies over continuous controls [1].
- **Practical training loop:** The rollout-and-update structure integrates well with simulator-based training and supports systematic iteration on reward and termination design [2].

These considerations motivate PPO as a strong baseline for manipulation, especially when training stability and predictable iteration cycles are prioritized.

2.1.0.6 Implementation in this thesis

In this thesis, PPO is used as the primary learning algorithm, together with GAE for advantage estimation, following standard actor-critic practice [3, 6]. The training loop is implemented with the `skrl` library, which provides modular PPO components (policy, value function, rollout storage, and optimization routines) and supports integration with simulator-based environments [7]. This allows the thesis to focus on environment formulation—observation design, action parameterization, reward shaping, termination conditions, and robustness measures such as randomization—while relying on a consistent and reproducible PPO implementation [7].

2.2 Physics Simulation in Isaac Sim 5.1

Physics-based simulation in Isaac Sim 5.1 is implemented as a layered stack that separates (i) scene and asset description, (ii) binding of these descriptions to a concrete backend, and (iii) numerical solvers that evolve the physical state over time. In the Omniverse ecosystem, physics properties are authored in Universal Scene Description (USD) through the `UsdPhysics` schema (engine-agnostic physics description) and `PhysXSchema` (backend-specific extensions), allowing assets and environments to be defined declaratively in the scene graph [8, 9]. During simulation, the `omni.physx` runtime parses the USD stage, constructs the corresponding physics scene, advances the simulation, and writes back poses and state to USD [9]. Isaac Sim 5.1 uses NVIDIA PhysX 5 as its primary physics backend, providing rigid-body dynamics, articulations, contacts, and particle-based phenomena, with optional Graphics Processing Unit (GPU)-accelerated pipelines for throughput-critical workloads such as robot learning experiments with many parallel environments [10, 11, 12].

A detailed understanding of the underlying physics models and numerical methods is required for two reasons. First, sim-to-real transfer is fundamentally limited by discrepancies between simulated and real dynamics (the *reality gap*), where unmodelled effects, incorrect parameters (e.g., friction and damping), and solver artefacts can lead to policies that exploit simulator-specific regularities [13, 14]. Second, in RL, the simulator acts as a data-generating process; thus, modelling and solver choices directly shape the transition distribution used for learning, influencing both policy quality and generalization [14, 15]. In practice, this implies that solver selection, time-step choice, contact modelling, and collision geometry approximations must be treated as part of the learning system design, not as interchangeable implementation details [15, 16].

Placeholder for *Isaac Sim physics stack (USD Physics / PhysXSchema*
→ omni.physx → PhysX 5 CPU/GPU solvers) image

Figure 2.1: Conceptual physics simulation stack in Isaac Sim 5.1: physics is authored in USD via `UsdPhysics` and `PhysXSchema`, interpreted by `omni.physx`, and simulated using PhysX 5 with optional GPU-accelerated pipelines.

2.2.1 Rigid Body Dynamics

Rigid-body simulation assumes that each object moves as a rigid transform, i.e., deformation is neglected and the state is represented by position/orientation and linear/angular

velocity. PhysX represents the simulated world as a set of actors (static, kinematic, dynamic) with attached collision shapes and material properties; dynamics are advanced under external loads (e.g., gravity), contact constraints, and joint constraints [11, 17]. In Isaac Sim, these concepts are exposed through USD physics schemas and their PhysX extensions, enabling rigid bodies, colliders, and constraints to be authored on scene primitives [10, 8].

Articulated bodies, joints, and robot mechanisms. Robotic manipulators are modelled as articulated mechanisms, i.e., rigid links connected by joints that constrain relative motion and optionally apply actuation (drives). PhysX provides an *articulation* abstraction that is simulated in reduced coordinates, where the configuration is parameterized by the root pose and joint coordinates rather than the world pose of each link [18]. This formulation is designed to improve numerical fidelity for mechanisms (e.g., reduced joint error and improved behavior under large mass ratios) compared to independently simulating rigid bodies connected by generic constraints [18]. In this thesis context, the tendon-driven arm and the attached gripper are simulated as chains of rigid links with joint constraints and joint drives, such that control inputs act on joint coordinates while the solver enforces inter-link constraints and contacts [18, 10]. PhysX also exposes explicit *articulation tendons* as additional constraint structures within an articulation, including fixed tendons (coupling joint coordinates) and spatial tendons (spring-damper and limit constraints defined over attachment trees across links), which are relevant when cable-like actuation or joint coupling must be represented at the physics level [18, 19].

Contacts and friction models. Rigid-body collision response is formulated as a constrained dynamics problem in which contacts prevent interpenetration and can transfer or dissipate energy through restitution and friction [20, 10]. In PhysX, friction and restitution are specified via materials attached to shapes; friction is parameterized by static and dynamic coefficients and can be combined for contacting pairs using configurable combine modes [17, 21]. Because contact-rich manipulation is common in robotic assembly, grasping, and textile handling, parameter choices for friction and restitution are directly coupled to learning outcomes and to sim-to-real robustness [14, 15].

TGS constraint solving in PhysX. Rigid-body dynamics with contacts and joints is solved in PhysX using iterative constraint solvers. PhysX supports a classic PGS-style solver and the newer TGS solver, with TGS introduced in PhysX 5.1 and recommended for improved convergence and joint drive accuracy in many scenarios [17]. Both solvers operate iteratively over constraint manifolds (contacts, joints, friction), and accuracy is

traded for performance by limiting iteration counts. A key motivation for temporal formulations is the observation that, for stiff constraint systems, replacing multiple solver iterations at a long time step by multiple smaller substeps can yield superior stability and accuracy at comparable computational cost [22, 16]. This perspective is consistent with PhysX guidance to consider smaller time steps (or substepping) when joint stability and contact behavior are insufficient under a fixed number of iterations, rather than indefinitely increasing iteration counts [16].

Simulation parameters and numerical tradeoffs. For robotics and RL, the most consequential rigid-body parameters are those controlling temporal discretization, solver convergence, and contact generation. The simulation time step h determines how frequently dynamics are integrated and how often the policy interacts with the environment; overly large h can lead to missed collisions, unstable joint behavior, and biased contact impulses, whereas smaller h improves accuracy at increased compute cost [23, 16]. Solver iteration counts control how many passes are made over constraints; PhysX allows per-actor iteration counts (position and velocity iterations), and the effective iterations for a connected island are determined by the maximum requested within that island [23, 17]. Contact generation is influenced by the pair `contactOffset` and `restOffset`: contact points are generated once shapes are within the combined contact distance, while the rest offset defines the target separation distance at rest, enabling predictive enforcement and reducing jitter at the cost of introducing a small separation bias [24]. Finally, friction coefficients (static/dynamic) and restitution parameters are specified in materials and substantially affect sliding, sticking, and impact behavior [21, 17].

Table 2.1: Selected rigid-body simulation parameters relevant to robot learning in Isaac Sim / PhysX.

Parameter	Location	Primary effect
Time step h	Physics scene	Accuracy vs. throughput; contact/joint stability
Solver iterations	Actor / island	Constraint convergence; joint error; contact quality
<code>contactOffset</code> / <code>restOffset</code>	Shape	Contact prediction and resting separation
Friction coefficients	Material	Sliding/sticking behavior; grasp stability [21]

GPU-accelerated rigid bodies for parallel RL environments. PhysX provides a GPU rigid-body pipeline that accelerates broad-phase, narrow-phase, contact generation, and constraint solving on CUDA-capable hardware, while exposing the same API concepts as the Central Processing Unit (CPU) pipeline [12]. Large performance gains are expected when many rigid bodies are active, which aligns with robot learning workloads

where hundreds to thousands of environment replicas are simulated in parallel to increase sample throughput [12, 25]. In IsaacLab, PhysX GPU simulation is enabled by default and can be configured explicitly; however, GPU simulation requires preallocation of internal buffers and may fail when insufficient capacities are specified, which makes sizing and profiling part of simulation configuration for large-scale training [26]. The broader motivation for GPU-resident simulation in RL is that physics stepping and policy learning can share device memory, avoiding CPU–GPU transfer bottlenecks; this design has been demonstrated to yield substantial throughput improvements in GPU-based robot learning platforms [25].

Collision detection and geometry approximations. Collision detection cost is highly sensitive to the chosen collider representation. Primitive shapes (e.g., spheres, boxes, capsules) provide the strongest performance and numerical robustness because intersection tests and contact generation are simple and well-conditioned [20]. Convex meshes provide a practical middle ground for complex objects by enabling efficient collision queries while approximating concave geometry; PhysX requires *convex mesh cooking* to transform input geometry into an optimized representation for collision detection [27]. In Isaac Sim, mesh colliders are often approximated by convex hulls by default for performance, with more expensive approximations available when needed [10]. When concavity is important for dynamic objects, convex decomposition (a set of convex parts) is typically preferred over a single triangle mesh collider because triangle-mesh contact generation is expensive and, depending on pipeline and version, may require additional structures. In PhysX 5.3+, dynamic triangle meshes in the GPU collision pipeline require an associated Signed Distance Field (SDF) to accelerate collision detection, increasing memory use but improving query performance [28]. Therefore, collider selection becomes a modelling choice that trades geometric fidelity against computational cost and solver stability, and this tradeoff must be balanced against the throughput requirements of RL training.

2.2.2 Cloth and Deformable Object Simulation

Deformable object simulation relaxes the rigid-body assumption by allowing the object’s shape to change under load. This introduces two coupled complexities: the state dimension increases dramatically (many vertices/particles instead of a single rigid transform), and contacts become configuration-dependent because deformation continuously alters the set of potential contact points and normals [29, 30]. For textiles, this coupling is particularly pronounced: applied forces not only translate the object but also create folds, wrinkles, and contact-rich self-interactions that alter future dynamics. These properties are repeatedly

identified as key contributors to the reality gap in robotic cloth manipulation [30].

Common modelling approaches. Three modelling families are most relevant in robotics simulation. First, mass–spring(–damper) models discretize cloth as a mesh whose vertices are connected by springs encoding stretch and (optionally) bending penalties; such models can be made stable with implicit integration techniques but require careful parameterization and can be numerically stiff [31]. Second, particle-based constraint methods represent cloth by particles and enforce desired behavior via geometric (holonomic) constraints, which can be solved iteratively with high robustness under collisions; Position-Based Dynamics (PBD) and its variants are canonical examples [32, 33]. Third, continuum formulations discretize elastic energy and constitutive laws (e.g., in Finite Element Method (FEM)) or use hybrid particle–grid methods (e.g., Material Point Method (MPM)), providing a pathway to parameters with clearer physical interpretation at increased computational cost [34, 35].

Challenges specific to cloth dynamics. Cloth exhibits high effective Degree of Freedom (DoF), frequent self-collision, and strong sensitivity to material parameters such as bending stiffness, friction, and damping. Robust self-contact and friction handling is widely recognized as a hard requirement for stable cloth simulation, particularly under large deformations and repeated collisions [36, 30]. From a learning perspective, this sensitivity implies that small modelling errors (e.g., in friction or damping) can change grasp success rates, sliding behavior, and fold formation, which in turn can qualitatively alter the learned policy’s strategy and its transferability to hardware [14, 30].

2.2.2.1 PBD — PhysX Particle Cloth

PhysX provides particle-based simulation through `PxPBDParticleSystem`, whose solver is explicitly described as using PBD for particle–particle dynamics and as supporting phenomena including cloth, inflatables, and other deformable behaviors [37, 38]. In Omniverse, particle simulation is GPU-accelerated and is documented as supporting fluids, granular media, and cloth, including interaction with rigid bodies and articulations [39]. At the feature level, particle cloth is treated as a particle-based PBD mass–spring-style system and is marked as deprecated in favor of newer deformable-body features, which highlights that “cloth” in Isaac Sim can refer to multiple, evolving solver families rather than a single canonical model [39, 40].

Time-step structure and iterative projection. In standard PBD, the deformable is represented by particle positions $\mathbf{x}_i \in \mathbb{R}^3$ with masses m_i ; desired behavior is enforced by a set of constraints $C_j(\mathbf{x})$ (equalities or inequalities), typically solved by iterative projection in a Gauss–Seidel fashion [32]. A schematic time step consists of: (i) updating velocities under external forces, (ii) predicting positions by an explicit step, (iii) iteratively projecting predicted positions to reduce constraint violations, and (iv) updating velocities from the corrected positions (and applying damping if configured) [32]. For a constraint $C_j(\mathbf{x}) = 0$, linearization yields a correction direction based on the constraint gradient. Using the inverse mass matrix $\mathbf{M}^{-1}$ (diagonal in the particle basis), one common update can be written as [32, 33]:

$$\Delta \mathbf{x} = k_j \cdot s_j \cdot \mathbf{M}^{-1} \bullet C_j(\mathbf{x}), \quad s_j = \frac{-C_j(\mathbf{x})}{\bullet C_j(\mathbf{x})^\top \mathbf{M}^{-1} \bullet C_j(\mathbf{x})}, \quad (2.2)$$

where $k_j \in [0, 1]$ is a user-defined stiffness-like scaling applied per constraint.

Effective stiffness and dependence on h and iteration count. A central practical implication of the iterative projection view is that the *effective* stiffness of constraints depends on numerical settings. Increasing the number of projection iterations typically yields stiffer constraint satisfaction at fixed h , while increasing h tends to reduce stability and can require either more iterations or smaller substeps to maintain comparable behavior [32, 33, 22]. This is beneficial for robustness and controllability, but it complicates mapping simulator parameters to physically measured material properties (e.g., bending stiffness in SI units), which is a recurring issue when calibrating cloth simulation for sim-to-real transfer [33, 14].

XPBD compliance formulation. XPBD extends PBD by introducing a compliance parameter and an explicit Lagrange multiplier state to reduce time-step dependence and recover meaningful constraint force estimates [33]. For compliance $\alpha \geq 0$ and time step h , the incremental multiplier update for a constraint can be expressed as [33]:

$$\Delta \lambda = \frac{-C(\mathbf{x}) - \frac{\alpha}{h^2} \lambda}{\sum_i w_i \|\bullet_{\mathbf{x}_i} C(\mathbf{x})\|^2 + \frac{\alpha}{h^2}}, \quad \Delta \mathbf{x}_i = w_i \bullet_{\mathbf{x}_i} C(\mathbf{x}) \Delta \lambda, \quad (2.3)$$

with inverse masses $w_i = 1/m_i$. This formulation decouples the user parameter α from the iteration count in a way that improves interpretability of stiffness-like behavior across different h and solver settings [33].

Why PBD is used in PhysX for particle cloth. PBD is well-aligned with GPU execution because constraints can be partitioned into independent sets and processed in parallel; graph coloring is a common strategy to schedule constraint batches without write conflicts on shared particles [41]. Omniverse documentation explicitly emphasizes GPU-accelerated PBD particles and their interaction with rigid bodies and articulations, which is essential for cloth manipulation scenes in robotics [39]. In addition, the PhysX particle system API is designed around a dedicated PBD solver type (`PxBBDParticleSystem`), reinforcing that particle cloth is treated as a specific, robust, high-throughput model family within the PhysX multiphysics portfolio [37, 38].

2.2.2.2 VBD — Newton Thin Deformables

While PhysX particle cloth is based on explicit prediction and iterative position projection, implicit methods aim to improve stability under stiffness and large time steps by solving a time-discrete optimization problem. VBD is introduced as a block coordinate descent solver for the variational form of implicit Euler time integration, targeting convergence to the implicit Euler solution while maintaining unconditional stability under limited iteration budgets [42].

Variational implicit Euler and global energy minimization. Let $\mathbf{x}_t$ denote positions, $\mathbf{v}_t$ velocities, h the time step, and $\mathbf{M}$ the mass matrix. In the variational formulation, the next positions are obtained by minimizing a global energy that combines inertia and potential energy [42]:

$$\mathbf{x}_{t+1} = \arg \min_{\mathbf{x}} G(\mathbf{x}), \quad G(\mathbf{x}) = \frac{1}{2h^2} \|\mathbf{x} - \mathbf{y}\|_{\mathbf{M}}^2 + E(\mathbf{x}), \quad (2.4)$$

where $\mathbf{y} = \mathbf{x}_t + h\mathbf{v}_t + h^2\mathbf{a}_{\text{ext}}$ and $E(\mathbf{x})$ is the total potential energy. Velocities follow from the implicit Euler relationship $\mathbf{v}_{t+1} = (\mathbf{x}_{t+1} - \mathbf{x}_t)/h$ [42].

Vertex-level block coordinate descent and local Newton step. VBD solves the global problem by iterating over vertices and updating one vertex at a time while temporarily holding the others fixed, yielding a vertex-level Gauss–Seidel pattern with favorable parallelization via vertex coloring [42]. For vertex i , a local energy is minimized:

$$\mathbf{x}_i \leftarrow \arg \min_{\mathbf{x}_i} G_i(\mathbf{x}), \quad G_i(\mathbf{x}) = \frac{m_i}{2h^2} \|\mathbf{x}_i - \mathbf{y}_i\|^2 + \sum_{j \in F_i} E_j(\mathbf{x}), \quad (2.5)$$

where F_i is the set of force/energy elements incident to vertex i . Because G_i has only three degrees of freedom, a local Newton step is formed by solving a 3×3 linear system per vertex [42]:

$$\mathbf{H}_i \Delta \mathbf{x}_i = \mathbf{f}_i, \quad (2.6)$$

with $\mathbf{H}_i$ the local Hessian and $\mathbf{f}_i$ the local gradient/force term.

Advantages for stiff, contact-rich thin deformables. The primary advantages highlighted for VBD are: (i) unconditional stability inherited from the energy descent property, (ii) convergence toward the implicit Euler solution as iterations increase, and (iii) GPU-oriented parallelization via vertex coloring with small, localized linear solves [42]. These properties are attractive for cloth-like thin deformables, which often exhibit near-inextensible stretch (high stiffness) combined with softer bending and frequent contact events; implicit/variational formulations can tolerate such stiffness ratios at larger time steps than explicit solvers with comparable iteration budgets [42].

Newton in the robotics simulation ecosystem. Newton is presented as an open-source, GPU-accelerated physics engine designed for robotics simulation and robot learning, built on NVIDIA Warp and intended to integrate with robot learning frameworks such as IsaacLab [43, 44]. In NVIDIA’s robotics communication, Newton is explicitly associated with thin-deformable simulation via a VBD solver and with multiphysics capabilities that include implicit MPM and other solver components, motivated by stability and performance for learning workflows [45, 44]. Accordingly, VBD is positioned as a solver choice when increased stability under stiff materials and larger time steps is required, complementing the high-throughput PBD-based particle cloth pathway in PhysX-centric stacks [45, 42].

2.2.2.3 Comparison: PBD vs. VBD for Cloth in RL Workflows

Mathematical targets and implications. The core distinction is the optimization target. PBD advances dynamics by predicting positions and then *projecting* them to reduce constraint violations, where the resulting behavior depends on solver scheduling and iteration count, and where constraint “stiffness” is typically not time-step invariant without extensions [32, 33]. VBD, in contrast, directly minimizes the variational implicit Euler energy and therefore targets a principled implicit integration objective; stability is maintained by construction via energy descent, and increased iteration budgets refine the approximation toward the implicit solution [42].

Practical tradeoffs for learning-centric simulation. For RL, PBD-based particle cloth is attractive because it is simple, robust under frequent contacts, and aligned with GPU-parallel execution through constraint batching and coloring strategies; this supports the high environment throughput required for policy optimization [39, 41, 25]. However, because effective stiffness depends on h and iteration count, parameter tuning can be difficult to interpret physically and can amplify sim-to-real sensitivity if policies rely on artefacts (e.g., overly damped folds or under-resolved frictional sticking) [33, 14]. VBD is attractive when stability under stiffness, large time steps, and mixed constraint types is prioritized, particularly for systems that combine near-inextensible stretch with softer bending; such regimes are common in cloth manipulation and can be challenging for explicit projection methods at fixed compute budgets [42, 45].

Relation to the Omniverse ecosystem and thesis choice. In Omniverse/Isaac Sim, particle cloth is documented as deprecated in favor of newer deformable-body features, indicating an ongoing transition toward more general deformable schemas and alternative solver families [39, 40]. For the experiments in this thesis, the PhysX/PBD particle cloth pathway is motivated by GPU throughput and robustness for large-scale training in Isaac Sim 5.1, whereas Newton/VBD is conceptually aligned with scenarios where greater stability under stiff cloth behavior and larger time steps is required (e.g., when cloth parameters or contact conditions become numerically challenging for projection-based solvers) [39, 42, 45].

2.3 NVIDIA Isaac Sim and IsaacLab Architecture

NVIDIA Isaac Sim (version 5.1) is used as the primary simulation environment in this thesis because it integrates robotics-oriented scene authoring, high-fidelity multi-physics simulation, and sensor-realistic rendering within a single Omniverse-based application stack [46]. In addition, the associated IsaacLab framework is used to structure the learning and evaluation workflow, because it provides a unified, modular interface for designing robotics tasks as MDPs and training policies at scale using GPU-accelerated parallel simulation [47]. The resulting toolchain aims to minimize integration overhead between simulation fidelity (e.g., cloth-rigid interactions), robotic middleware interfaces, and the training pipeline used for robot learning.

2.3.1 Isaac Sim in the Omniverse application stack

Isaac Sim is implemented as a reference application on top of NVIDIA Omniverse and is designed to support development, simulation, and testing of AI-driven robots in physically based virtual environments [46]. A central architectural decision is the use of USD as the scene representation and interchange format: robotics assets and environments are expressed as a hierarchical scene graph with reusable primitives, enabling non-destructive composition and scalable instancing workflows [46]. This representation is used as the common “data layer” across physics, rendering, and robotics-specific tooling, which reduces interface mismatches between asset creation and simulation-time execution.

Isaac Sim is built on Omniverse Kit, which is a modular application framework based on lightweight plugins and an extension mechanism for composing functionality [46]. Consequently, Isaac Sim capabilities (e.g., physics interfaces, sensor simulation, and robotics middleware bridges) are exposed as extensions that can be enabled, configured, and scripted from Python, which supports both interactive development in the graphical application and headless execution in automated pipelines [46]. This extension-centric architecture is relevant to robot learning workloads because it enables the simulator to be configured as a reproducible experiment runtime, rather than as a monolithic interactive tool.

Placeholder for *layered architecture of Isaac Sim + IsaacLab* image

Figure 2.2: Schematic view of the software stack used in this thesis. Isaac Sim is built on Omniverse Kit with USD as the scene data layer and PhysX 5 as the physics backend; RTX-based rendering is used for sensor simulation. IsaacLab is implemented on top of Isaac Sim and provides task abstractions (e.g., manager-based MDP environments) and training utilities (e.g., multi-GPUs execution).

2.3.2 Core simulation features in Isaac Sim 5.1

Isaac Sim combines high-fidelity physics, photorealistic rendering, and robotics integration within a single runtime [46]. These features are not independent: physics fidelity affects contact-rich manipulation, rendering realism affects perception and sim-to-real transfer, and middleware integration affects how simulation is connected to external robot software stacks. The most relevant capabilities for this thesis are summarized below.

- **Omniverse and USD-centric scene representation:** Scenes are represented in USD, which serves as the unifying data interchange format for robot assets,

environments, and simulation metadata [46]. This enables structured scene graphs and supports scalable instancing patterns for large simulation workloads.

- **High-fidelity physics based on PhysX 5:** Isaac Sim uses a GPU-based PhysX engine and provides access to rigid-body and articulated dynamics required for robotics simulation [46]. GPU rigid-body simulation in PhysX provides GPU-accelerated broad phase, contact generation, and constraint solving when enabled, while other components remain on the CPU [12].
- **Integrated deformables and cloth-relevant dynamics:** Omni PhysX exposes GPU-accelerated particle simulation based on PBD, which supports effects including cloth-like particle systems and their interaction with articulations and other physics objects [39]. This is directly relevant for tasks where cloth dynamics and contact interactions must be represented in the same simulation runtime.
- **Photorealistic RTX-based sensor simulation:** Isaac Sim supports multi-sensor RTX rendering and uses GPU access to simulate sensors such as cameras and lidar in an industrially scalable manner [46]. RTX-based sensor simulation is exposed via dedicated extensions (e.g., RTX sensors) within the Kit extension framework [48].
- **Robotics middleware support via ROS 2 bridging:** A dedicated ROS 2 Bridge extension is provided to publish and subscribe to ROS 2 topics and services from within Isaac Sim, enabling bidirectional communication between simulation and external ROS 2 systems [48].

2.3.2.1 GPU acceleration and parallel simulation

GPU acceleration in the Isaac ecosystem is achieved at multiple levels, ranging from physics backend execution to batched simulation of many environments. At the physics-backend level, PhysX GPU rigid bodies are configured at the scene level and enable GPU-accelerated rigid-body pipeline components including broad phase, contact generation, and the constraint solver [12]. Since not all features are executed on the GPU, the performance benefit depends strongly on scene composition and on whether large numbers of active bodies are simulated concurrently [12]. This architecture is aligned with robot learning workloads in which many similar environments are executed in parallel to increase sample throughput.

At the application level, Isaac Sim exposes explicit support for cloning and replicating environments. In particular, the `isaacsim.core.cloner` extension is provided to clone prims and environments efficiently and to optionally filter collisions across clones, which is a practical requirement when many environments share a single process runtime [48]. This cloning functionality forms a key bridge between a high-fidelity simulator and the

throughput requirements of RL, since scalable policy training is typically bottlenecked by the rate of environment rollouts rather than by single-scene fidelity.

2.3.3 IsaacLab as a unified robot learning framework

IsaacLab is used in this thesis as the primary framework for structuring robot learning experiments on top of Isaac Sim. IsaacLab is described as the successor to Isaac Gym and is intended to extend GPU-native simulation into large-scale, multi-modal robot learning, while remaining integrated with Isaac Sim for high-fidelity physics and RTX-based sensing [47]. The framework is designed to unify workflows spanning RL, imitation learning, and motion planning, while exposing reusable abstractions for robot assets, environments, and training pipelines [47].

A key motivation for IsaacLab is the unification of two prior lines of development in the Isaac ecosystem. First, Isaac Gym demonstrated that physics simulation and policy learning can be executed entirely on the GPU with direct tensor access, yielding large speedups over CPU-based simulators when training complex robot policies [25]. Second, the Orbit framework established a modular environment library on top of Isaac Sim, providing benchmark tasks (including cloth folding) and supporting policy training and demonstration collection using GPU-based parallelization [49]. IsaacLab is positioned as a consolidated framework that carries forward the GPU-native paradigm of Isaac Gym and the modular task and environment design principles of Orbit, but within an Isaac Sim-integrated runtime that supports richer sensing and broader multi-physics capabilities [47].

2.3.3.1 Manager-based MDP workflow and code reuse

IsaacLab structures simulation tasks explicitly as MDPs, i.e., as environments that provide state (or observations), actions, and reward/termination signals suitable for learning. Within the manager-based workflow, a base environment is composed from reusable managers (e.g., action, observation, and event managers) and is then extended to an RL task environment by adding reward, termination, command, and curriculum components [50]. This decomposition is relevant for thesis-scale experimentation because it separates environment implementation from task specification: instead of modifying core environment classes directly, task-specific behavior is encoded in configuration objects, which enables consistent reuse across tasks and reduces duplication across experimental variants [50].

The manager-based formulation also encourages a stable interface between simulation and training. In the documented workflow, the `ManagerBasedRLEnv` interface returns MDP-relevant signals (including reward and termination status) while remaining compatible

with standard environment creation patterns (e.g., via `gymnasium.make`), which supports integration with multiple RL libraries without rewriting the simulator interface layer [50]. The resulting abstraction is intended to enable systematic experimentation across robot morphologies and task configurations while preserving a consistent training loop.

2.3.3.2 Distributed training support in IsaacLab

Robot learning in high-dimensional, contact-rich tasks often requires a large number of environment interactions, which motivates distributed execution. IsaacLab provides explicit support for multi-GPU and multi-node RL training, with documented workflows using PyTorch distributed tooling (e.g., `torchrun`) and a process-per-GPU execution model in which each process instantiates its own IsaacLab environment and synchronizes gradients across processes [51]. This design is aligned with on-policy RL settings, in which large batch sizes and parallel rollouts can substantially increase training throughput, provided that simulator stepping and observation extraction can be executed efficiently.

The documented multi-GPU pipeline is also operationally relevant: each process is responsible for its own environment instance and rollout collection buffers, while synchronization occurs only during gradient updates [51]. This reduces coupling between simulation instances and avoids centralized simulation bottlenecks, at the expense of higher per-process resource usage. For thesis experiments that require extensive policy iteration or hyperparameter exploration, this design provides a practical path toward scaling without abandoning the integrated Isaac Sim runtime.

2.3.3.3 Imitation learning and motion planning integration

Beyond RL, IsaacLab supports imitation learning workflows that combine human demonstration segments with automated planning. The SkillGen pipeline is documented as integrating motion planning into the IsaacLab Mimic demonstration workflow, thereby generating collision-free trajectories between skill segments using an external planner [52]. In this workflow, `cuRobo` is used as the underlying motion planner for generating smooth, collision-free motions, and the planner is explicitly described as GPU-accelerated [52].

`cuRobo` itself is formulated as a GPU-parallel motion generation approach for manipulators, with a design that exploits massive parallelism (e.g., many optimization seeds and parallel collision checks) to achieve low-latency planning and inverse kinematics computations [53]. The availability of such a planner within the IsaacLab ecosystem is relevant for tasks where expert trajectories must be generated efficiently, either to bootstrap learning (e.g., as demonstrations) or to define hybrid learning-and-planning pipelines.

2.3.4 Rationale for selecting Isaac Sim 5.1 with IsaacLab

The combined selection of Isaac Sim 5.1 and IsaacLab is driven by the need to simultaneously address (i) deformable/cloth dynamics, (ii) contact-rich manipulation in articulated robots, and (iii) scalable learning infrastructure. These requirements are tightly coupled in the targeted thesis use case: cloth manipulation requires reliable coupling between deformable dynamics and rigid contacts, while learning-based policies require large numbers of rollouts to converge.

Integrated cloth and deformable dynamics are supported within the Isaac Sim / PhysX stack through GPU-accelerated particle-based simulation that can represent cloth-like behavior and interact with articulations and other physics objects within a shared simulation scene [39]. In addition, IsaacLab explicitly highlights support for deformable objects (including cloth) within a framework built on Isaac Sim and designed for multi-modal learning, indicating that deformable interactions are treated as a first-class concern in the simulation-to-learning pipeline [47]. This integration is beneficial because it avoids ad-hoc coupling between an external deformable simulator and a separate rigid-body engine, which would otherwise require custom synchronization, contact handling, and state transfer between solvers.

Scalability is addressed through GPU-native simulation and distributed training utilities. Isaac Gym established that GPU-resident simulation and learning with direct tensor access can yield large improvements in end-to-end training throughput compared to CPU-based simulation paired with GPU-based neural network training [25]. IsaacLab extends this paradigm within an Isaac Sim-integrated stack and provides documented support for multi-GPU and multi-node training, enabling scale-out execution for RL workloads when training is bottlenecked by rollout collection [47, 51]. When combined with Isaac Sim cloning utilities for replicated environments [48], the simulator can be configured to support large-scale parallel rollouts within a consistent runtime, which is particularly advantageous in on-policy scenarios where sample throughput dominates wall-clock training time.

Finally, robotics integration requirements are addressed at the middleware level. Isaac Sim exposes a ROS 2 Bridge extension for publishing and subscribing to ROS 2 topics and services through OmniGraph nodes and action graphs [48]. This bridge supports a common development pattern in which simulation is used for software-in-the-loop or hardware-in-the-loop testing, and it provides a practical interface to connect simulation-time controllers and perception to external robot software stacks without bespoke transport layers.

2.3.5 Comparison to alternative simulation platforms

The choice of simulator is constrained by trade-offs between fidelity, performance, and integration effort. For this thesis, the critical differentiator is the ability to combine contact-rich articulated manipulation with cloth dynamics and scalable learning infrastructure within a unified and scriptable runtime. Alternative platforms are briefly contrasted below to justify the selection of Isaac Sim 5.1 with IsaacLab for this specific use case.

MuJoCo is a widely used physics engine for model-based control that emphasizes efficient dynamics and contact simulation, and it is commonly adopted as a baseline simulator in continuous-control research [54]. Its design makes it well-suited for rigid-body control benchmarks and for iterative controller development when the primary bottleneck is per-step simulation performance. However, the thesis use case requires integrated cloth dynamics and sensor-realistic rendering within the same architecture, and these requirements typically necessitate additional tooling beyond a core rigid-body simulator.

PyBullet is widely used as a Python-accessible simulation interface built around the Bullet physics engine, and it is often adopted in robotics prototyping due to its ease of scripting and broad ecosystem integration [55]. The underlying Bullet engine is documented in the context of real-time physics simulation and is frequently used in interactive applications [56]. In practice, PyBullet is frequently selected when rapid iteration and lightweight deployment are prioritized, but the simulator stack used in this thesis additionally requires an integrated high-fidelity rendering and sensor simulation pipeline and a unified large-scale training framework, which are provided natively by Isaac Sim and IsaacLab rather than assembled from separate components.

SOFA is an open-source simulation framework primarily targeted at medical simulation and is designed around an advanced software architecture for building complex deformable simulations by composing algorithms, solvers, and collision models [57]. This makes SOFA a strong candidate for high-fidelity deformable modeling, including scenarios where accurate soft-tissue mechanics are required. Nevertheless, the overall thesis workflow requires not only deformable simulation but also scalable robot learning infrastructure with high-throughput parallel environment execution and integrated robotics middleware interfaces, which are not the primary design objectives of SOFA and typically require additional integration layers.

Overall, Isaac Sim 5.1 with IsaacLab is selected because it provides an integrated software architecture that aligns with the requirements of cloth manipulation and scalable robot learning. The Isaac stack combines a USD-based scene representation, PhysX-backed multi-physics simulation, RTX-based sensing, and robotics middleware integration in a

Table 2.2: Qualitative comparison of simulators with respect to thesis requirements.

Platform	Primary strength (per source)
Isaac Sim + IsaacLab	Integrated Omniverse stack with PhysX, RTX, and RL tooling [46, 47]
MuJoCo	Efficient physics engine for model-based control [54]
PyBullet/Bullet	Accessible real-time physics simulation ecosystem [55, 56]
SOFA	Modular framework for complex deformable simulation [57]

single Omniverse runtime [46, 48]. IsaacLab further provides a principled MDP-based task design workflow and documented multi-GPU/multi-node training support, which reduces the engineering effort required to perform large-scale learning experiments while retaining simulation fidelity [50, 51, 47].

2.4 Tendon-Driven Robot Kinematics and Modeling

Tendon-driven robots are articulated mechanisms in which joint motion is generated by tensile elements (tendons) that transmit forces from remote actuators to the joints through pulleys, guides, or routing channels. In this actuation paradigm, the actuators and transmissions can be placed proximally, while the distal structure remains lightweight. This design is widely used in continuum and cable-driven mechanisms, particularly when low distal inertia, compact joint packaging, or intrinsic compliance is desired [58, 59]. At the same time, the mechanical transmission characteristics of tendons introduce additional modeling complexity. The mapping between actuator commands and joint motion depends on geometry (routing and moment arms), tendon elasticity, and non-idealities such as friction and hysteresis, which are frequently reported as dominant error sources in cable- and tendon-driven systems [60, 61].

2.4.1 Tendon-driven actuation principles

In tendon-driven architectures, actuation is typically realized by commanding tendon displacement (spooling length) or tendon tension. For common antagonistic arrangements, a joint is actuated by at least two tendons such that differential tension generates a net joint torque while co-contraction modifies apparent stiffness. When tendons span multiple joints, a single tendon displacement can induce coupled motions, which is advantageous for compact designs but complicates decoupled joint control [58]. The mechanical benefit of remote actuation is that motors and gear reductions can be located near the base,

reducing distal mass and inertia and thereby enabling higher angular accelerations and dynamic performance for a given torque budget [59].

The same properties that reduce distal inertia also reduce transmission rigidity. Tendon stretch, cable-pulley compliance, and frictional effects can cause configuration-dependent motion errors, phase lags, and hysteresis loops under load. These effects are not only relevant for force control but also for kinematic accuracy because they break the assumption of a unique, memoryless mapping between actuator displacement and end-effector pose [60, 61]. As a result, tendon-driven systems are often controlled using either (i) local joint sensing (e.g., encoders placed near joints) to close the loop in joint space, or (ii) model-based compensation and identification to improve the actuator-to-joint mapping, depending on the available sensing and the targeted accuracy [60].

2.4.2 Kinematic modeling: from tendon space to task space

Rigid-link jointed mechanisms. For tendon-driven mechanisms built from rigid links and discrete joints, the kinematic structure in task space is formally identical to that of a direct-drive serial manipulator once the joint coordinates are defined. The principal difference is that the commanded variables are naturally expressed in tendon space (tendon lengths or tensions) rather than joint space. Consequently, the overall mapping is typically decomposed into two stages: (i) a transmission model from tendon variables to generalized joint coordinates, and (ii) a conventional rigid-body forward kinematic chain from joint coordinates to end-effector pose. In practice, the transmission model is required because routing geometry and tendon elasticity determine how a commanded spool displacement converts into a joint angle, and multi-joint tendons can create coupling terms that are absent in direct-drive designs [60].

Continuum and cable-driven bending mechanisms. For tendon-driven continuum robots, joint coordinates are not associated with discrete revolute joints but with distributed deformation of a flexible backbone. A widespread modeling approach is the piecewise constant-curvature assumption, in which the backbone is approximated as a sequence of constant-curvature arcs. This assumption reduces an effectively infinite-dimensional shape description to a finite set of arc parameters, enabling closed-form forward and differential kinematics for many designs [62]. The assumption is frequently used as a baseline model and remains attractive for control and planning because it yields tractable Jacobians and efficient computations. However, tendon friction and hysteresis can substantially degrade the predictive accuracy of purely geometric constant-curvature mappings,

motivating extended models that incorporate frictional memory effects and tension propagation along the tendons [61].

Implications for inverse kinematics and control. In tendon-driven systems, the Inverse Kinematics (IK) problem is frequently formulated in actuator space (tendon lengths) rather than in joint space. Even if a rigid-link IK solution exists in terms of joint angles, the implementation of that solution on real tendon-driven hardware can be limited by transmission nonlinearities. In particular, hysteresis implies that the same tendon command can yield different end-effector poses depending on motion history and external loading, such that a purely kinematic IK solution may not be repeatable without feedback [61]. For learning-based control pipelines, this motivates the separation between an *idealized kinematic model* used for planning or workspace analysis and a *simulation/control model* that includes compliance and control bandwidth limitations to represent actuation uncertainty [60].

2.4.3 Robot configuration used in this project

The robotic system considered in this thesis combines a virtual positioning stage with a tendon-driven arm and a parallel-jaw gripper. The complete assembly is modeled as a serial chain in which a planar repositioning stage provides coarse positioning of the arm base, and a tendon-driven wristed arm provides dexterous local manipulation. The design builds on the tendon-driven robot concept and mechanical arm design described by Klein [63].

Kinematic structure. The manipulator is composed of three modules: (i) a 2-DoF linear stage modeled as two orthogonal prismatic joints, (ii) a 3-DoF tendon-driven arm modeled as three revolute joints, and (iii) a two-finger gripper attached at the wrist flange for grasping. For workspace and dexterity measures that depend on end-effector pose, the gripper opening is not treated as a kinematic DoF affecting pose (it affects grasp state rather than tool pose) and is therefore excluded from the pose workspace computation.

A minimal generalized coordinate vector for pose-related analysis is defined as

$$\mathbf{q} = \begin{bmatrix} q_y & q_z & q_1 & q_2 & q_3 \end{bmatrix}^T,$$

where q_y and q_z denote the prismatic stage displacements and q_1, q_2, q_3 denote the arm revolute joint angles (e.g., one proximal bending joint and two wrist tilt axes). The joint

limits bound the feasible configuration space and are imposed explicitly in the workspace analysis in subsection 2.4.6.

Placeholder for *robot kinematic chain schematic* image

Figure 2.3: Schematic of the kinematic chain used for modeling and workspace analysis: two orthogonal prismatic joints for base repositioning, followed by a three-revolute-joint tendon-driven arm and an end-effector frame located at the gripper mounting interface.

Table 2.3: Pose-relevant degrees of freedom for workspace analysis. Joint limit values are defined by the robot specification and are applied during sampling.

Joint	Type	Joint limits
q_y (stage)	prismatic	$[q_{y,\min}, q_{y,\max}]$
q_z (stage)	prismatic	$[q_{z,\min}, q_{z,\max}]$
q_1 (arm)	revolute	$[q_{1,\min}, q_{1,\max}]$
q_2 (arm)	revolute	$[q_{2,\min}, q_{2,\max}]$
q_3 (arm)	revolute	$[q_{3,\min}, q_{3,\max}]$

Rationale for the modular decomposition. The stage–arm decomposition separates coarse positioning from local dexterity. From a modeling viewpoint, this separation is advantageous because it allows the tendon-driven arm (which is sensitive to transmission compliance and cable routing) to be analyzed both with and without the additional stage DoFs. In addition, it supports environment design and task placement by making the reachable workspace tunable through stage limits without changing the arm geometry [63].

2.4.4 Simulation considerations for tendon-driven systems in Isaac Sim

Tendon-driven mechanisms pose specific simulation challenges because their behavior depends on cable routing, compliance, and frictional effects that are not captured by joint kinematics alone. In USD-based simulation pipelines, these effects must either be represented explicitly using tendon/cable primitives or approximated through coupled joints and compliant actuation.

Articulation tendons and coupled DoFs. Omniverse Physics (PhysX backend) provides an articulation tendon concept with two main variants: fixed tendons (coupling joint

coordinates) and spatial tendons (imposing constraints on distances between selected links through attachment points). These constructs are intended to represent coupled joint relationships and cable-like constraints inside articulated mechanisms [64]. In principle, joint coupling through fixed tendons can emulate certain tendon-driven topologies by enforcing linear relationships between generalized coordinates. Spatial tendons can represent a cable path between attachments but still abstract away detailed contact mechanics inside routing guides [64].

URDF mimic joints as a pragmatic approximation. When a robot is imported via Unified Robot Description Format (URDF), coupled motion can be encoded by mimic joints. The Isaac Sim URDF importer can apply the PhysX mimic joint mechanism to joints carrying a mimic tag, enabling linear coupling between a reference joint and a mimicking joint [65]. This mechanism is frequently used to represent mechanically linked joints such as parallelogram linkages or differential couplings. For tendon-driven approximations, mimic joints can be used to enforce idealized coupling relationships that would otherwise arise from shared tendon routing. It is noted in the Omni Physics documentation that mimic joints can introduce numerical instability when enforced as stiff constraints and that compliance can be introduced to improve stability when competing constraints apply [64].

MJCF tendon elements. MuJoCo XML Format (MJCF) models (MuJoCo XML) include tendon elements to express fixed and spatial tendons explicitly. The Isaac Sim MJCF importer exposes a tendon element for creating cable-like constraints that can span multiple bodies and joints, including fixed and spatial tendon types [66]. This provides an additional modeling route for tendon-driven mechanisms when an MJCF description is available or when it is convenient to prototype tendon constraints in MuJoCo-style syntax before conversion to USD.

Limitations relevant to tendon-driven fidelity. Although tendon primitives and mimic couplings allow tendon-like behavior to be introduced into an articulation, several fidelity gaps remain for tendon-driven robots: (i) distributed friction along tendon routing and hysteresis are typically not captured by ideal constraints, (ii) tendon stretch and tension propagation require tuned compliance parameters to remain stable at simulation time steps, and (iii) multi-contact routing through pulleys and guides is usually simplified to a small set of attachment points. These limitations are significant because tendon friction and hysteresis are reported to dominate motion errors in tendon-driven systems in applications such as surgical robotics, and improved prediction accuracy is

obtained only when frictional effects are included in the mapping [61, 60]. As a consequence, tendon-driven robots are often simulated using a hybrid strategy: the *kinematic chain* is represented by standard joints, while tendon effects are represented by coupled joints (tendons or mimic constraints) and by compliant joint drives tuned to reproduce the apparent stiffness/damping observed in the physical system. Where higher fidelity is required, custom actuation models that map tendon commands to joint torques with friction/hysteresis compensation are required [60].

2.4.5 Precision, dynamics, and compliance: implications for cloth manipulation

Tendon-driven designs differ from direct-drive manipulators in both achievable dynamics and precision characteristics. Reduced distal inertia can enable rapid wrist motions and high dynamic bandwidth in configurations that would be limited by motor mass and gear-box inertia if actuators were placed distally [59]. However, because the actuation forces are transmitted through compliant tendons and routed interfaces, the closed-loop behavior is sensitive to friction, elastic stretch, and hysteresis effects. These phenomena can manifest as configuration-dependent tracking error and reduced repeatability compared to stiff, direct-drive solutions, and they motivate calibration and model-based compensation strategies when high positional accuracy is required [60, 61].

For cloth interaction tasks, the same compliance mechanisms that degrade open-loop positioning precision can be advantageous. Cloth manipulation is characterized by frequent contact, uncertain contact locations, and large modeling error due to deformable object dynamics. In this context, compliance can reduce peak contact forces, mitigate damage to the environment, and tolerate contact uncertainty by allowing small deviations from commanded trajectories without large impulsive forces [67, 58]. The safety-performance tradeoff in physical interaction is classically addressed by introducing compliance in the actuation/transmission, which reduces impact severity and can improve robustness in contact-rich tasks at the cost of reduced rigidity [67]. Therefore, tendon-driven designs can be regarded as mechanically aligned with manipulation scenarios in which force limiting and safe contact are prioritized over sub-millimeter absolute accuracy.

2.4.6 Workspace analysis and dexterity metrics

The workspace of a serial manipulator is the set of all end-effector poses that are reachable under joint limits and kinematic constraints. Workspace analysis is used to verify that a

robot can reach task-relevant regions and to identify configurations that should be avoided due to singularities or poor motion quality [68, 69].

Forward kinematics and sampling motivation. Forward kinematics (Forward Kinematics (FK)) is defined as the mapping from a joint configuration $\mathbf{q}$ to an end-effector pose, i.e., position and orientation of a designated tool frame. For a general serial chain, this mapping can be written as

$$\mathbf{T}_{EE} = f(\mathbf{q}), \quad (2.7)$$

where $\mathbf{T}_{EE} \in SE(3)$ denotes a homogeneous transformation of the end-effector frame. In many practical systems, analytical derivation of workspace boundary surfaces is intractable due to high DoF, joint coupling, or non-standard kinematics. In such cases, the workspace is frequently approximated numerically by sampling joint space and evaluating FK repeatedly. The Monte Carlo approach is widely used because it is simple to implement, directly incorporates joint limits, and does not require explicit boundary derivation [68].

Monte Carlo workspace sampling and voxel aggregation. In the Monte Carlo approach, configurations are sampled from a prescribed distribution over the admissible joint ranges. In the simplest implementation, each joint q_i is sampled independently and uniformly within its limit interval. For each sample $\mathbf{q}^{(k)}$, the corresponding end-effector pose is computed by FK. The resulting set of poses is then aggregated into a discrete spatial representation, e.g., Cartesian voxels, to estimate reachability statistics per region. This computation is conceptually aligned with established robotics software workflows that generate reachable workspace point sets paired with joint configurations [70].

When voxel aggregation is used, the workspace is partitioned into a regular grid of bins (voxels) in position space (and optionally orientation space). Each sample contributes to the occupancy count of the voxel containing the sampled pose. This discretization supports additional metrics beyond mere reachability, including density-based indicators of how many joint-space configurations map to a given region [69].

Placeholder for *workspace voxelization and density visualization* image

Figure 2.4: Example visualization of a sampled reachable workspace. End-effector positions obtained from Monte Carlo FK sampling are aggregated into Cartesian voxels. Voxel occupancy represents reachability density, and additional overlays (e.g., manipulability) can be displayed to indicate dexterity variation across task space.

Yoshikawa manipulability. Dexterity is commonly evaluated using Jacobian-based indices. Let $J(\mathbf{q})$ denote the geometric Jacobian that maps joint velocities to end-effector twist. The Yoshikawa manipulability measure is defined as [71]

$$w(\mathbf{q}) = \sqrt{\det(J(\mathbf{q}) J(\mathbf{q})^\top)}. \quad (2.8)$$

This scalar is proportional to the volume of the manipulability ellipsoid and becomes zero at kinematic singularities where the Jacobian loses rank. Higher values indicate that small joint motions can produce motion in a wider range of Cartesian directions, which is desirable for tasks requiring dexterous repositioning and local adjustments [71].

Inverse condition number as an isotropy index. A complementary measure is derived from the Jacobian condition number. Salisbury and Craig introduced the use of Jacobian conditioning to identify well-conditioned (isotropic) operating points and to quantify error amplification through the kinematic mapping [72]. Let $\sigma_{\min}$ and $\sigma_{\max}$ denote the minimum and maximum singular values of $J(\mathbf{q})$. The inverse condition number is defined as

$$\kappa_{\text{inv}}(\mathbf{q}) = \frac{\sigma_{\min}(J(\mathbf{q}))}{\sigma_{\max}(J(\mathbf{q}))}. \quad (2.9)$$

This index lies in $[0, 1]$, approaches zero near singular configurations, and equals one at perfectly isotropic configurations (equal singular values), where motion/force transmission properties are directionally uniform [72].

Reachability density. Reachability density is defined here as the number of sampled configurations whose end-effector poses fall within a given voxel. Under uniform joint sampling, this count is proportional to the joint-space volume that maps into the voxel region; thus, higher density indicates that more distinct configurations can realize poses in that region. This concept is closely related to workspace density formulations that assign each voxel the number of reachable points (or a normalized probability density) and interpret higher density as higher positional/orientational reach richness and, in discretely actuated systems, higher achievable pose resolution [69].

Implications for task-space design in learning-based manipulation. For manipulation tasks in cluttered scenes and for RL-based control, the workspace and dexterity metrics directly inform environment layout and action-space design. Task objects (e.g., pick targets, cloth placement zones) are preferably positioned in regions with high

reachability density and consistently high manipulability to reduce the probability that the robot is driven into near-singular configurations during exploration. Similarly, action bounds for end-effector motion primitives can be restricted to dexterous regions to improve learning stability and to reduce large corrective motions caused by kinematic ill-conditioning [71, 72, 69]. When tendon-driven compliance is present, these considerations become more important because poor kinematic conditioning can amplify transmission uncertainties into larger Cartesian errors, which in turn increases undesired contact forces during cloth interaction [60, 67].

2.5 Policy Learning Pipeline

Reinforcement learning-based control is typically realized as an iterative interaction loop between an agent (policy) and an environment (simulation), in which behavior is improved by maximizing an expected return defined over episodes [1, 73]. In IsaacLab, task construction is formalized through an explicit MDP specification and a software architecture that decomposes environment logic into reusable terms for observations, actions, rewards, terminations, events, and curriculum scheduling [74, 75, 76]. This section describes the standard RL pipeline under this workflow and derives an MDP formulation suitable for cloth grasping and stretching tasks in simulation.

2.5.1 Agent–Environment Cycle and Markov Decision Processes

An MDP is commonly defined as a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $\mathcal{P}(s' \mid s, a)$ denotes the transition kernel, $r(s, a, s')$ denotes the immediate reward, and $\gamma \in [0, 1)$ denotes the discount factor [73, 1]. The RL objective is formulated as maximizing the expected discounted return under a policy π :

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, a_t, s_{t+1}) \right], \quad a_t \sim \pi(\cdot \mid o_t), \quad s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t), \quad (2.10)$$

where T denotes the episode horizon and o_t denotes the agent’s observation at time step t [1, 73].

For robotic manipulation tasks, especially those involving deformable objects, the *true* state s_t (e.g., all cloth vertex/particle states and contact states) is typically not directly observable, and the policy is conditioned on a partial observation $o_t = g(s_t)$ constructed

from sensors or engineered features [77, 78]. This setting is naturally described as a partially observable MDP (Partially Observable Markov Decision Process (POMDP)), in which observations are related probabilistically to the hidden state and optimal decision making may require memory or state estimation [79]. In practice, many manipulation policies are implemented as *memoryless* mappings $\pi(a \mid o)$, such that the observation vector (or a short history of observations) is treated as the effective policy input [1, 77].

The standard on-policy learning pipeline alternates between (i) collecting rollouts by executing the current policy in the environment and (ii) updating the policy parameters by optimizing a surrogate objective using the collected trajectories [6]. For example, PPO explicitly follows this alternation and performs multiple epochs of minibatch optimization over a clipped surrogate objective to stabilize policy gradient updates [6]. Regardless of the specific algorithm, this structure implies that the environment must provide: batched observations o_t , the ability to apply actions a_t , a reward signal r_t , and episode termination flags to delineate trajectories for return/advantage estimation [1, 6].

2.5.2 IsaacLab MDP Workflow for Task Construction

IsaacLab provides two task design workflows: a *manager-based* workflow, in which environment functionality is decomposed into modular managers, and a *direct* workflow, in which environment logic is implemented in a single class in a more monolithic style [75, 74]. The manager-based workflow is explicitly organized around MDP components such as observations, actions, rewards, terminations, and curriculum, where each component is represented by configuration classes (implemented with a Python `@configclass` pattern) and is coordinated by the environment instance [74, 75, 76].

Placeholder for *diagram of IsaacLab manager-based MDP workflow:*
observation/action/reward/termination/curriculum managers
coordinated by env.step() image

Figure 2.5: Conceptual decomposition of an IsaacLab MDP into managers. In the manager-based workflow, action processing, observation computation, reward accumulation, and termination checks are implemented as modular terms and orchestrated by the environment step loop.

In the manager-based workflow, IsaacLab defines reusable *term functions* (e.g., joint-state observations, generic reward primitives, and standard termination checks) in a dedicated MDP module, which is designed to be consumed by corresponding managers [76, 80]. Concretely, observation terms are configured via an observation manager, action terms via an action manager, reward terms via a reward manager, and termination terms via

a termination manager [76, 80]. Additionally, an event manager is typically used for randomization and episodic events, and a curriculum manager is provided to schedule changes to environment parameters or term weights over training [76, 80].

The direct workflow is supported for cases where decomposition is inconvenient or where large blocks of logic are implemented in optimized kernels, but it retains the same high-level requirements imposed by RL: an environment class must provide observations, apply actions, compute rewards, and implement reset/termination logic [75, 81]. Both workflows are compatible with ecosystem interfaces influenced by the OpenAI Gym API, which standardizes methods such as `reset()` and `step()` for RL benchmarking and algorithm development [82, 83].

2.5.3 MDP Definition for Cloth Manipulation

The cloth manipulation task considered in this thesis is characterized by (i) high-dimensional deformable-object dynamics, (ii) contact-rich interactions during grasping, and (iii) task objectives that are naturally expressed through geometric measures such as cloth spread/coverage, corner-to-corner distances, and alignment with a target configuration [77, 78, 84]. While the *true* simulator state may include all internal cloth degrees of freedom, the MDP induced for learning is typically specified through an observation function, an action parameterization, a shaped reward, and explicit termination conditions.

Table 2.4: Recommended MDP specification elements for cloth grasp-and-stretch in simulation.

Component	Design choice	Rat.
Observations o_t	Proprioception + cloth features / vision	Partial observability and h
Actions a_t	Joint targets or end-effector deltas + gripper	Continuous control with c
Reward r_t	Coverage/stretching + grasp success + penalties	Dense shaping for spars
Termination	Timeout + success + safety/failure events	Stable episodes and sa

2.5.3.1 State and observations

Underlying state. In an MDP formulation, the state $s_t \in \mathcal{S}$ is assumed to satisfy the Markov property, i.e., s_{t+1} is conditionally independent of past states given (s_t, a_t) [73, 1]. For deformable-object manipulation, s_t may be conceptualized as a concatenation of robot states (joint positions/velocities, tendon states, actuator internal states) and cloth states (e.g., all cloth vertex/particle positions and velocities, plus contact and constraint states) [77, 78]. This state is generally too high-dimensional to use directly for learning on

real hardware, and it is often not directly measurable even in simulation when the goal is to preserve a realistic sensing setup [77, 78].

Observation design. The policy input is therefore defined as an observation vector $o_t \in \mathcal{O}$ constructed from simulated sensors and engineered features. In cloth-like deformable object tasks, a central difficulty is that sufficient low-dimensional state representations are hard to define and that observation is typically partial or noisy, motivating POMDP modelling assumptions [77, 79]. In simulation, observations may be selected along a spectrum from privileged (full cloth state) to realistic (vision/tactile-based). The following observation groups are commonly used for cloth manipulation and align with IsaacLab’s modular observation manager concept [77, 78, 76, 80]:

- **Robot proprioception:** Joint positions/velocities, end-effector pose, gripper opening, and optionally actuator-related signals (e.g., efforts) to support closed-loop manipulation [85, 76].
- **Cloth configuration features:** Reduced descriptors such as cloth corner positions, edge/contour points, keypoint sets, or a heightmap/coverage estimate, reflecting the observation biases reported in cloth-shaping systems that track corners and contours [78].
- **Visual observations:** RGB, depth, or RGB-D camera inputs. Image-based policies have been trained to infer grasp points and pull vectors for fabric smoothing, and depth has been shown to provide valuable information in such settings [84]. Benchmark studies also emphasize that learning purely from visual observations can be substantially more challenging than learning from ground-truth state in deformable tasks [77].
- **Contact and event signals:** Binary/continuous indicators derived from contact sensing (e.g., cloth grasp detected, contact forces above threshold) to disambiguate grasp success and failure events under partial observability [75].

In IsaacLab manager-based environments, these observation components are operationalized by defining observation terms and grouping them into observation groups, which are then evaluated each environment step to produce the policy input tensor [76, 80].

2.5.3.2 Action space

Low-level versus higher-level actions. For robotic manipulation, the action space $\mathcal{A}$ can be defined at different abstraction levels. At the lowest level, actions may be joint torques $\boldsymbol{\tau} \in \mathbb{R}^n$ applied to an n -DoF manipulator, corresponding to direct actuation in the

robot dynamics model [85]. Alternatively, actions may be expressed as joint velocity commands, joint position targets, or incremental joint displacements, which are then realized through an internal controller in the simulator or robot stack [85]. In deformable manipulation, action abstraction is frequently used to reduce exploration complexity and to stabilize contact interactions, since learning directly in torque space can induce highly non-smooth exploration when contact constraints and deformable dynamics are present [78].

Action spaces compatible with IsaacLab terms. IsaacLab explicitly supports action terms that map normalized policy outputs to robot commands via an action manager [76, 80]. This interface enables several action parameterizations relevant to cloth grasp-and-stretch:

- **Joint-level continuous control:** Actions are mapped to joint targets (e.g., position or velocity) for selected joint subsets through configured action terms [76]. The general batched action tensor is processed per environment instance and applied during the physics step loop [81].
- **Cartesian end-effector motion primitives:** Actions are defined as incremental end-effector translations/rotations (and gripper commands), which are converted to joint targets through inverse kinematics or task-space controllers available in IsaacLab/Isaac Sim toolchains [86, 75]. This abstraction aligns with common cloth manipulation formulations that predict pick points and pull vectors rather than raw joint torques [84].
- **Robust gripper open/close actions:** For grasping-heavy tasks, IsaacLab provides an action term that converts a continuous gripper input into binary open/close commands using a threshold mechanism, with the stated aim of producing more predictable grasping behavior [76]. This mapping is suitable when grasp success is more sensitive to discrete state (open vs. closed) than to continuous gripper width modulation.

For cloth manipulation with a parallel-jaw gripper, a practical action vector is often defined as

$$\mathbf{a}_t = [\Delta \mathbf{x}_t^\top \quad \Delta \boldsymbol{\phi}_t^\top \quad a_t^{\text{grip}}]^\top,$$

where $\Delta \mathbf{x}_t \in \mathbb{R}^3$ and $\Delta \boldsymbol{\phi}_t \in \mathbb{R}^3$ denote incremental Cartesian translation and rotation commands, and a_t^{grip} denotes a gripper command (continuous or binary) [84, 76]. In IsaacLab, such a design is realized by composing multiple action terms under the action manager, allowing joint-space actuation and gripper commands to be processed consistently in a single step [80, 75].

2.5.3.3 Reward function design

Success criteria for cloth grasping and stretching. Reward design for cloth manipulation is difficult because task success is geometric and often sparse (e.g., “the cloth is sufficiently spread”), while exploration under contact and occlusion is challenging [77, 78]. In deformable-object benchmarks, spreading or smoothing tasks have been operationalized using coverage/area metrics computed from top-down projections of the cloth configuration [77]. Similarly, sequential fabric smoothing work has used the objective of maximizing cloth coverage as a success measure when learning pick-and-pull actions from visual observations [84].

Dense reward decomposition. To support stable learning, a dense reward can be constructed as a weighted sum of interpretable terms:

$$r_t = w_{\text{grasp}} r_t^{\text{grasp}} + w_{\text{stretch}} r_t^{\text{stretch}} + w_{\text{align}} r_t^{\text{align}} - w_{\text{act}} r_t^{\text{act}} - w_{\text{viol}} r_t^{\text{viol}}, \quad (2.11)$$

where each term targets a specific behavioral component and weights are tuned to balance shaping strength [1, 75].

A concrete instantiation suitable for grasp-and-stretch includes:

- **Grasp acquisition** r_t^{grasp} : A sparse positive reward is assigned when the cloth is successfully grasped (e.g., when the gripper is closed and cloth contact/grasp conditions are satisfied), since grasp failure prevents downstream stretching behavior. This mirrors the emphasis on successful pick selection in cloth shaping pipelines [78].
- **Spreading/stretching** r_t^{stretch} : A natural choice is a coverage-based metric, such as the top-down area covered by the cloth, which is explicitly used as the reward in the SoftGym `SpreadCloth` task [77]. When the task is formulated as stretching between grasp points (e.g., two corners), r_t^{stretch} may alternatively be defined as the increase in distance between target cloth keypoints, consistent with pick-and-pull vector formulations [84].
- **Alignment/placement** r_t^{align} : If the cloth is required to be stretched into a target pose (e.g., aligned on a conveyor or within a region of interest), an auxiliary term can penalize deviations between current and desired keypoint positions or between the cloth centroid and a target centroid. Distance-based terms are widely used in deformable manipulation benchmarks that compare current particle positions to a target configuration [77].
- **Action regularization** r_t^{act} : Smoothness penalties (e.g., $\|a_t\|^2$) or effort penalties discourage high-frequency control signals and can improve numerical stability and

hardware transfer compatibility [85].

- **Constraint/safety violations** r_t^{viol} : Penalties can be applied when joint limits are approached, self-collisions occur, or the cloth leaves the workspace, complementing termination logic and discouraging destructive exploration [76].

In IsaacLab manager-based tasks, each component in Equation 2.11 is implemented as a reward term with an associated weight and optional parameters, enabling systematic ablations and incremental shaping [75, 76]. This design is particularly aligned with deformable tasks where a single sparse success reward is often insufficient to drive exploration [77, 78].

2.5.3.4 Episode termination conditions

Timeout and truncation. Finite-horizon episodes are typically enforced through a maximum episode length T to bound return estimation and stabilize training statistics [1]. IsaacLab provides a standard termination term that triggers episode termination when the episode length exceeds the configured limit (`time_out`) [76]. In vectorized execution, environment stepping is designed such that terminated sub-environments are reset internally by the `step()` implementation, and it is therefore recommended that `reset()` is called only once prior to the first `step()` call [81].

Success termination. A success condition for cloth grasp-and-stretch is most naturally defined as achieving a threshold on a geometric metric, such as: (i) area coverage exceeding a fraction of the maximum possible area (coverage-based completion) [77], or (ii) keypoint alignment error falling below a tolerance (target-configuration completion) [77]. In a manager-based implementation, such conditions are introduced as termination terms (custom or built-in) evaluated each step by the termination manager [76, 80].

Failure termination. To prevent unproductive exploration and to approximate physical safety constraints, failure termination conditions are typically included, such as:

- **Robot constraint violations:** Episode termination when joint positions exceed soft limits is supported by built-in termination primitives (e.g., `joint_pos_out_of_limit`) [76].
- **Task failure events:** Termination when the cloth leaves the workspace, when grasp is lost irrecoverably (e.g., cloth is dropped and not re-grasped within a horizon), or when excessive collisions occur. These conditions are task-specific and are typically implemented as custom termination terms [75].

In addition to providing episode boundaries, termination events can be incorporated into the reward via explicit termination penalties, which are supported as reusable reward terms that penalize non-timeout terminations [76].

2.5.4 Reward Shaping and Curriculum Strategies

Reward shaping and policy invariance. Reward shaping refers to modifying or augmenting the environment reward signal to accelerate learning by providing denser feedback for intermediate behaviors [1]. In RL theory, a central concern is whether reward transformations preserve the optimal policy. A sufficient condition is potential-based shaping, in which an additional shaping reward is constructed from a potential function $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ such that

$$F(s_t, s_{t+1}) = \gamma \Phi(s_{t+1}) - \Phi(s_t), \quad (2.12)$$

and the shaped reward is defined as $r'_t = r_t + F(s_t, s_{t+1})$ [87, 88]. It has been shown that, under this construction, the optimal policy of the shaped MDP is preserved while learning dynamics and exploration behavior can be improved [88]. This principle is particularly relevant for cloth tasks, where sparse terminal rewards (e.g., “cloth fully stretched”) provide limited guidance during exploration in high-dimensional state spaces [77, 78].

Curriculum learning. Curriculum learning formalizes the idea that training can be accelerated by presenting tasks in an order of increasing difficulty, rather than sampling uniformly from the full difficulty distribution from the outset [89]. For deformable-object manipulation, curriculum strategies are frequently motivated by the large configuration space and the dependence of success on first achieving prerequisite behaviors (e.g., reliable grasp before stretching) [78]. In a cloth grasp-and-stretch setting, a curriculum can be instantiated by gradually increasing: (i) initial cloth disorder (wrinkles, random folds), (ii) friction/contact difficulty, (iii) domain randomization ranges (material parameters, observation noise), or (iv) task precision requirements (tighter alignment tolerances).

Curriculum support in IsaacLab. IsaacLab natively supports curriculum specification through a curriculum manager, and includes built-in curriculum terms that modify reward weights or environment parameters according to step-wise schedules (e.g., `modify_reward_weight`, `modify_env_param`) [76]. This enables curriculum learning to be operationalized without restructuring the core environment logic: shaped reward weights can be increased as competence improves, and environment parameters controlling task

difficulty can be updated online during training [76, 80]. In the manager-based workflow, such curricula are configured alongside observation, action, reward, and termination specifications, supporting controlled experimentation with staged training protocols [75, 74].

2.5.5 GPU-Vectorized Training and Environment Wrappers

Vectorized environments on the GPU. High sample throughput is essential for deep RL in robotics, particularly for contact-rich tasks with deformable dynamics. GPU-accelerated simulation frameworks have demonstrated substantial speedups by executing many environment instances in parallel and by avoiding CPU-GPU transfer bottlenecks between simulation and learning [25, 86]. Isaac Gym established an end-to-end GPU pipeline in which physics simulation and neural network training reside on the GPU and communicate through tensor interfaces, enabling large-scale batched rollouts [25]. IsaacLab is positioned as the successor to this paradigm, extending GPU-native simulation with additional capabilities (e.g., richer sensors, actuator models, and support for deformable objects) while remaining oriented toward large-scale parallel learning [86].

In IsaacLab, RL environments are designed to run as vectorized simulations with a configurable number of sub-environments (`num_envs`), where observations and actions are batched per step [81]. The environment implements a Gym-like interface, but it is not itself a `gym.vector.VectorEnv`; instead, wrappers are used to adapt to the varying vector-environment conventions of different learning libraries [81, 90].

Environment wrappers and library integration. The OpenAI Gym interface popularized the concept of wrapping environments to modify observations, rewards, and logging behavior without changing the underlying task implementation [82]. IsaacLab follows this design pattern and states that RL environments inheriting from its base classes implement the `gymnasium.Env` interface and are compatible with `gymnasium.Wrapper` [83]. Wrappers are used both for generic purposes (e.g., enforcing call order, recording videos) and for adapting to RL library-specific APIs [83, 90].

IsaacLab explicitly provides wrappers that convert the environment I/O to match the expectations of multiple RL libraries (e.g., vectorized environment APIs that require particular tensor/array formats) and recommends applying learning-framework wrappers last, since such wrappers may modify interpretation of environment calls [90, 91]. This design preserves a single environment implementation while enabling interchangeability of training backends, which is consistent with the manager-based objective of modularity and reuse [75, 91].

Scene replication for parallelism. Vectorization at the simulation level is enabled not only by batched stepping but also by scene replication. Isaac Sim provides a “Cloner” interface designed to simplify constructing scenes with many replicated environment instances and includes utilities for generating clone paths, computing transforms, and filtering collisions between clones [92]. The same documentation emphasizes that such vectorized copies are beneficial for collecting trajectories in parallel during RL training [92]. Together with IsaacLab’s batched environment APIs, this supports scalable data collection for policy learning in simulation [81, 86].

2.6 Prior Work in Robotic Cloth Manipulation

Robotic manipulation of cloth and garments is studied under the broader umbrella of deformable object manipulation, where the object configuration evolves in a high-dimensional space and is only partially observable through sensors. Compared to rigid-body manipulation, task execution is strongly affected by self-occlusion, frictional contacts, and uncertainty of material parameters such as bending stiffness, stretching stiffness, and surface friction, which collectively induce large variability across rollouts even when the same action primitives are executed [93, 94, 95]. In prior work, a wide range of strategies is reported, spanning analytical pipelines based on geometric reasoning and perception-driven state machines, as well as learning-based methods (including deep Imitation Learning (IL) and deep RL) trained in simulation or with limited real-world interactions [95].

2.6.1 Task taxonomy and recurring technical challenges

Cloth manipulation tasks are commonly grouped into (i) *state acquisition* tasks that expose informative configurations (e.g., lifting, unfolding, and spreading), (ii) *surface conditioning* tasks (e.g., smoothing and flattening to reduce wrinkles), and (iii) *goal-shape* tasks such as folding or dressing, which require steering the cloth toward a specified topological and geometric configuration [96, 95]. In benchmark suites, these tasks are frequently decomposed into multi-step sequences (e.g., unfold $\rightarrow$ smooth $\rightarrow$ fold), because reliable folding performance is typically conditioned on having reached a canonical, low-occlusion intermediate state [95, 97, 98].

Three technical difficulties recur across the literature. First, state estimation is challenged by self-occlusion and the lack of a compact, task-invariant representation; as a result, many approaches either rely on low-dimensional proxies (e.g., corner locations,

wrinkle features, or coverage metrics) or learn representations that are designed to transfer across cloth instances [93, 94]. Second, long-horizon credit assignment is required for folding and for multi-phase pipelines, since success or failure may only be determined after several intermediate actions have been executed [95, 99]. Third, generalization is impeded by variability in garment categories, material properties, textures, and initial configurations; this issue is explicitly highlighted in modern benchmark analyses, where algorithm performance is shown to be sensitive to the initial garment state and to the task horizon [99, 94].

2.6.2 Model-based and analytical approaches

Early systems for cloth manipulation were predominantly designed as perception-driven state machines whose transitions were defined by geometric cues and task-specific heuristics. A representative example is the towel-folding pipeline of Maitin-Shepard *et al.* [100], where grasp points were inferred from multiple-view geometric cues, and a sequence of re-grasps and table-assisted manipulations was executed to fold and stack towels; quantitative success was reported on previously unseen towels, including trials with a single towel and with a pile of towels [100]. Such pipelines demonstrate that reliable cloth handling can be achieved when perception primitives (e.g., corner detection and cloth boundary inference) are sufficiently robust and when the task can be decomposed into controllable sub-skills under partial observability [100, 95].

A second line of analytical work models cloth folding as a geometric or physics-informed planning problem. The folding planner of Miller *et al.* [101] formalized laundry folding by reasoning about fold sequences and cloth geometry, thereby enabling folding plans to be generated for polygonal cloth shapes and then executed by a robot. Complementary to purely geometric planners, predictive simulation has been integrated into trajectory optimization: Li *et al.* [102] optimized end-effector trajectories in an off-line simulator by minimizing the discrepancy between a simulated folded shape and a target, and then transferred the resulting trajectories to a physical dual-arm system. These approaches illustrate the central trade-off in model-based methods: explicit models and optimization reduce trial-and-error interaction, but require simulation fidelity and parameterization accuracy that are difficult to guarantee for cloth contacts and material behavior [102, 95].

A third family of methods employs closed-loop vision feedback with hand-crafted state features, thereby bridging analytical design and learning. Petrík and Kyrki [103] proposed feedback-based folding of fabric strips in which a low-dimensional state was extracted from camera images and controller parameters were trained in simulation after calibrating the simulation to better match real fabric behavior; performance was then compared exper-

imentally to alternative folding strategies on real hardware [103]. This design pattern is recurring: model structure and feedback variables are specified analytically, while learning is used to tune parameters or to account for dynamics that are difficult to model precisely [95, 103].

2.6.3 Learning-based approaches

Learning-based cloth manipulation is often categorized by whether an explicit dynamics model is used during decision making. In *model-free* methods, a policy (or value function) is optimized directly from interaction data without explicitly learning a forward transition model for planning; in *model-based* methods, a predictive model is learned and then used by a planner (e.g., trajectory optimization or Model Predictive Control (MPC)) [94, 95].

2.6.3.1 Learning policies for smoothing and flattening

Fabric smoothing (also described as flattening) is frequently used as an intermediate objective because it reduces self-occlusion and yields configurations that are easier to perceive and fold [95]. Seita *et al.* [84] learned sequential pick-and-pull smoothing policies from image observations by using an algorithmic supervisor in simulation that had access to full state information, while Domain Randomization (DR) was applied on image appearance and sensing modalities to improve transfer. Policies were evaluated both in simulation and in physical experiments on the da Vinci Research Kit (dVRK), and improvements in coverage were demonstrated for multi-action smoothing episodes [84]. The reported design illustrates a frequent motif in cloth learning systems: privileged state information is used during data generation or supervision in simulation, while the deployed policy is conditioned on realistic observations such as Red-Green-Blue plus Depth (RGB-D) images [84, 94].

Complementary to imitation-based smoothing, model-based visual dynamics has been used to generalize across tasks by replanning toward goal images. Hoque *et al.* [104] introduced VisuoSpatial Foresight, where a visual dynamics model was learned from domain-randomized simulated RGB-D data and then used for planning to accomplish multi-step fabric smoothing and folding. The approach was evaluated in simulation and on the dVRK without requiring task demonstrations at training or test time, and the contribution of depth sensing to performance was empirically analyzed [104]. Such results motivate the use of model-based policies when multi-step reasoning is required, but they also underline the computational cost and sensitivity associated with learning predictive models of deformable dynamics from images [104, 94].

2.6.3.2 Unfolding and spreading via quasi-static and dynamic primitives

Unfolding and spreading tasks are often evaluated via coverage-based metrics, because exposing the garment surface is viewed as a prerequisite for subsequent perception and folding modules [96, 95]. In contrast to purely quasi-static pick-and-place strategies, dynamic manipulation has been shown to improve efficiency and effective workspace. Ha and Song [97] proposed FlingBot, a self-supervised learning framework for cloth unfolding using a dual-arm pick–stretch–fling primitive, and high coverage was reported within a small number of actions on novel cloths; the approach was also fine-tuned on a real dual-arm system and compared against quasi-static baselines [97]. A related perspective is provided by Cloth Funnels, where canonicalized-alignment was formulated as a funneling task that maps diverse crumpled configurations into a compact set of structured, visible states using a mixture of dynamic flings and quasi-static pick-and-place actions, and downstream ironing and folding were demonstrated using this standardized intermediate representation [98].

Model-free RL has also been investigated in purely simulated benchmarks, often achieving strong in-simulator performance but requiring extensive interaction data. SoftGym was introduced as a suite of simulated deformable object manipulation benchmarks (including cloth), providing standardized environments and an `OpenAI Gym`-style interface to compare learning algorithms under different observability settings [77]. In such benchmarks, policies can be trained with large-scale sampling in simulation; however, it is repeatedly observed that learned policies can be brittle under distribution shifts in initial states, materials, and observation noise, thus motivating research into generalizable representations and robust transfer pipelines [77, 94, 99].

2.6.3.3 Real-world learning with limited interaction

While simulation is often used to address data requirements, learning directly in the real world has also been explored when action spaces are discretized and when self-supervised data collection is feasible. Lee *et al.* [105] demonstrated goal-conditioned fabric folding with one hour of autonomous real-robot experience, without requiring simulation or manually engineered rewards beyond sparse success signals. The approach highlights that, for certain task formulations (e.g., discrete pick-and-place folding actions with image goals), sample-efficient real-world learning can be achieved, albeit typically at the cost of restricting action parameterizations and relying on extensive on-robot data collection [105, 95].

2.6.4 Benchmarks, datasets, and evaluation protocols

The need for reproducible evaluation is a persistent theme in cloth manipulation, because systems are often validated on custom garments, private benchmarks, or task-specific metrics. Garcia-Camacho *et al.* [96] addressed this issue by proposing benchmarks for three core tasks (spreading a tablecloth, folding a towel, and dressing), along with standardized objects, multiple difficulty levels, and quantitative quality measures, enabling comparison across bimanual platforms. In simulated learning, SoftGym provides a set of open benchmarks for deformable objects (including cloth) and is often used to quantify algorithmic progress under controlled simulator settings [77].

Recent work has shifted toward garment-centric benchmarks that incorporate multiple garment categories and that explicitly address sim-to-real evaluation. GarmentLab was proposed as a unified simulation and benchmark for garment manipulation, featuring diverse garments, tasks, and robotic systems, and it incorporates multiple simulation methods (including FEM and PBD) as well as a reproducible real-world benchmark; empirical evaluation in GarmentLab indicates that vision-based methods can be sensitive to initial garment states and that RL-based algorithms can struggle with long-horizon tasks requiring multi-step reasoning [99]. In parallel, Real Garment Benchmark (RGBench) was presented to benchmark garments and simulators with a large asset collection and a protocol intended to quantify simulator accuracy against measured real garment dynamics, thereby framing the sim-to-real gap itself as an evaluation target [106]. Competition-style evaluation has also emerged: De Gusseme *et al.* [107] reported a dataset and benchmark for grasp pose selection in in-air cloth unfolding, derived from the ICRA 2024 Cloth Competition and designed to support head-to-head comparison across perception strategies under a standardized setup.

Benchmarks referred to as “GarmentBench” have been mentioned in informal discussions of garment manipulation; however, an unambiguous peer-reviewed robotics benchmark under this exact name could not be identified from authoritative sources at the time of writing.

Placeholder for *taxonomy of cloth tasks and benchmark mapping* image

Figure 2.6: Illustration of common cloth manipulation tasks (unfolding, smoothing, folding, dressing) and representative benchmarks/datasets used to evaluate them. The figure is intended to visualize where simulation-only benchmarks, sim-to-real benchmarks, and real-only protocols are typically positioned.

2.6.5 Sim-to-real transfer strategies for deformable objects

Robust sim-to-real transfer is widely recognized as more difficult for deformable objects than for rigid objects, because small modeling errors in contacts, friction, or material parameters can accumulate into large state deviations over multi-step horizons [94, 95]. This issue is exacerbated for dynamic cloth actions, where inertial effects and high-acceleration motions amplify simulator inaccuracies. Blanco-Mulero *et al.* [108] quantified this challenge by benchmarking the sim-to-real gap for both quasi-static and dynamic cloth manipulation using a real-world dataset and multiple simulation engines, thereby demonstrating that simulator choice, stability, and observation alignment can materially affect transfer conclusions [108].

Several complementary strategies for sim-to-real are reported, and they are commonly combined in successful systems. First, *parameter randomization* is frequently applied to cloth properties (e.g., stiffness and friction) and to observation generation (e.g., textures and lighting) so that the real world is covered as a plausible variation of the simulated training distribution. This principle is commonly referred to as DR and has been established as a practical transfer mechanism in robotics [109]. In cloth-specific settings, DR has been applied to train image-conditioned policies in simulation that are subsequently executed on physical cloth, including sequential fabric smoothing with a surgical robot [84] and visuomotor fabric manipulation with a learned dynamics model [104].

Second, *visual domain adaptation* has been used to reduce the appearance gap between simulated and real observations when policies are conditioned on images. While prominent demonstrations are reported in rigid grasping (where synthetic imagery is adapted to real images without requiring labeled real data) [110], closely related ideas in cloth manipulation frequently take the form of learned visual descriptors or correspondences trained in simulation and then deployed on real cloth. Ganapathi *et al.* [111] learned dense visual correspondences in simulation and used them to support smoothing and folding behaviors on real fabrics, effectively transferring perception primitives learned with privileged simulation signals to real sensory inputs.

Third, *teacher-student distillation* has been employed to combine policies, compress models, or transfer from privileged training signals to deployable observation spaces. Policy distillation, in which a student policy is trained to imitate one or multiple teacher policies, provides a mechanism to consolidate specialized behaviors into a single policy [112]. In deformable object manipulation, this mechanism has been used explicitly to increase generalization across diverse configurations: Wang *et al.* [113] learned robot-assisted dressing policies in simulation from partial point clouds and used policy distillation to merge policies trained on different pose ranges into a single policy. In the same system, guided

domain randomization was introduced by training a domain-randomized policy to imitate policies trained without randomization, and extensive real-world evaluations on human participants were reported [113]. Although dressing differs from tabletop folding and smoothing, the underlying transfer challenge (complex cloth–environment contacts with partial observability) is shared, and the reported methodology provides an existence proof that distillation-based transfer can be viable in deformable settings [113].

2.6.6 Open problems and positioning of the thesis

Survey and review literature consistently identifies three open problems that remain central despite progress in individual tasks. *Generalization across fabrics and garments* is limited, because policies trained on a narrow distribution of cloth properties, sizes, and topologies can fail when material stiffness ratios or garment categories change substantially [93, 94, 99]. *Robustness to perception noise and partial observability* remains a bottleneck, especially when cloth state is inferred from depth or point cloud observations under self-occlusion; this affects both analytical pipelines (which rely on corner or wrinkle detection) and learned policies (which may overfit to simulator rendering artifacts) [94, 108]. *Long-horizon, multi-step reasoning* is still challenging, as benchmark analyses report performance degradation as task horizons increase and as manipulation requires sequential subgoals with intermediate regrasping and state acquisition [95, 99].

In the present thesis, these observations motivate an approach in which cloth manipulation skills are learned in simulation with a focus on (i) policy representations that remain effective under partial observability, (ii) explicit treatment of sim-to-real gaps through systematic randomization and calibration of cloth interaction parameters, and (iii) task decomposition and curriculum design to make long-horizon cloth objectives learnable. A further distinguishing aspect is the targeted manipulation platform: whereas much prior work is demonstrated on conventional rigid-link manipulators and dual-arm systems, the investigated tendon-driven arm introduces additional actuation dynamics and compliance properties that are directly relevant to safe textile handling but are underrepresented in cloth manipulation benchmarks and learning studies.

2.7 Robot and Scene Description Formats

Robotic learning workflows in simulation typically require at least two layers of model description: (i) a *robot model* capturing kinematic structure and physical properties, and (ii) a *scene/world model* capturing the placement of robots and objects, lighting and sensors,

and simulator-specific physics configuration. In the USD-based *Omniverse* ecosystem, the scene is represented as a composed USD stage, while robots may be authored in robotics-oriented formats such as URDF or MJCF and subsequently imported into USD for simulation. This section explains the role of USD, URDF, and MJCF in this separation of concerns and clarifies how these formats are used in this thesis.

Placeholder for *format conversion and integration pipeline* image

Figure 2.7: Schematic pipeline illustrating how robot descriptions (e.g., URDF or MJCF) are converted into USD assets and integrated with environment elements (rigid objects, cloth, lights, sensors) on a composed USD stage.

2.7.1 USD as a scene graph for simulation worlds

Core scene description concept. USD is a system for encoding scalable, hierarchically organized data (static as well as time-sampled) with the primary purpose of interchange and augmentation across digital content creation applications [115]. Beyond being a file encoding, USD is designed to *compose* a single stage from scene description distributed across many layers and assets, supporting sparse overrides and asset aggregation via composition operators such as references and variants [115]. This composition-centric design is the main reason USD is used as the foundation of *NVIDIA Omniverse*-based applications, including robotics simulation: a simulation world can be assembled from reusable assets (robots, fixtures, conveyors, cloth objects), while experiment-specific changes (randomization parameters, object placements, material settings) can be authored as non-destructive overrides.

Extensibility via schemas. A central mechanism in USD is *schema-based extensibility*: standardized or custom schemas define what properties are available on a prim and how data are interpreted by downstream tools [115, 8]. For robotics simulation, this is crucial because a scene must not only store geometry and transforms, but also physics and sensor semantics. In practice, simulation-ready USD stages rely on multiple cooperating schema families: (i) rendering-related schemas (e.g., light prims), and (ii) physics-related schemas that define rigid bodies, collisions, articulation joints, and constraints, enabling a simulator backend to interpret the stage as a dynamical system.

Lighting and sensor-relevant elements. Lighting in USD is represented by the USD lighting schema *UsdLux*, which provides a portable representation for common light types

and attributes (e.g., intensity/exposure and color) intended for interchange across rendering environments [116]. This is directly relevant for robotics simulation, because perception pipelines and synthetic data generation depend on reproducible lighting setups. The use of common light schemas also reduces simulator lock-in at the scene-description level, since the same lighting structure can be understood by multiple Omniverse tools and render backends [116].

Physics properties encoded in USD. Physics semantics in USD are provided by *UsdPhysics*, which defines prim and property schemas for representing physics simulation state and configuration on a stage [8]. Within this schema family, rigid bodies, joints, and constraints are described as USD prims and applied Application Programming Interface (API) schemas, allowing physics engines to identify which parts of the stage are simulated and how they are constrained [8]. Joint actuation is commonly represented via drive attributes that behave as implicit damped springs, i.e., forces/accelerations are defined via stiffness and damping terms toward a target position/velocity [8]. In Omniverse, additional simulator-specific tuning parameters are exposed through *PhysX schema additions*; for example, PhysX joint schemas provide per-axis parameters and actuator performance characteristics extending the generic *UsdPhysics* joint representation [117].

2.7.2 URDF as a robotics-oriented robot model format

Scope and typical usage. URDF is an Extensible Markup Language (XML)-based format used in the *Robot Operating System* ecosystem for specifying robot geometry and organization [118]. A URDF model typically focuses on the robot as a kinematic structure, described by rigid links connected through joints, plus the geometric information used for visualization and collision checking, and the physical properties required for rigid-body simulation (e.g., link inertias) [119]. This focus makes URDF a common interchange format for robotic manipulators and mobile robots, especially when robots are also integrated into *ROS* pipelines for control, state publishing, and visualization [118].

Kinematic structure, joint limits, and inertial properties. Within the URDF workflow, links are augmented with collision geometry and inertial parameters, and joints are augmented with joint dynamics and limits. The ROS 2 URDF tutorials explicitly emphasize that collision and inertial properties are needed for simulation, and that joint dynamics parameters (e.g., friction) can be specified as part of the joint description [119]. Joint limits (position bounds, effort/velocity bounds) are also represented at the joint

level in URDF-based tooling, and are typically used both for controller safety and for ensuring physically plausible simulation states [119].

Structural limitations relevant to tendon mechanisms. A key limitation of URDF is that, in standard robotics tooling, only serial kinematic chains are supported in a direct and portable way; more complex kinematic dependencies are generally handled through additional simulator- or controller-specific mechanisms [120]. A commonly supported exception is the *mimic joint* concept, which allows joint coordinates to be coupled through a linear dependency. Mimic joints are explicitly described as a pragmatic abstraction used to represent simplified closed-loop dependencies or mechanically coupled degrees of freedom, such as belt-and-pulley systems [120]. This capability is relevant for tendon-driven or cable-driven mechanisms when coupling can be approximated as a simple linear relationship. However, tendon routing with wrapping, varying moment arms, elastic cable stretch, frictional loss, and multi-joint coupling is generally not fully representable with the standard URDF kinematic abstraction alone [120]. In such cases, either simulator-specific extensions or higher-level actuation models must be introduced.

2.7.3 MJCF as MuJoCo’s native modeling language

Positioning and expressiveness. MJCF is the native XML modeling language of *MuJoCo* and is designed to expose a rich set of model elements while remaining scalable through default mechanisms and compile-time processing [121]. In the original MuJoCo description, model construction is performed either through a high-level XML model file (MJCF) or programmatic API calls, with the MJCF file intended as an intuitive text description of the model to be compiled into the simulator’s internal representation [54]. Importantly, MuJoCo documentation explicitly describes URDF as “popular but more limited”, and positions MJCF as the modeling format used to access MuJoCo’s full expressiveness [121].

Tendons as first-class elements. For tendon-driven robots, a central feature of MuJoCo’s modeling language is that tendons are directly supported as model elements. MuJoCo documentation describes tendon geometry as minimum-path-length strings with wrapping and via-point constraints, and notes that robotics-specific structures such as pulleys and coupled degrees of freedom are provided as part of the tendon mechanism [122]. Tendons can be used for actuation as well as to impose inequality/equality constraints on tendon length [122]. This contrasts with standard URDF usage, where cable routing and tendon geometry are not part of the core, portable robot description abstraction [120, 121].

Relationship to URDF-based models. MuJoCo tooling supports loading both MJCF and URDF descriptions, while explicitly stating that the URDF representation covers only a subset of the available model elements and features [121]. Consequently, MJCF is often selected when the simulation must include advanced actuation and constraint structures (including tendons) that are difficult to express in URDF without simulator-specific extensions [121, 122].

2.7.4 Format use in this thesis and tendon-driven modeling workarounds

Responsibilities of each format in the pipeline. In the context of Isaac Sim and Isaac Lab, USD is treated as the *runtime scene format*: robots and environment assets are ultimately represented as USD so that they can be loaded, referenced, and simulated on a USD stage [123]. Robot-centric description formats such as URDF or MJCF are primarily used as *authoring or interchange formats* for the robot model itself. Their primary responsibility is to capture: (i) robot topology (links and joints), (ii) joint types and joint limits, and (iii) rigid-body physical properties (e.g., inertial parameters and collision geometry) required for simulation [119, 121]. By contrast, USD is used to integrate the robot with the rest of the simulated world and to attach simulation-specific semantics (physics schema applications, light prims, and renderer-facing scene elements) [115, 8, 116].

Which format is used for the tendon-driven robot and why. In this thesis, the custom tendon-driven arm component is provided as a *modified* URDF description and is imported into Isaac Sim, where it becomes a USD robot asset for simulation. This choice is motivated by the practical availability of URDF as a widely used robotics interchange format and by the fact that Isaac Sim provides an explicit import workflow that converts URDF robot descriptions into USD assets for use on a USD stage [124]. In addition, Isaac Lab recommends converting external robot model formats into USD using the provided importers (including URDF and MJCF importers) and then performing subsequent editing, tuning, and large-scale simulation using the USD representation [123]. The final integrated robot is therefore simulated as USD within Isaac Sim, consistent with Omniverse’s USD-native scene representation [123].

Integration of robot and scene as USD assets. Once the robot components exist as USD assets, integration into a complete simulation scene is performed by placing them on the USD stage together with the environment geometry and task objects (including cloth and supporting structures) and then attaching physics and rendering semantics through

the relevant schemas [8, 116]. In Omniverse-based robotics workflows, modular robot assemblies are commonly created by combining separate USD robot assets (e.g., a base, an arm, and a gripper) via fixed joints. Isaac Sim’s *Robot Assembler* tool is explicitly designed for this use case: it combines two USD assets through a physically simulated fixed joint and produces a resulting USD asset that can be saved and loaded as a single composite robot [125].

URDF modifications and tendon modeling workarounds. Standard URDF does not provide a portable, first-class representation of tendon routing with wrapping, elastic elements, and multi-joint cable coupling comparable to MJCF’s tendon mechanism [122, 121]. Therefore, tendon-driven mechanisms are often approximated in URDF-to-simulator pipelines by mapping tendon actuation to joint-level drive targets (position/velocity/effort) and by encoding only those couplings that can be captured with simple kinematic relationships (e.g., mimic joints) [120]. In Omniverse workflows, practical URDF-to-USD conversion constraints further motivate simplification: Isaac Lab explicitly documents that some URDF tags used in other simulators (e.g., `<transmission>` and `<gazebo>` tags) are not supported by the URDF importer, and that pre-processing may therefore remove such tags prior to conversion [123]. This limitation is particularly relevant for tendon-driven robots if tendon routing or actuation coupling has been encoded via transmission-like blocks: such elements may need to be removed or replaced by simulator-native constructs after import.

In the present thesis draft, the tendon routing/transmission constructs in the imported arm description are simplified such that each tendon-driven joint is represented as an actuated revolute joint driven directly by joint targets rather than an explicit cable model. Moreover, joint limits are enforced in the description to improve stability, while complex tendon dynamics (stretch, multi-joint coupling, backlash) are not explicitly simulated at this stage.

Why PhysX tendon features are not used in the current implementation. Although PhysX articulations support tendon constraints at the articulation level, including fixed tendons (constraints on joint positions) and spatial tendons (distance constraints along attachment points) [126], limitations in simulation fidelity and behavior have been documented for articulation tendons in Omniverse Physics. Specifically, articulation tendons are listed among known limitations, and workarounds are recommended: mimic-joint style coupling is suggested for fixed tendon behavior, and external forces are suggested to mimic spatial tendon effects [127]. In this thesis, these limitations motivate a simplified approach in which tendon effects are approximated via direct joint drives (spring-damper-

like actuation toward joint targets) rather than via explicit PhysX tendon constraints, thereby prioritizing stability and controllability of the learning environment over detailed tendon mechanics.

- **URDF role:** The tendon-driven arm geometry, kinematic chain, joint limits, collision shapes, and rigid-body inertial properties are authored in URDF and then converted for Omniverse simulation [119, 124].
- **USD role:** The robot (after conversion) and the environment assets are integrated on a composed USD stage; physics and lighting semantics are expressed via *UsdPhysics* and *UsdLux*, and PhysX-specific parameters are expressed via PhysX schema additions [8, 116, 117].
- **MJCF role:** MJCF is treated as a common alternative robot description format in the broader robotics and learning literature, with particularly strong support for tendon mechanisms in MuJoCo; it is not used as the primary authoring format in this thesis, but conversion into USD is supported in Isaac Lab when MJCF assets exist [54, 122, 123].
- **Tendon workarounds:** Due to both representational limitations of URDF and documented limitations of articulation tendon fidelity in Omniverse Physics, tendon actuation is approximated through joint drives and coupling simplifications rather than explicit tendon constraints [120, 127].

Table 2.5: Structured comparison of representative prior work in robotic cloth and garment manipulation, covering analytical/model-based pipelines, learning-based approaches, and benchmarks.

Work	Method class	Robot type	Task focus	Simulation and sim-to-real
Maitin-Shepard <i>et al.</i> [100]	Vision-based analytical pipeline (multi-view grasp inference, state machine)	Dual-arm mobile manipulator	Towel unfolding and folding	Real-only evaluation
Miller <i>et al.</i> [101]	Geometric planning (fold sequence reasoning)	Dual-arm manipulator	Laundry folding plans for cloth shapes	Planning model; real execution reported
Li <i>et al.</i> [102]	Model-based (predictive simulation + trajectory optimization)	Dual-arm manipulator	Garment folding trajectories	Off-line simulation; sim-to-real execution
Doumanoglou <i>et al.</i> [114]	Perception + planning pipeline (multi-stage)	Dual-arm manipulator	Folding clothes from cluttered configurations	Real-world pipeline; limited reliance on simulation
Petrík and Kyrki [103]	Feedback controller with learned parameters	Single-arm manipulator	Fabric strip folding	Calibrated simulation for training; real evaluation
Seita <i>et al.</i> [84]	Learning-based (IL with algorithmic supervision)	dVRK surgical robot	Sequential fabric smoothing/flattening	Simulation + DR; sim-to-real execution
Hoque <i>et al.</i> [104]	Model-based (learned visual dynamics + MPC)	dVRK surgical robot	Multi-step smoothing and folding	Simulation training with DR; real execution
Ha and Song [97]	Learning-based (self-supervised policy for dynamic primitive)	Dual-arm manipulator	Cloth unfolding via flinging	Learned strategy; real fine-tuning reported
Canberk <i>et al.</i> [98]	Learning-based (multi-primitive canonicalization)	Multi-arm system	Canonicalized-alignment for downstream tasks	Simulation (PyFlex-family) + real demonstrations
Lin <i>et al.</i> [77]	Benchmark	N/A (simu-	Cloth/rope/fluid	Simulation-

Table 2.6: Roles of USD, URDF, and MJCF in robotics simulation pipelines.

Format	Primary scope	Type of data
USD	Scene graph with composition	World assembly, physics, rendering
URDF	Robot kinematics + basic rigid-body data	Robot model interchange
MJCF	Full MuJoCo model description	Expressive robot/environment models in simulation

3 Methodology

3.1 Overview of Approach

3.2 Simulation Environment Setup

3.3 Robot Model Integration

In this section, the simulation setup for the custom tendon-driven robotic manipulator is presented. The complete robot model (Figure 3.1) consists of three modular components: a 2-DoF articulated base providing linear motion, a 3-DoF tendon-driven arm mounted atop the base, and a Robotiq 2F-140 two-finger gripper as the end-effector. All components are integrated in NVIDIA Isaac Sim 5.1.0, under the project’s Methodology and Implementation framework, to emulate the physical robot’s behavior in a virtual environment. The following subsections detail each component’s design and integration, the assembly of the full manipulator using Isaac Sim’s tools, and the validation of its control setup.

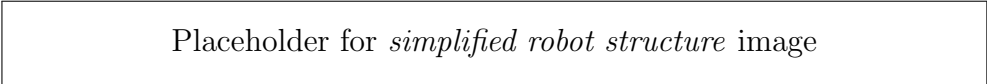
A rectangular box with a thin black border, containing the text "Placeholder for *simplified robot structure* image".

Figure 3.1: Simplified structure of the integrated robot model in Isaac Sim, showing the 2-DoF linear base, the 3-DoF tendon-driven arm, and the Robotiq 2F-140 gripper attached as end-effector.

3.3.1 2-DoF Articulated Base (Linear Y–Z Stage)

The articulated base is a two-degree-of-freedom linear stage that provides planar positioning for the arm along the horizontal Y -axis and vertical Z -axis. It is designed exclusively in simulation, with no direct real-world counterpart, serving as a versatile foundation to adjust the arm’s position within the workspace. The base’s prismatic joints allow translation of the arm $\pm 0.4\text{m}$ laterally (Y direction) and -0.5m downward in height (Z direction) from a defined origin (with $Z = 0$ at the upper limit) - see Table 3.1 for a summary of joint travel ranges. These limits yield a horizontal span of 0.8m and a vertical adjustment of

0.5 m, chosen to cover the required operating area of the project’s frame while avoiding singular configurations or collisions at extremes.

In implementation, the base was constructed from scratch within Isaac Sim using the native USD robot schema definitions (rather than importing from URDF). It consists of rigid link geometry (modeled as simple shapes) and two linear actuators defined as prismatic joints along the global Y and Z axes. Each prismatic joint is configured with a physics drive (stiffness and damping) for position control, and appropriate limit stops at the specified travel range. The entire base assembly was built to fit the physical dimensions of the project’s frame and mounting interface, ensuring that when the arm is attached, its workspace aligns with the intended real-world environment. Because the base is purely virtual, it enabled rapid prototyping of different mounting heights and lateral positions for the arm without hardware modifications. All structural components of the base were kept simple and lightweight in simulation to maintain real-time performance.

3.3.2 3-DoF Tendon-Driven Arm

Mounted on the base’s vertical slider is the tendon-driven manipulator arm with three actuated degrees of freedom. This arm’s design and geometry are derived from a prior Computer-Aided Design (CAD) model developed in a master’s thesis by Klein (2023)[63]. The original arm is a cable-driven (tendon-driven) mechanism inspired by a human arm’s musculo-tendon structure, featuring an upper arm, forearm, and wrist. In our simulation, the arm was imported via a modified URDF file based on Klein’s CAD model. The URDF was adjusted for compatibility with IsaacSim - for example, custom transmission elements representing the tendon routing were simplified or removed, and each tendon-driven joint was defined as a standard revolute joint with direct actuation. The arm is rigidly attached to the top of the base (on the moving Z -axis platform) so that its shoulder reference frame coincides with the base’s moving frame.

Each of the arm’s three joints is modeled in IsaacSim as a revolute articulation with a position/force drive to emulate the tendon actuation. In the physical design, these joints would be driven by tendons via remote actuators, but in simulation they are controlled directly by setting joint angle targets and applying spring-damper forces. The first joint (shoulder/elbow pitch) allows approximately 172° of rotation ($\pm 86^\circ$, i.e. ± 1.5 rad, from neutral), corresponding to the primary bending of the “elbow” in the vertical plane as described by Klein [63]. The wrist mechanism of the arm is implemented as two consecutive revolute joints (perpendicular axes) representing the wrist’s flexion/extension and abduction/adduction; each of these has a range of $\pm 46^\circ$ (± 0.8 rad). These ranges are in line with the mechanical constraints reported in the original design [63]. For simulation

stability, hard joint limits are imposed in the URDF/USD definition to prevent the solver from pushing the joints beyond their intended travel.

Tendon Actuation Modeling: Instead of using PhysX’s built-in tendon simulation feature for articulations, the tendon-driven joints are simulated with direct joint drives. NVIDIA’s documentation notes that the PhysX articulation tendon feature can exhibit inadequate fidelity and unstable behavior under certain conditions [128]. In particular, spatial tendon constraints may lead to solver instabilities or unrealistic force feedback. To avoid these issues, we opted not to use PhysX “physxTendon” elements. Instead, each joint is given a Proportional-Derivative (PD) control (a stiff spring with damping) that attempts to maintain the target angle, thereby mimicking the effect of a pretensioned tendon pulling the joint into position. The drive stiffness and damping values were initially chosen based on the estimated elasticity and friction in the tendons, and later fine-tuned using the Gain Tuning tool (see Section 3.3.5). This approach provides a stable yet simplified representation of tendon actuation: the joints behave as if driven by motors with compliance, without modeling cable stretch or complex tendon dynamics. We acknowledge that the tendon dynamics (e.g., stretch, backlash, and multi-joint coupling) are not explicitly simulated at this stage - incorporating a more realistic tendon model or a dedicated cable simulation is left for future work.

Collision Geometry: The arm’s collision shapes were generated using convex hull approximations of the CAD model geometry. Each link (upper arm, forearm, wrist links) is assigned a convex collision mesh computed from its visual mesh. Using convex hulls simplifies the collision geometry and is recommended for better physics performance in Isaac Sim. This ensures robust contact simulation while avoiding the computational cost and potential instability of complex mesh collisions. The convex colliders reasonably approximate the arm’s shape (covering joint protrusions, etc.) and were verified to avoid interpenetration in all valid joint configurations. With these collision models in place, the arm can interact with the environment (e.g., grasped objects or potential obstacles) in simulation without excessive physics solver load.

3.3.3 Robotiq 2F-140 Gripper Integration

For the end-effector, the Robotiq 2F-140 adaptive gripper was selected from IsaacSim’s built-in asset library. This is a two-finger parallel gripper well-suited to the project’s manipulation tasks. The 2F-140 model was chosen for its wide stroke and proven compatibility: it can grip a range of object sizes with an adjustable finger span up to 140mm, and it provides a significant grip force for secure holding. According to the manufacturer’s specifications, the 2F-140 offers a maximum stroke of 140mm and an adjustable grip force

from about 10N up to 125N[129]. It is rated for handling payloads up to roughly 2.5kg in a form-fit grasp, while the gripper’s own weight is about 1kg. These characteristics make it capable of picking and holding typical objects in our application (such as mixed waste items) without exceeding the arm’s load capacity.

In simulation, the Robotiq gripper is loaded as a self-contained articulated asset (with its finger linkage and drive mechanism pre-defined by NVIDIA’s asset). This greatly simplified integration: rather than modeling a custom gripper, the ready-made 2F-140 module was attached to the robot arm and controlled through its single actuated degree of freedom (the synchronized motion of the two fingers). The gripper’s coupling interface follows the standard ISO-50 flange, and it was directly compatible with the mounting point on the wrist of the tendon-driven arm. Notably, this gripper has been used in other Isaac Sim and Isaac Lab setups (e.g., with Universal Robots UR10 arms), indicating robust support and reliability in the simulation environment. Its inclusion here not only meets the functional requirements (adaptive gripping of various objects) but also leverages existing calibration and tuning knowledge from prior implementations. The official product sheet and instruction manual provided further technical details and usage guidelines, which were consulted to ensure the simulated gripper’s behavior (speed, force, fingertip friction, etc.) aligns with the real device [129].

3.3.4 Modular Assembly via Isaac Sim’s Robot Assembler

With the base, arm, and gripper defined as separate components, the complete robot was assembled within IsaacSim using the Robot Assembler tool. The Robot Assembler is an interactive utility provided by IsaacSim that allows users to combine multiple robot sub-assemblies into a single coherent model by creating fixed joints between them. In our case, we performed a two-step assembly: (1) attaching the 3-DoF tendon-driven arm onto the linear base, and (2) attaching the 2F-140 gripper onto the arm’s wrist flange. IsaacSim’s documentation explicitly supports these use cases - for example, assembling a robot arm to a moving base, or attaching a gripper to a robot arm, are typical workflows[128]. Using the Robot Assembler UI (accessible via Tools → Robotics → Asset Editor → Robot Assembler in Isaac Sim), the base was designated as the “Base Robot,” and the arm was added as the “Attach Robot” at the appropriate attach point on the base (the top plate of the Z-axis slider). The attach point corresponded to a pre-defined locator frame on the base where the arm’s bottom joint should coincide. Once the arm was assembled to the base, the combined base+arm was treated as a new single robot. Then, similarly, the gripper asset was attached to the arm: the arm’s URDF included a tool flange link at the wrist, which we used as the attach point for the gripper (with the gripper as “Attach Robot”

in the tool). The assembler automatically created a fixed joint linking the gripper’s base to the arm’s flange, aligning them based on their coordinate frames. After each assembly operation, the resulting composite robot (base+arm+gripper) was saved to a new USD file so that it can be loaded in the future as one complete manipulator model without needing to repeat the manual assembly steps.

Because the Robot Assembler uses a physically simulated fixed joint to bond the parts, the attached components behave as a single articulated body during simulation. This means the base, arm, and gripper move together without drift when physics is enabled, but they also remain modular in the project structure. One major advantage of this modular assembly approach is the flexibility for future hardware replacement or reconfiguration. Each subsystem (base, arm, gripper) can be swapped out or modified independently. For instance, if a different gripper model is to be tested in the future, it can replace the 2F-140 in the assembly with minimal effort; or if the project evolves to use a physical linear axis or a different arm, those could be integrated by the same method. The assembled manipulator’s design is therefore not monolithic - it is reconfigurable by design. Figure 3.2 illustrates an exploded view of the robot, showing the three modules (linear base, tendon arm, and gripper) and their attachment relationships. Such a view highlights the modular interfaces: the mounting plate between base and arm, and the wrist flange between arm and gripper.

Placeholder for *exploded view of assembly image*

Figure 3.2: Exploded view of the robot assembly, showing (from bottom) the 2-DoF linear base, the 3-DoF tendon-driven arm, and the Robotiq 2F-140 gripper. The arrows indicate the attachment points: the arm mounts onto the base’s top slide, and the gripper attaches to the arm’s wrist flange. This modular design allows each component to be independently developed or replaced.

After assembly, the final integrated robot has a total of five controlled joints (two prismatic joints in the base and three revolute joints in the arm, not counting the underactuated motion of the gripper’s fingers which are driven by one actuator). Table 3.1 summarizes the joint types and their ranges of motion in the integrated model. These ranges reflect both the mechanical design limits and the safety constraints imposed in simulation. By verifying the combined kinematic chain, we ensured that the robot can reach the required workspace for the intended tasks (e.g., covering the sorting area and adjusting to various bin heights) while respecting joint limits. All joints are configured with appropriate drive parameters and limit enforcement as discussed previously. In the next subsection, we address how the control gains for these joints were tuned and validated.

Table 3.1: Actuated joints of the integrated robot and their motion ranges.

Joint	Type & Axis	Range of Motion
Base Linear Y	Prismatic (horizontal slide)	−0.4 m to +0.4 m
Base Linear Z	Prismatic (vertical lift)	−0.5 m to 0.0 m (downwards)
Arm Joint 1	Revolute (shoulder/elbow pitch)	−86° to +86° (± 1.5 rad)
Arm Joint 2	Revolute (wrist tilt X-axis)	−46° to +46° (± 0.8 rad)
Arm Joint 3	Revolute (wrist tilt Y-axis)	−46° to +46° (± 0.8 rad)

3.3.5 Control Gain Tuning and Validation

Once the robot model was assembled, a critical step was to validate and tune the joint control gains to achieve stable and accurate motion. We utilized IsaacSim’s Gain Tuner extension for this purpose. The Gain Tuner provides tools to adjust the proportional position stiffness and damping parameters of each joint’s drive and to test the robot’s response to control inputs in simulation. According to NVIDIA’s documentation, the Gain Tuner is intended to help find a pair of stiffness and damping gains for each joint such that the robot can follow commanded trajectories with the expected behavior[128]. In practice, we performed automated gain testing on all five joints: the tool applied small-amplitude sinusoidal position commands and step changes within each joint’s allowable range, while we observed the joint’s motion response. For each joint, the stiffness (analogous to a P-gain) was increased until the joint responded quickly to commands without significant steady-state error, and the damping (D-gain) was adjusted to critically damp the response (minimizing overshoot and oscillation). Isaac Sim’s Gain Tuner provided visual feedback in the form of plots comparing commanded vs. actual joint trajectories, which was invaluable for iterative tuning and verifying stability.

Through this gain tuning process, we confirmed that the base’s linear axes and the arm’s revolute joints could be moved under position control with smooth and stable behavior. In particular, the heavy vertical axis (base Z) required a higher damping ratio to counteract gravity-induced oscillations, and the relatively lightweight wrist joints could use lower stiffness to reflect their tendon-driven compliance. After tuning, the robot’s joints exhibit near-critical damping: when given a step command, each joint reaches the target quickly (within 0.1-0.3 s settling time) with minimal overshoot (typically < 5% of the motion range) and no sustained oscillation. These tuned gains were then fixed in the simulation configuration for all subsequent experiments. This control validation step was essential to ensure that high-level control or planning algorithms (addressed in later chapters) would interact with a robot model that behaves in a realistic and predictable manner. By using

the Gain Tuner, we effectively calibrated the simulated actuators to mimic a well-tuned physical controller, thereby laying a sound foundation for the robot’s performance in simulation.

3.4 Reinforcement Learning Setup

3.4.1 State Space

3.4.2 Action Space

3.4.3 Reward Space

3.4.4 RL Algorithm and Training Parameters

3.4.5 MDP Workflow in IsaacLab

3.5 Initial Experiments

To work towards the final implementation, initial experiments were conducted focusing on basic tasks such as reaching a specified pose and performing simple pick-and-place operations. These experiments served to validate the robot model and the reinforcement learning setup within the simulation environment. As comparison points, baseline results example implementations from IsaacLab with the UR10 robot arm were utilized.

3.5.1 Reach Pose Task

The reach pose task involved training the tendon-driven robot to move its end-effector to a designated position in 3D space. The performance was evaluated based on the accuracy of reaching the target position within a defined tolerance.

3.5.2 Simple Pick and Place Task

In the simple pick-and-place task, the robot was trained to grasp an object from one location and place it at another specified location. The success of this task was measured by the robot’s ability to securely grasp the object and accurately place it at the target location.

3.5.3

3.6 Advanced Task Implementation

3.7 Software and Hardware Integration

4 Results

4.1 Preliminary Experiments

4.2 Performance of Tendon-Driven Robot on Simple Tasks

4.3 Cloth Manipulation Task Results

4.4 Summary of Key Results

5 Discussion

5.1 Evaluation of Outcomes

5.2 Comparison with Related Work

5.3 Limitations of the Study

5.4 Practical Implications

5.5 Recommendations and Lessons Learned

6 Conclusion

6.1 Conclusion

6.2 Outlook

Bibliography

- [1] SUTTON, Richard S.; BARTO, Andrew G.: *Reinforcement Learning: An Introduction*, second edition, The MIT Press, 2018, ISBN 9780262039246.
- [2] KOBER, Jens; BAGNELL, J. Andrew; PETERS, Jan: *Reinforcement Learning in Robotics: A Survey*, in The International Journal of Robotics Research, 2013, Vol. 32 (11), pp. 1238–1274.
- [3] SCHULMAN, John; MORITZ, Philipp; LEVINE, Sergey; JORDAN, Michael I.; ABBEEL, Pieter: *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, in arXiv preprint arXiv:1506.02438, 2015.
- [4] TOBIN, Josh; FONG, Rachel; RAY, Alex; SCHNEIDER, Jonas; ZAREMBA, Wojciech; ABBEEL, Pieter: *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*, in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [5] PENG, Xue Bin; ANDRYCHOWICZ, Marcin; ZAREMBA, Wojciech; ABBEEL, Pieter: *Sim-to-Real Transfer of Robotic Control with Dynamics Randomization*, in arXiv preprint arXiv:1710.06537, 2018.
- [6] SCHULMAN, John; WOLSKI, Filip; DHARIWAL, Prafulla; RADFORD, Alec; KLIMOV, Oleg: *Proximal Policy Optimization Algorithms*, in arXiv preprint arXiv:1707.06347, 2017.
- [7] SERRANO-MUÑOZ, Antonio; CHRYSOSTOMOU, Dimitrios; BØGH, Simon; ARANA-AREXOLALEIBA, Nestor: *skrl: Modular and Configurable Library for Reinforcement Learning*, in Journal of Machine Learning Research, 2023, Vol. 24 (254), pp. 1–7.
- [8] PIXAR ANIMATION STUDIOS: *UsdPhysics: USD Physics Schema*, 2021, openUSD API documentation; generated on 2021-07-15.,
URL https://openusd.org/docs/api/usd_physics_page_front.html (Accessed on 2026-03-03).
- [9] NVIDIA: *Omni PhysX — Overview*, 2026,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/

-
- `extensions/runtime/source/omni.physx/docs/index.html` (Accessed on 2026-03-03).
- [10] NVIDIA: *Physics Simulation Fundamentals — Isaac Sim 5.1.0 Documentation*, 2026,
URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/physics/simulation_fundamentals.html (Accessed on 2026-03-03).
- [11] NVIDIA: *Welcome to PhysX — PhysX 5.1.3 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/index.html> (Accessed on 2026-03-03).
- [12] NVIDIA: *GPU Rigid Bodies*, 2022, physX 5.1.0 documentation, last updated Nov 08, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/GPURigidBody.html> (Accessed on 2026-03-03).
- [13] JAKOBI, Nick; HUSBANDS, Phil; HARVEY, Inman: *Noise and the Reality Gap: The Use of Simulation in Evolutionary Robotics*, in *Advances in Artificial Life (ECAL 1995)*, *Lecture Notes in Computer Science*, Vol. 929, Vol. 929, pp. 704–720, Springer.
- [14] SALVATO, Erica; FENU, Gianfranco; MEDVET, Eric; PELLEGRINO, Felice Andrea: *Crossing the Reality Gap: A Survey on Sim-to-Real Transferability of Robot Controllers in Reinforcement Learning*, in *IEEE Access*, 2021, Vol. 9, pp. 153171–153187.
- [15] PENG, Xue Bin; ANDRYCHOWICZ, Marcin; ZAREMBA, Wojciech; ABBEEL, Pieter: *Sim-to-Real Transfer of Robotic Control with Dynamics Randomization*, in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3803–3810.
- [16] NVIDIA: *Best Practices Guide — PhysX 5.4.1 Documentation*, 2024,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/BestPractices.html> (Accessed on 2026-03-03).
- [17] NVIDIA: *Rigid Body Dynamics — PhysX 5.1.1 Documentation*, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.1/docs/RigidBodyDynamics.html> (Accessed on 2026-03-03).
- [18] NVIDIA: *Articulations — PhysX 5.1.3 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/docs/Articulations.html> (Accessed on 2026-03-03).

-
- [19] NVIDIA: *Tendon Visual Authoring — Omni Physics Documentation*, 2025,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/extensions/ux/source/omni.usdphysics.ui/docs/dev_guide/tendon_authoring.html (Accessed on 2026-03-03).
- [20] NVIDIA: *Rigid Body Collision — PhysX 5.1.0 Documentation*, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/RigidBodyCollision.html> (Accessed on 2026-03-03).
- [21] NVIDIA: *PxMaterial — PhysX 5.1.0 API Reference*, 2022,
URL https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/_build/physx/latest/class_px_material.html (Accessed on 2026-03-03).
- [22] MACKLIN, Miles; STOREY, Kier; LU, Michelle; TERDIMAN, Pierre; CHENTANEZ, Nuttapong; JESCHKE, Stefan; MÜLLER, Matthias: *Small Steps in Physics Simulation*, 2019,
URL <https://mmacklin.com/smallsteps.pdf> (Accessed on 2026-03-03).
- [23] NVIDIA: *Simulation — PhysX 5.4.1 Documentation*, 2024,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Simulation.html> (Accessed on 2026-03-03).
- [24] NVIDIA: *Advanced Collision Detection — PhysX 5.1.2 Documentation*, 2022,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.2/docs/AdvancedCollisionDetection.html> (Accessed on 2026-03-03).
- [25] MAKОВИYCHUK, Viktor; WAWRZYNIAK, Lukasz; GUO, Yunrong; LU, Michelle; STOREY, Kyle; MACKLIN, Miles; HOELLER, David; RUDIN, Nikita; ALLSHIRE, Arthur; HANDA, Ankur; STATE, Gavriel; GIRDHAR, Ankur; KUMAR, Tushar; FINK, Jonathan; MEREL, Josh; TASSA, Yuval: *Isaac Gym: High Performance GPU Based Physics Simulation for Robot Learning*, in *NeurIPS 2021 Datasets and Benchmarks Track (Round 2)*.
- [26] NVIDIA: *isaacsim — IsaacLab Documentation (Simulation Configuration and GPU Physics)*, 2025,
URL <https://isaac-sim.github.io/IsaacLab/main/source/api/lab/isaacsim.html> (Accessed on 2026-03-03).
- [27] NVIDIA: *Geometry — PhysX 5.4.1 Documentation*, 2024,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Geometry.html> (Accessed on 2026-03-03).

-
- [28] NVIDIA: *Geometry — PhysX 5.3.0 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.3.0/docs/Geometry.html> (Accessed on 2026-03-03).
- [29] NEALEN, Andrew; MÜLLER, Matthias; KEISER, Richard; BOXERMAN, Eddy; CARLSON, Mark: *Physically Based Deformable Models in Computer Graphics*, in *Computer Graphics Forum*, 2006, Vol. 25 (4), pp. 809–836.
- [30] LONGHINI, Alberta; WANG, Yufei; GARCIA-CAMACHO, Irene; BLANCO-MULERO, David; MOLETTA, Marco; WELLE, Michael; ALENYÀ, Guillem; YIN, Hang; ERICKSON, Zackory; HELD, David; BORRÀS, Júlia; KRAGIC, Danica: *A Review of Robotic Cloth Manipulation*, in *Annual Review of Control, Robotics, and Autonomous Systems*, 2025.
- [31] BARAFF, David; WITKIN, Andrew: *Large Steps in Cloth Simulation*, in *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*, pp. 43–54.
- [32] MÜLLER, Matthias; HEIDELBERGER, Bruno; HENNIX, Marcus; RATCLIFF, John: *Position Based Dynamics*, in *Journal of Visual Communication and Image Representation*, Vol. 18, Vol. 18 (2), pp. 109–118.
- [33] MACKLIN, Miles; MÜLLER, Matthias; CHENTANEZ, Nuttapong: *XPBD: Position-Based Simulation of Compliant Constrained Dynamics*, in *Proceedings of the 9th International Conference on Motion in Games (MIG)*, 2016.
- [34] SIFAKIS, Eftychios; BARBIČ, Jernej: *FEM Simulation of 3D Deformable Solids: A Practitioner’s Guide to Theory, Discretization and Model Reduction*, in *ACM SIGGRAPH 2012 Courses*.
- [35] JIANG, Chenfanfu; SCHROEDER, Craig; TERAN, Joseph; STOMAKHIN, Alexey; SELLE, Andrew: *The Material Point Method for Simulating Continuum Materials*, in *ACM Transactions on Graphics*, 2016, Vol. 35 (4).
- [36] BRIDSON, Robert; FEDKIW, Ronald; ANDERSON, John: *Robust Treatment of Collisions, Contact and Friction for Cloth Animation*, in *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '02)*, pp. 594–603.
- [37] NVIDIA: *Particle System — PhysX 5.1.3 Documentation*, 2023,
URL <https://nvidia-omniverse.github.io/PhysX/physx/5.1.3/docs/ParticleSystem.html> (Accessed on 2026-03-03).

-
- [38] NVIDIA: *PxPBDParticleSystem — PhysX API Reference*, 2023,
URL https://nvidia-omniverse.github.io/PhysX/physx/5.3.1/_api_build/class_px_p_b_d_particle_system.html (Accessed on 2026-03-03).
- [39] NVIDIA: *Particles*, 2026, omni Physics documentation, last updated Jan 20, 2026,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/particles/particles.html (Accessed on 2026-03-03).
- [40] NVIDIA: *Omniverse Physics and PhysX SDK Limitations — Isaac Sim 5.1.0 Documentation*, 2026,
URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/physics/physics_resources.html (Accessed on 2026-03-03).
- [41] FRATARCANGELI, Marco; PELLACINI, Fabio: *A GPU-Based Implementation of Position Based Dynamics for Interactive Deformable Bodies*, in *Journal of Graphics Tools*, 2013, Vol. 17 (3), pp. 59–66.
- [42] CHEN, Anka He; LIU, Ziheng; YANG, Yin; YUKSEL, Cem: *Vertex Block Descent*, in *ACM Transactions on Graphics (Proceedings of SIGGRAPH)*, 2024, Vol. 43 (4).
- [43] NEWTON PHYSICS CONTRIBUTORS: *Newton Physics Documentation*, 2025,
URL <https://newton-physics.github.io/newton/> (Accessed on 2026-03-03).
- [44] NVIDIA: *Announcing Newton, an Open-Source Physics Engine for Robotics Simulation*, 2025,
URL <https://developer.nvidia.com/blog/announcing-newton-an-open-source-physics-engine> (Accessed on 2026-03-03).
- [45] NVIDIA: *Train a Quadruped Locomotion Policy and Simulate Cloth Manipulation with NVIDIA Isaac Lab and Newton*, 2025,
URL <https://developer.nvidia.com/blog/train-a-quadruped-locomotion-policy-and-simulate-cloth-manipulation> (Accessed on 2026-03-03).
- [46] NVIDIA: *What Is Isaac Sim?*, 2026, isaac Sim Documentation (v5.1.0), last updated Jan 28, 2026,
URL <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html> (Accessed on 2026-03-03).
- [47] NVIDIA; MITTAL, Mayank; ROTH, Pascal; TIGUE, James; RICHARD, Antoine; ZHANG, Octi; DU, Peter; SERRANO-MUÑOZ, Antonio; YAO, Xinjie; ZURBRÜGG, René; RUDIN, Nikita; WAWRZYNIAK, Lukasz; ET AL.: *Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning*, arXiv, 2025, doi:10.48550/

-
- arXiv:2511.04831, arXiv:2511.04831,
URL <https://arxiv.org/abs/2511.04831> .
- [48] NVIDIA: *Isaac Sim: Extensions API*, 2025, isaac Sim Documentation (v5.1.0), last updated Oct 17, 2025,
URL <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/py/index.html> (Accessed on 2026-03-03).
- [49] MITTAL, Mayank; YU, Felix; ZHAO, Jitendra; CHENG, Yuanhao; DIMITROV, Dimitar; QIU, Louis; VELONAKI, Mari; ZHU, Yuke: *Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments*, in IEEE Robotics and Automation Letters, 2023, Vol. 8 (6), pp. 3740–3747.
- [50] THE ISAAC LAB PROJECT DEVELOPERS: *Creating a Manager-Based RL Environment*, 2026, isaac Lab Documentation, last updated Mar 03, 2026,
URL https://isaac-sim.github.io/IsaacLab/v1.0.0/source/tutorials/03_envs/create_manager_rl_env.html (Accessed on 2026-03-03).
- [51] THE ISAAC LAB PROJECT DEVELOPERS: *Multi-GPU and Multi-Node Training*, 2026, isaac Lab Documentation, last updated Mar 03, 2026,
URL https://isaac-sim.github.io/IsaacLab/main/source/features/multi_gpu.html (Accessed on 2026-03-03).
- [52] THE ISAAC LAB PROJECT DEVELOPERS: *SkillGen for Automated Demonstration Generation*, 2026, isaac Lab Documentation, last updated Mar 03, 2026,
URL <https://isaac-sim.github.io/IsaacLab/main/source/overview/imitation-learning/skillgen.html> (Accessed on 2026-03-03).
- [53] SUNDARALINGAM, Balakumar; HARI, Siva Kumar Sastry; FISHMAN, Adam; GARRETT, Caelan; VAN WYK, Karl; BLUKIS, Valts; MILLANE, Alexander; OLEYNIKOVA, Helen; HANDA, Ankur; RAMOS, Fabio; RATLIFF, Nathan; FOX, Dieter: *cuRobo: Parallelized Collision-Free Minimum-Jerk Robot Motion Generation*, arXiv, 2023, doi:10.48550/arXiv.2310.17274, arXiv:2310.17274,
URL <https://arxiv.org/abs/2310.17274> .
- [54] TODOROV, Emanuel; EREZ, Tom; TASSA, Yuval: *MuJoCo: A Physics Engine for Model-Based Control*, in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5026–5033.
- [55] PYBULLET CONTRIBUTORS: *Bullet Real-Time Physics Simulation: PyBullet*, 2026, project website,
URL <https://pybullet.org/> (Accessed on 2026-03-03).

-
- [56] COUMANS, Erwin: *Bullet Physics Simulation*, in *ACM SIGGRAPH 2015 Courses*.
- [57] ALLARD, J.; COTIN, S.; FAURE, F.; BENSOUSSAN, P.-J.; POYER, F.; DURIEZ, C.; DELINGETTE, H.; GRISONI, L.: *SOFA – an open source framework for medical simulation*, in *Studies in Health Technology and Informatics*, 2007, Vol. 125, pp. 13–18.
- [58] RAO, Priyanka; PEYRON, Quentin; LILGE, Sven; BURGNER-KAHRS, Jessica: *How to Model Tendon-Driven Continuum Robots and Benchmark Modelling Performance*, in *Frontiers in Robotics and AI*, 2021, Vol. 7, pp. 630245.
- [59] NEMOTO, Takeru; WALTER, Jonas; BACHMANN, Christian; GERLICH, Matthias; REITELSHÖFER, Sebastian; FRANKE, Jörg: *Highly Dynamic 2-DOF Cable-Driven Robotic Wrist Based on a Novel Topology*, in *IEEE Robotics and Automation Letters*, 2022, Vol. 7 (2), pp. 5727–5734.
- [60] MIYASAKA, Muneaki; HAGHIGHIPANAH, Mohammad; LI, Yangming; MATHESON, Joseph; LEWIS, Andrew; HANNAFORD, Blake: *Modeling Cable-Driven Robot With Hysteresis and Cable-Pulley Network Friction*, in *IEEE/ASME Transactions on Mechatronics*, 2020, Vol. 25 (2), pp. 1095–1104.
- [61] KATO, Takahisa; OKUMURA, Ichiro; KOSE, Hidekazu; TAKAGI, Kiyoshi; HATA, Nobuhiko: *Tendon-driven continuum robot for neuroendoscopy: validation of extended kinematic mapping for hysteresis operation*, in *International Journal of Computer Assisted Radiology and Surgery*, 2016, Vol. 11 (4), pp. 589–602.
- [62] III, Robert J. Webster; JONES, Bryan A.: *Design and Kinematic Modeling of Constant Curvature Continuum Robots: A Review*, in *The International Journal of Robotics Research*, 2010, Vol. 29 (13), pp. 1661–1683.
- [63] KLEIN, Max: *Arbeitsraumanalyse, Simulation und Bewegungsplanung eines seilgetriebenen robotischen Manipulators*, Masterarbeit, Friedrich-Alexander-Universität Erlangen-Nürnberg, 2023.
- [64] NVIDIA: *Articulations – Omni Physics (Articulation Tendons and Mimic Joint Compliance)*, 2026,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/107.3/dev_guide/rigid_bodies_articulations/articulations.html (Accessed on 2026-03-03).
- [65] NVIDIA: *URDF Importer Extension – Isaac Sim Documentation*, 2026,
URL https://docs.isaacsim.omniverse.nvidia.com/6.0.0/importer_

-
- exporter/ext_isaacsim_asset_importer_urdf.html (Accessed on 2026-03-03).
- [66] NVIDIA: *MJCFTendon – Isaac Sim Python API Documentation*, 2025,
URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/py/api/classisaacsim_1_1asset_1_1importer_1_1mjcf_1_1_m_j_c_f_tendon.html
(Accessed on 2026-03-03).
- [67] BICCHI, Antonio; TONIETTI, Giovanni: *Fast and “Soft-Arm” Tactics: Dealing with the Safety-Performance Tradeoff in Robot Arms Design and Control*, in IEEE Robotics & Automation Magazine, 2004, Vol. 11 (2), pp. 22–33.
- [68] RASTEGAR, J.; FARDANESH, B.: *Manipulation workspace analysis using the Monte Carlo Method*, in Mechanism and Machine Theory, 1990, Vol. 25 (2), pp. 233–239.
- [69] DONG, Hui; DU, Zhijiang; CHIRIKJIAN, Gregory S.: *Workspace density and inverse kinematics for planar serial revolute manipulators*, in Mechanism and Machine Theory, 2013, Vol. 70, pp. 508–522.
- [70] THE MATHWORKS, INC.: *Generate reachable workspace of robot in environment (generateRobotWorkspace)*, 2026,
URL <https://www.mathworks.com/help/robotics/ref/generaterobotworkspace.html> (Accessed on 2026-03-03).
- [71] YOSHIKAWA, Tsuneo: *Manipulability of Robotic Mechanisms*, in The International Journal of Robotics Research, 1985, Vol. 4 (2), pp. 3–9.
- [72] SALISBURY, J. Kenneth; CRAIG, John J.: *Articulated Hands: Force Control and Kinematic Issues*, in The International Journal of Robotics Research, 1982, Vol. 1 (1), pp. 4–17.
- [73] PUTERMAN, Martin L.: *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, John Wiley & Sons, New York, NY, USA, 1994, ISBN 978-0-471-61977-2.
- [74] NVIDIA: *Manager Configuration — Getting Started With Isaac Lab*, 2026,
URL <https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-lab/latest/train-your-second-robot-with-isaac-lab-03-implementing-mdp-with-managers.html> (Accessed on 2026-03-03).
- [75] THE ISAAC LAB PROJECT DEVELOPERS: *Task Design Workflows — Isaac Lab Documentation*, 2026,

-
- URL https://isaac-sim.github.io/IsaacLab/main/source/overview/core-concepts/task_workflows.html (Accessed on 2026-03-03).
- [76] THE ISAAC LAB PROJECT DEVELOPERS: *isaacsim.envs.mdp* — *Isaac Lab Documentation*, 2026,
URL <https://isaac-sim.github.io/IsaacLab/main/source/api/lab/isaacsim.envs.mdp.html> (Accessed on 2026-03-03).
- [77] LIN, Xingyu; WANG, Yufei; OLKIN, Jake; HELD, David: *SoftGym: Benchmarking Deep Reinforcement Learning for Deformable Object Manipulation*, in *Proceedings of the 2020 Conference on Robot Learning, Proceedings of Machine Learning Research*, Vol. 155, Vol. 155, pp. 432–448, PMLR.
- [78] KADI, Halid Abdulrahim; TERZIĆ, Kasim: *Data-Driven Robotic Manipulation of Cloth-like Deformable Objects: The Present, Challenges and Future Prospects*, in *Sensors*, 2023, Vol. 23 (5), pp. 2389.
- [79] SPAAN, Matthijs T. J.: *Partially Observable Markov Decision Processes*, in WIERING, Marco A.; VAN OTTERLO, Martijn (Eds.), *Reinforcement Learning: State-of-the-Art*, Springer, 2012, pDF draft distributed by the author; appears in the Springer volume.
URL <https://www.st.ewi.tudelft.nl/mtjspaans/pub/Spaan12pomdp.pdf>
- [80] THE ISAAC LAB PROJECT DEVELOPERS: *isaacsim.managers* — *Isaac Lab Documentation*, 2026,
URL <https://isaac-sim.github.io/IsaacLab/main/source/api/lab/isaacsim.managers.html> (Accessed on 2026-03-03).
- [81] THE ISAAC LAB PROJECT DEVELOPERS: *isaacsim.envs* — *Isaac Lab Documentation*, 2026,
URL <https://isaac-sim.github.io/IsaacLab/main/source/api/lab/isaacsim.envs.html> (Accessed on 2026-03-03).
- [82] BROCKMAN, Greg; CHEUNG, Vicki; PETTERSSON, Ludwig; SCHNEIDER, Jonas; SCHULMAN, John; TANG, Jie; ZAREMBA, Wojciech: *OpenAI Gym*, in arXiv preprint arXiv:1606.01540, 2016.
- [83] THE ISAAC LAB PROJECT DEVELOPERS: *Wrapping environments* — *Isaac Lab Documentation*, 2026,
URL https://isaac-sim.github.io/IsaacLab/main/source/how-to/wrap_rl_env.html (Accessed on 2026-03-03).

-
- [84] SEITA, Daniel; GANAPATHI, Aditya; HOQUE, Ryan; HWANG, Minho; CEN, Edward; TANWANI, Ajay Kumar; BALAKRISHNA, Ashwin; THANANJEYAN, Brijen; ICHNOWSKI, Jeffrey; JAMALI, Nawid; YAMANE, Katsu; IBA, Soshi; CANNY, John F.; GOLDBERG, Ken: *Deep Imitation Learning of Sequential Fabric Smoothing From an Algorithmic Supervisor*, in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 9651–9658, IEEE.
- [85] SICILIANO, Bruno; SCIAVICCO, Lorenzo; VILLANI, Luigi; ORIOLO, Giuseppe: *Robotics: Modelling, Planning and Control*, Springer London, 2009.
- [86] NVIDIA; MITTAL, Mayank; ROTH, Pascal; TIGUE, James; RICHARD, Antoine; ZHANG, Octi; DU, Peter; SERRANO-MUÑOZ, Antonio; YAO, Xinjie; ZURBRÜGG, René; RUDIN, Nikita; ET AL.: *Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning*, in arXiv preprint arXiv:2511.04831, 2025.
- [87] NG, Andrew Y.; HARADA, Daishi; RUSSELL, Stuart J.: *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*, in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML 1999)*, pp. 278–287, Morgan Kaufmann, San Francisco, CA, USA.
- [88] ASMUTH, John; LITTMAN, Michael L.; ZINKOV, Robert: *Potential-based Shaping in Model-based Reinforcement Learning*, in *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence (AAAI-08)*, pp. 604–609, AAAI Press.
- [89] BENGIO, Yoshua; LOURADOUR, Jérôme; COLLOBERT, Ronan; WESTON, Jason: *Curriculum Learning*, in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09)*, pp. 41–48, ACM Press, Montreal, QC, Canada.
- [90] THE ISAAC LAB PROJECT DEVELOPERS: *Training with an RL Agent — Isaac Lab Documentation*, 2026,
URL https://isaac-sim.github.io/IsaacLab/main/source/tutorials/03_envs/run_rl_training.html (Accessed on 2026-03-03).
- [91] THE ISAAC LAB PROJECT DEVELOPERS: *isaacsim_rl — Isaac Lab Documentation*, 2026,
URL https://isaac-sim.github.io/IsaacLab/main/source/api/lab_rl/isaacsim_rl.html (Accessed on 2026-03-03).
- [92] NVIDIA: *Getting Started with Cloner — Isaac Sim Documentation*, 2026,
URL https://docs.isaacsim.omniverse.nvidia.com/6.0.0/isaac_lab_tutorials/tutorial_cloner.html (Accessed on 2026-03-03).

-
- [93] SANCHEZ, Jose; CORRALES, Juan-Antonio; BOUZGARROU, Belhassen-Chedli; MEZOUAR, Youcef: *Robotic Manipulation and Sensing of Deformable Objects in Domestic and Industrial Applications: A Survey*, in *The International Journal of Robotics Research*, 2018.
- [94] GU, Feida; ZHOU, Yanmin; WANG, Zhipeng; JIANG, Shuo; HE, Bin: *A Survey on Robotic Manipulation of Deformable Objects: Recent Advances, Open Challenges and New Frontiers*, in *arXiv preprint*, 2023.
- [95] LONGHINI, Andrea; WELLE, Michael C.; ERICKSON, Zackory; KRAGIC, Danica: *A Review of Robotic Cloth Manipulation*, in *Annual Review of Control, Robotics, and Autonomous Systems*, 2025.
- [96] GARCIA-CAMACHO, Irene; LIPPI, Martina; WELLE, Michael C.; YIN, Hang; ANTONOVA, Rika; VARAVA, Anastasiia; BORRÀS, Julia; MARINO, Alessandro; ALENYÀ, Guillem; KRAGIC, Danica: *Benchmarking Bimanual Cloth Manipulation*, in *IEEE Robotics and Automation Letters*, 2020.
- [97] HA, Huy; SONG, Shuran: *FlingBot: The Unreasonable Effectiveness of Dynamic Manipulation for Cloth Unfolding*, in *Proceedings of the Conference on Robot Learning (CoRL)*.
- [98] CANBERK, Alper; CHI, Cheng; HA, Huy; BURCHFIELD, Benjamin; COUSINEAU, Eric; FENG, Siyuan; SONG, Shuran: *Cloth Funnels: Canonicalized-Alignment for Multi-Purpose Garment Manipulation*, in *arXiv preprint*, 2022.
- [99] LU, Haoran; WU, Ruihai; LI, Yitong; LI, Sijie; ZHU, Ziyu; NING, Chuanruo; SHEN, Yan; LUO, Longzan; CHEN, Yuanpei; DONG, Hao: *GarmentLab: A Unified Simulation and Benchmark for Garment Manipulation*, in *Advances in Neural Information Processing Systems*.
- [100] MAITIN-SHEPARD, Jeremy; CUSUMANO-TOWNER, Marco; LEI, Jiayuan; ABBEEL, Pieter: *Cloth Grasp Point Detection based on Multiple-View Geometric Cues with Application to Robotic Towel Folding*, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.
- [101] MILLER, Stephen; VAN DEN BERG, Jur; FRITZ, Mario; DARRELL, Trevor; GOLDBERG, Ken; ABBEEL, Pieter: *A Geometric Approach to Robotic Laundry Folding*, in *The International Journal of Robotics Research*, 2012, Vol. 31 (2), pp. 249–267.
- [102] LI, Yinxiao; YUE, Yonghao; XU, Danfei; GRINSFUD, Eitan; ALLEN, Peter K.: *Folding Deformable Objects using Predictive Simulation and Trajectory Optimization*, in

- [103] PETRÍK, Vladimír; KYRKI, Ville: *Feedback-based Fabric Strip Folding*, in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [104] HOQUE, Ryan; SEITA, Daniel; TANWANI, Ajay Kumar; JAMALI, Nawid; IBA, Soshi; CANNY, John; GOLDBERG, Ken: *VisuoSpatial Foresight for Multi-Step, Multi-Task Fabric Manipulation*, in *Proceedings of Robotics: Science and Systems (RSS)*.
- [105] LEE, Robert; WARD, Daniel; COSGUN, Akansel; DASAGI, Vibhavari; LEITNER, Jürgen; CORKE, Peter: *Learning Arbitrary-Goal Fabric Folding with One Hour of Real Robot Experience*, in *Proceedings of the Conference on Robot Learning (CoRL)*.
- [106] HU, Wenkang; TANG, Xincheng; E, Yanzhi; LI, Yitong; SHU, Zhengjie; LI, Wei; WANG, Huamin; YANG, Ruigang: *Real Garment Benchmark (RGBench): A Comprehensive Benchmark for Robotic Garment Manipulation featuring a High-Fidelity Scalable Simulator*, in arXiv preprint, 2025.
- [107] DE GUSSEME, Victor-Louis; LIPS, Thomas; PROESMANS, Remko; HIETALA, Julius; LEE, Giwan; CHOI, Jiyoung; ET AL.: *A Dataset and Benchmark for Robotic Cloth Unfolding Grasp Selection: The ICRA 2024 Cloth Competition*, in *The International Journal of Robotics Research*, 2026.
- [108] BLANCO-MULERO, David; ET AL.: *Benchmarking the Sim-to-Real Gap in Cloth Manipulation*, in *IEEE Robotics and Automation Letters*, 2024.
- [109] TOBIN, Josh; FONG, Rachel; RAY, Alex; SCHNEIDER, Jonas; ZAREMBA, Wojciech; ABBEEL, Pieter: *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*, in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [110] BOUSMALIS, Konstantinos; IRPAN, Alex; WOHLHART, Paul; BAI, Yunfei; KELCEY, Matthew; KALAKRISHNAN, Mrinal; DOWNS, Laura; IBARZ, Julian; PASTOR, Peter; KONOLIGE, Kurt; LEVINE, Sergey; VANHOUCKE, Vincent: *Using Simulation and Domain Adaptation to Improve Efficiency of Deep Robotic Grasping*, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.
- [111] GANAPATHI, Aditya; SUNDARESAN, Pranav; THANANJEYAN, Brijen; BALAKRISHNA, Ashwin; SEITA, Daniel; ICHNOWSKI, Jeffrey; CANNY, John; GOLDBERG, Ken: *Learning Dense Visual Correspondences in Simulation to Smooth and Fold*

Real Fabrics, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.

- [112] RUSU, Andrei A.; COLMENAREJO, Sergio Gomez; GÜLÇEHRE, Çağlar; DESJARDINS, Guillaume; KIRKPATRICK, James; PASCANU, Razvan; MNIH, Volodymyr; KAVUKCUOGLU, Koray; HADSELL, Raia: *Policy Distillation*, in arXiv preprint, 2015.
- [113] WANG, Yufei; SUN, Zhanyi; ERICKSON, Zackory; HELD, David: *One Policy to Dress Them All: Learning to Dress People with Diverse Poses and Garments*, in *Proceedings of Robotics: Science and Systems (RSS)*.
- [114] DOUMANOGLU, Andreas; STRIA, Jan; PELEKA, Georgia; MARIOLIS, Ioannis; PETRÍK, Vladimír; KARGAKOS, Andreas; WAGNER, Libor; HLAVÁČ, Václav; KIM, Tae-Kyun; MALASSIOTIS, Sotiris: *Folding Clothes Autonomously: A Complete Pipeline*, in *IEEE Transactions on Robotics*, 2016.
- [115] PIXAR ANIMATION STUDIOS: *USD Frequently Asked Questions*, 2021, openUSD documentation (release 26.03); © 2021 Pixar Animation Studios.,
URL <https://openusd.org/release/usdfaq.html> (Accessed on 2026-03-03).
- [116] PIXAR ANIMATION STUDIOS: *UsdLux: USD Lighting Schema*, 2021, openUSD API documentation; generated on 2021-07-15.,
URL https://openusd.org/docs/api/usd_lux_page_front.html (Accessed on 2026-03-03).
- [117] NVIDIA: *Physx Joint Schema — Omni Physics Documentation*, 2026, omni Physics documentation; last updated on Jan 20, 2026.,
URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/joints/physx_joint_schema.html (Accessed on 2026-03-03).
- [118] OPEN ROBOTICS: *URDF (Unified Robot Description Format) — ROS 2 Documentation (Rolling)*, 2026, rOS 2 Rolling documentation (accessed 2026-03-03).,
URL <https://docs.ros.org/en/rolling/Tutorials/Intermediate/URDF/URDF-Main.html> (Accessed on 2026-03-03).
- [119] OPEN ROBOTICS: *Adding Physical and Collision Properties to a URDF Model — ROS 2 Documentation (Foxy)*, 2026, rOS 2 Foxy (EOL) tutorial page (accessed 2026-03-03).,
URL <https://docs.ros.org/en/foxy/Tutorials/Intermediate/URDF/Adding-Physical-and-Collision-Properties-to-a-URDF-Model.html> (Accessed on 2026-03-03).

-
- [120] THE HARDWARE_INTERFACE CONTRIBUTORS: *Joint Kinematics for ros2_control*, 2026, package documentation describing mimic joints and limitations of serial-chain URDF kinematics (accessed 2026-03-03)., URL https://docs.ros.org/en/rolling/p/hardware_interface/doc/joints_userdoc.html (Accessed on 2026-03-03).
- [121] DEEPMIND TECHNOLOGIES LIMITED: *MuJoCo Documentation: Modeling*, 2026, muJoCo documentation; describes MJCF design and URDF vs. MJCF scope (accessed 2026-03-03)., URL <https://mujoco.readthedocs.io/en/stable/modeling.html> (Accessed on 2026-03-03).
- [122] DEEPMIND TECHNOLOGIES LIMITED: *MuJoCo Documentation: Overview*, 2026, muJoCo documentation (version 2.3.7); includes tendon geometry and coupling description (accessed 2026-03-03)., URL <https://mujoco.readthedocs.io/en/2.3.7/overview.html> (Accessed on 2026-03-03).
- [123] THE ISAAC LAB PROJECT DEVELOPERS: *Importing a New Asset*, 2026, isaac Lab documentation; last updated on Mar 03, 2026., URL https://isaac-sim.github.io/IsaacLab/main/source/how-to/import_new_asset.html (Accessed on 2026-03-03).
- [124] NVIDIA: *Tutorial: Import URDF — Isaac Sim Documentation (5.1.0)*, 2026, isaac Sim documentation; last updated on Jan 28, 2026., URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/importer_exporter/import_urdf.html (Accessed on 2026-03-03).
- [125] NVIDIA: *Robot Assembler — Isaac Sim Documentation (5.1.0)*, 2026, isaac Sim documentation; last updated on Jan 28, 2026., URL https://docs.isaacsim.omniverse.nvidia.com/5.1.0/robot_setup/assemble_robots.html (Accessed on 2026-03-03).
- [126] NVIDIA: *Articulations — PhysX SDK Documentation (5.5.0)*, 2024, physX documentation page including articulation tendons (fixed/spatial); dated Dec 04, 2024., URL <https://nvidia-omniverse.github.io/PhysX/physx/5.5.0/docs/Articulations.html> (Accessed on 2026-03-03).
- [127] NVIDIA: *Known Physics Limitations — Omni Physics Documentation*, 2026, omni Physics documentation; last updated on Jan 20, 2026.,

URL https://docs.omniverse.nvidia.com/kit/docs/omni_physics/latest/dev_guide/guides/current_limitations.html (Accessed on 2026-03-03).

- [128] NVIDIA: *Isaac Sim Documentation*, 2023, nVIDIA Omniverse Isaac Sim documentation,

URL <https://docs.omniverse.nvidia.com/isaacsim/latest/index.html> (Accessed on 2026-03-03).

- [129] ROBOTIQ INC.: *2F-85 and 2F-140 Adaptive Gripper Instruction Manual – Specifications*, 2019, product datasheet,

URL <https://robotiq.com/products/adaptive-grippers> .

Source Directory